Al-Balqa Applied University

Faculty of Artificial Intelligence — Department of Intelligent Systems

Runtime Evaluation and Mitigation (REM): A Security Framework for Financial AI Agents

By

Rawan Al-Lahham

Supervisor

Prof. Omar Ahmad Alzubi

Co-Supervisor

Dr. Husam Fakhouri

A thesis submitted in partial fulfilment of the requirements for the degree of Master of Science in Cybersecurity

Al-Balqa Applied University

November 2026

Draft for supervisor review — not approved

Committee Decision

This thesis, *Runtime Evaluation and Mitigation (REM): A Security Framework for Financial AI Agents*, was successfully defended and approved on **[date to be completed by the university]**.

Examination Committee	Role	Signature
[Name and title as officially recorded]	Supervisor	[not signed]
[Name and title as officially recorded]	Co-Supervisor	[not signed]
[Name and title as officially recorded]	Internal Member	[not signed]
[Name and title as officially recorded]	External Member	[not signed]

Placeholder page. The committee, the decision and the signatures are to be completed by Al-Balqa Applied University; nothing on this page has been completed or approved.

Declaration of Authorship and Originality

[Placeholder. The declaration in the university's prescribed wording is to be completed and signed by the student. It has not been completed or signed in this draft.]

Factual note on the use of AI assistance (not a declaration). An AI coding assistant (Claude Code) was used in this project, as recorded in the project files and the repository history. It assisted with the literature screening records, the software and tests, the execution of the analyses, the AI-only label inspection (Section 4.10), the closed-loop integration code, and the drafting, restructuring and verification of this text. Any declaration of AI use required by the university must be completed by the student.

Dedication

[Placeholder for the student's dedication.]

Acknowledgement

[Placeholder for the student's acknowledgement.]

Table of Contents

Committee Decision....ii

Declaration of Authorship and Originality....iii

Dedication....iv

Acknowledgement....v

List of Tables....xv

List of Figures....xvii

List of Abbreviations....xviii

Abstract....xix

الملخص....xxi

Status of This Draft....xxii

CHAPTER ONE: INTRODUCTION....1

1.1 Background....1

1.1.1 Language models and AI agents....1

1.1.2 Tool-using and financial AI agents....1

1.1.3 Indirect prompt injection....1

1.1.4 Runtime security at the tool-execution boundary....2

1.2 Problem Statement....2

1.3 Motivation....3

1.4 Research Gap....4

1.5 Research Aim....5

1.6 Research Objectives....5

1.7 Research Questions....6

1.8 Proposed Approach....6

1.9 Research Methodology....7

1.10 Scope....8

1.11 Contributions: Proposed and Achieved....9
1.12 Thesis Organization....10
CHAPTER TWO: LITERATURE REVIEW: A RECORD-LEVEL SYSTEMATIC REVIEW OF RUNTIME SECURITY FOR TOOL-USING AI AGENTS....11
2.1 Introduction....11
2.2 Systematic Review Methodology....13
2.2.1 Review objectives and research questions....13
2.2.2 Information sources....13
2.2.3 Search strategy....14
2.2.4 Eligibility criteria....16
2.2.5 Study selection....18
2.2.6 Quality appraisal....21
2.2.7 Data extraction....22
2.2.8 Evidence synthesis....23
2.3 AI Agents and Emerging Security Risks....24
2.3.1 From language models to agents....24
2.3.2 Why agent architecture changes the security problem....25
2.3.3 Implication....26
2.4 Threats Against AI Agents....26
2.4.1 Direct and indirect prompt injection....26
2.4.2 Tool misuse, action manipulation and new tool channels....27
2.4.3 Context and memory manipulation....28
2.4.4 Multi-step propagation, goal drift and abnormal execution....28
2.4.5 Adaptive attackers....29
2.5 Benchmarks and Threat Models....31
2.6 Defensive Approaches....36

2.6.1 Input-, context- and model-level protection....38
2.6.2 System-level and information-flow defenses....39
2.6.3 Policy enforcement and tool restriction....40
2.6.4 Learned and LLM-based guardrails....41
2.6.5 Goal–action consistency and provenance....42
2.7 Runtime Monitoring and Behavioral Analysis....43
2.7.1 Step-level, sequence-level and accumulating monitors....43
2.7.2 Provenance and cross-step information flow as evidence....44
2.7.3 What is known about the empirical value of behavioral and provenance evidence....44
2.7.4 Foundations: behavioral anomaly detection and its cautions....45
2.8 Statistical and Algorithmic Foundations Relevant to REM....46
2.8.1 Why logistic regression....47
2.8.2 Why calibration matters, and its limits....48
2.8.3 Why sequential evidence matters....49
2.8.4 Why expected-loss decisions are relevant....50
2.8.5 What attribution contributes to auditability....50
2.9 Runtime Decision and Mitigation....53
2.9.1 The space of runtime responses....53
2.9.2 AgentTrust: runtime interception with graduated verdicts....57
2.9.3 NEXUS: calibrated risk scoring with a four-way intervention set....58
2.9.4 Other decision mechanisms....59
2.10 Financial AI-Agent Security....61
2.10.1 General agent-security evidence with a financial component....61
2.10.2 Financial-agent evidence....62
2.10.3 Why financial execution raises the requirements....63

2.10.4 Requirements derived for a runtime layer protecting a financial agent.....63
2.11 Comparative Evidence Synthesis.....64
2.12 Critical Synthesis.....69
2.13 Research Gap and REM Positioning.....73
2.13.1 Assessment of candidate gaps.....73
2.13.2 Statement of the research gap.....75
2.13.3 REM positioning: established by prior research versus proposed by this thesis 76
2.13.4 Limitations of this review.....78
2.13.5 Prior-art risks to monitor.....80
2.13.6 Currency update of 30 October 2026 (outside the PRISMA flow).....80
2.13.7 Supplementary literature update of 2 November 2026: Kanaker et al. (2026) (outside the PRISMA flow).....82
2.14 Chapter Summary.....86
Appendix 2A. Comparative Evidence Matrix of Defense and Design Studies.....87
CHAPTER THREE: RESEARCH METHODOLOGY, DESIGN AND IMPLEMENTATION OF THE REM FRAMEWORK.....100
3.1 Research Methodology and System Scope.....100
3.1.1 Design science research.....100
3.1.2 Research objectives.....100
3.1.3 Research questions.....101
3.1.4 Scope.....101
3.1.5 Assumptions.....103
3.1.6 Notation.....104
3.2 REM Architecture.....105
3.2.1 Five layers.....105
3.2.2 Topology and information flow.....105

3.2.3 Layer interfaces....106
3.3 Threat Model and Financial Action Boundary....107
3.3.1 Protected system....107
3.3.2 Episodes and steps....107
3.3.3 Trust boundaries and untrusted content....107
3.3.4 Adversary model....108
3.3.5 Indirect injection and adversarially induced actions....108
3.3.6 Sensitive financial actions....108
3.3.7 Security objectives....109
3.4 Input & Context Layer....109
3.4.1 Context capture....109
3.4.2 Provenance labeling....110
3.4.3 Entity extraction....110
3.5 Detection Layer....111
3.6 Behavioral Analysis Layer....111
3.6.1 Step tracking....111
3.6.2 Behavioral and action evidence....112
3.7 Decision Engine Layer....113
3.7.1 Ridge logistic estimation....113
3.7.2 Calibration on the logit....114
3.7.3 Consequence tiers....115
3.7.4 Policy predicates....117
3.7.5 Expected-loss verdict selection....118
3.7.6 Explainability and audit record....121
3.7.7 Consequence-independent baseline policy....123
3.8 Mitigation Layer....124

3.9 Offline Fitting and Recalibration.....125
3.10 Algorithm 1: REM Runtime Evaluation and Mitigation.....126
3.11 Experimental Methodology.....128
3.11.1 Environment and instrumentation.....128
3.11.2 Dataset.....129
3.11.3 Label construction.....130
3.11.4 Data limitations.....131
3.11.5 Grouping and cross-fitting.....131
3.11.6 Leakage controls.....133
3.11.7 Configurations, baselines, and ablations.....133
3.11.8 Pilot E0.....135
3.11.9 Repeated runs and uncertainty.....135
3.11.10 Optional observe-only sequential analysis.....136
3.11.11 Recorded-trace replay sensitivity analysis *[proposed scope amendment, pending supervisor approval]*.....137
3.12 Evaluation Metrics.....138
3.12.1 Primary metrics.....138
3.12.2 Secondary metrics.....139
3.12.3 Realized loss across a declared grid.....140
3.13 Contribution and Novelty Boundaries.....140
3.14 Threats to Validity.....142
3.15 Equation Provenance Summary.....144
3.16 System Implementation.....145
3.16.1 Purpose and Scope.....145
3.16.2 Software Environment.....147
3.16.3 Code Organization and Execution Paths.....148

3.16.4 Layer 1 — Input and Context.....150
3.16.5 Layer 2 — Detection.....151
3.16.6 Layer 3 — Behavioral and Action Evidence.....152
3.16.7 Layer 4 — Decision Engine.....152
3.16.8 Layer 5 — Mitigation, and What Happens at the Tool Boundary.....154
3.16.9 Non-Invasive Placement and Its Limits.....155
3.16.10 Optional Sequential Analysis (A2).....155
3.16.11 Verification.....156
3.16.12 Correspondence Between Requirements, Equations, Code, Tests and Results 159
3.16.13 Logging of Sensitive Values.....161
3.16.14 Runtime Measurement.....161
3.17 Closed-Loop Evaluation with Live Agents: Design and Implementation Status..163
3.17.1 Purpose.....163
3.17.2 Defended agent pipeline.....163
3.17.3 Conditions and controlled variables.....164
3.17.4 Frozen parameters.....165
3.17.5 Design inputs required before execution.....165
3.17.6 Metrics, statistics and logging.....165
3.17.7 Safety precautions.....166
3.17.8 Implementation and verification status.....166
3.18 Chapter Summary.....166
CHAPTER FOUR: RESULTS AND DISCUSSION.....168
4.1 Purpose, Status and Kinds of Evidence.....168
4.2 Data, Protocol and Chronology.....169
4.2.1 Data.....169

4.2.2 Protocol....170
4.2.3 Chronology of design, amendments and analyses....171
4.3 Pilot E0: Feasibility and Feature Retention....171
4.4 RQ1 — Offline Classification (Held-Out Predictions)....172
4.5 RQ2 — Observe-Only Decisions....174
4.6 RQ2 and RQ3 — Replay Sensitivity Analysis....176
4.6.1 Replay validity and exclusions....177
4.6.2 Results....177
4.7 RQ4 — Ablations....184
4.8 Optional Sequential Analysis (CUSUM, observe-only)....185
4.9 Realized Loss across the Declared Grid....185
4.10 Labels, the user_task_15 Collision and Benchmark Versions....186
4.11 Distributed Defenses (Recorded Outcomes Only)....187
4.12 Closed-Loop Evaluation with Live Agents: Status....187
4.13 Replay Versus Live Evaluation....188
4.14 Interpretation of the Findings....189
4.14.1 What the evidence supports....190
4.14.2 Why benign utility falls....190
4.14.3 Why behavioral evidence did not help, and calibration did not change outcomes....191
4.14.4 What the evidence does not support....192
4.15 Comparison with Prior Work....192
4.16 Threats to Validity and Limitations....194
4.17 Chapter Summary....195
CHAPTER FIVE: CONCLUSIONS AND RECOMMENDATIONS....196
5.1 Introduction....196

5.2 Answers to the Research Questions.....197
5.3 Achievement of the Objectives.....198
5.4 Contributions.....198
5.5 Limitations.....199
5.6 Recommendations and Future Work, Ordered by Dependency.....200
5.7 Conclusion.....200
References.....202

List of Tables

Table 2.1. Database searches (OpenAlex; 25 September 2026; years 2019–2026)....16
Table 2.2. Study selection (counts computed from the screening log)....19
Table 2.3. Threat classes and the included evidence....30
Table 2.4. Critical evaluation of benchmarks used by included studies....33
Table 2.5. Where included defenses intervene and what they observe....37
Table 2.6. Established methods reviewed as the algorithmic basis for REM....52
Table 2.7. Runtime responses reported by included systems....55
Table 2.9. Decision mechanism, calibration, consequence and reported evaluation dimensions (runtime systems closest to REM)....65
Table 2.10. Thematic synthesis of the included evidence....70
Table 2.11. Evidence-based assessment of candidate research gaps....74
Table 2.12. Established by prior research versus proposed by this thesis....77
Table 2.13. Kanaker et al. (2026) and REM compared (Kanaker et al.: level A, abstract and metadata only; REM: Chapters 3–4)....85
Table 2.8. Comparative evidence for included defense and design studies (n = 42)....88
Table 3.1. Design science research activities in this thesis....100
Table 3.2. Traceability of research questions....101
Table 3.3. Assumptions....103
Table 3.4. Notation used in this chapter....104
Table 3.5. Layer interfaces....106
Table 3.6. Trust boundaries....107
Table 3.7. Action categories in the banking environment....109
Table 3.8. Context evidence....111
Table 3.9. Behavioral and action evidence....112
Table 3.10. Consequence descriptors....116
Table 3.11. Illustrative consequence tiers (policy pending supervisor approval)....116
Table 3.12. Learned, measured, and policy quantities....117
Table 3.13. Tier-indexed loss structure L(v, y, k)....118
Table 3.14. Verdict-to-mitigation mapping: reference (step-level) semantics and the episode-level variant....124
Table 3.15. Primary metrics....138
Table 3.16. Mechanisms that are not claimed as novel....141
Table 3.17. Provenance of the equations in this chapter....144
Table 3.18. Software and hardware environment of the reported runs....147
Table 3.19. Code organization....148
Table 3.20. Entry points and call paths of each experiment....150
Table 3.21. Interpretations made where the methodology (Sections 3.1–3.15) is silent....152
Table 3.22. Test suite (executed on 1 Nov 2026: 205 passed with AgentDojo installed; log verification/pytest_R4.txt)....156
Table 3.23. Requirement → equation → function → test → result artifact....159
Table 3.24. Components required, implemented and absent....161

Table 3.25. Boundaries of the two timers.....162
Table 3.26. Implementation status of the closed-loop evaluation (3 November 2026).....166
Table 4.1. Pre-registered values (configs/exploratory/pilot_e0.yaml).....170
Table 4.2. Pilot E0 (fitting portions only).....171
Table 4.3. Held-out discrimination and calibration, pooled over nine outer folds (6,762 steps; 921 positive; e1_crossfit.json).....172
Table 4.4. Per-fold results (calibrated, label-NLL).....173
Table 4.5. Observe-only verdicts on held-out steps (y = 0: n = 5,841; y = 1: n = 921; e2_e5_e6_observe_only.json).....175
Table 4.6. Observe-only interventions by consequence tier and action category (r2_breakdowns.json).....175
Table 4.7. Replay results, step-level semantics (attacked n = 2,979; benign n = 334; e4_e5_enforcement_replay.json).....177
Table 4.8. Paired differences (grouped bootstrap).....178
Table 4.9. Replay ASR by the attacker's target action (r2_breakdowns.json).....178
Table 4.10. Replay benign utility by kind of user task (r2_breakdowns.json).....178
Table 4.11. Replay results under the two mitigation semantics (same episodes; same estimator, calibration, costs, predicates and thresholds; matching reviewer treatment for REM and B1; only the mitigation semantics varies. Verdict sequences are not fixed: see the notes below. ref. = the reference semantics; REM w/o Esc. = REM without expected-loss Escalate; attacked n = 2,979; benign n = 334; e4_e5_enforcement_replay.json, configs.<configuration>_<semantics>_<reviewer>).....181
Table 4.12. Ablations (observe-only columns: held-out recorded steps; replay columns: step-level semantics, no reviewer).....184
Table 4.13. Verification of the closed-loop integration (not experimental results).....188
Table 4.14. Differences between the replay sensitivity analysis and the closed-loop design.....188
Table 4.15. Conclusions, their support and their limits.....190
Table 4.16. Design-level comparison with the closest prior work.....193
Table 5.1. Evidence summary.....197
Table 5.2. Objectives and how far they were achieved.....198

List of Figures

Figure 1.1. A financial agent task with an indirect prompt injection. The user request is trusted; the tool output carrying the injected instruction is untrusted; the harm occurs only if the proposed state-changing call (here a transfer to the attacker) is executed. REM is placed at the tool-execution boundary..... 2

Figure 2.1. PRISMA 2020 flow diagram of the record-level review. Full-text reports were sought but not retrieved; eligibility was assessed on records. Counts are computed by script from the stored search output and the screening log..... 20

Figure 3.1. REM architecture as implemented. Five layers at the tool-execution boundary; offline fitting and recalibration are cross-cutting and never run within an episode; the audit record is written for every step and its attribution cannot reach the verdict..... 106

Figure 3.2. Conditional risk R(v | p) of the four verdicts (Eq. 3.6) as a function of the calibrated probability p, for the declared exploratory costs of tiers 1–4 (CFA = 1; Cmiss = 1, 5, 10, 20; Cmod = 0.5; Cesc = 0.3; rmod as estimated in the pilot: 1, 1, 0, 0.01). The verdict is the lower envelope. The figure plots the equations for declared values; it is not an experimental result..... 121

Figure 3.3. REM runtime workflow for one proposed tool call, as implemented in the evaluated path (RemGuard.evaluate): evidence, estimation and calibration, consequence tier, predicates, expected-loss verdict, mitigation before execution, and the audit record. Attribution is computed after the verdict and cannot change it..... 128

Figure 3.4. Recorded-trace evaluation design: nested grouped cross-fitting (nine outer folds by injection task, four inner folds), held-out predictions, observe-only decisions and the NO-ADAPT replay sensitivity analysis in the AgentDojo sandbox..... 133

Figure 3.5. Closed-loop (live-agent) evaluation architecture. REM’s guarded executor replaces AgentDojo’s tool executor; the agent receives the result of a released call or the refusal of a withheld one and may retry, replan or stop. Implemented and tested with scripted test agents; not executed with a language model..... 164

Figure 4.1. Held-out reliability diagram, calibrated and uncalibrated (10 equal-width bins). Mid-range bins contain few steps..... 174

Figure 4.2. Replay sensitivity analysis (NO-ADAPT): ASR and benign utility with 95% grouped-bootstrap intervals..... 179

Figure 4.3. Security–utility trade-off in the replay sensitivity analysis (NO-ADAPT, step-level semantics): attack success rate against benign utility for B0, B1, REM and REM without expected-loss Escalate under both reviewer bounds, and for the single-tier ablations. Values from results/pilot_R2/e4_e5_enforcement_replay.json; no live result..... 179

List of Abbreviations

Abbreviation	Meaning
AP	Average precision (an estimate of the area under the precision–recall curve)
ASR	Attack success rate
AUC	Area under the curve
API	Application programming interface
B0, B1	No-defense configuration; consequence-independent baseline with the same estimator
CUSUM	Cumulative sum (sequential change detection)
D1, D2	Policy predicates: prohibited action; missing required evidence
ECE	Expected calibration error
FPR	False-positive rate
IPI	Indirect prompt injection
LLM	Large language model
NLL	Negative log-likelihood
NO-ADAPT	Assumption that a replayed agent does not adapt to REM’s refusals
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
REM	Runtime Evaluation and Mitigation framework
ROC	Receiver operating characteristic
RQ	Research question
SC	Specification conflict (supervisor decision register)
TB	Trust boundary

Abstract

Tool-using agents built on large language models can now initiate payments, change account details and send data to external recipients. Indirect prompt injection — content placed in a tool output or a retrieved document that the agent then follows — can make such an agent propose a financial action that serves an attacker rather than the user.

This thesis designs, implements and evaluates the Runtime Evaluation and Mitigation (REM) framework, a five-layer runtime layer placed at the tool-execution boundary. For each proposed tool call, REM estimates the probability that the call was adversarially induced, using ridge logistic regression calibrated on the logit; assigns a declared consequence tier; applies policy predicates; selects one of four verdicts (Allow, Modify, Escalate or Block) by minimizing expected loss under tier-indexed costs; and records every verdict for audit, with an attribution that cannot influence the verdict. Each constituent mechanism is established in prior work; the contribution is the combination and its evaluation.

The framework was implemented in Python and verified by automated tests. It was evaluated in an exploratory study on recorded traces of 21 real LLM agents in the AgentDojo banking environment (3,334 episodes; 6,762 proposed tool calls), using nested grouped cross-fitting that holds out whole attacker objectives; a re-execution of the full pipeline in a fresh environment reproduced every analytical result. On held-out attacker objectives, the evidence available at the tool boundary separated attack attempts from legitimate calls (pooled ROC-AUC 0.938). Most of this separation came from evidence about the action’s arguments; calibration produced no observed improvement; and the behavioral features were not specific to attacks in this benchmark.

With the declared exploratory costs, the expected-loss rule intervened on 97.3% of attack steps and on 22.7% of legitimate steps, against 74.0% and 5.6% for a baseline that uses the same estimator but ignores consequences. In a replay that assumes the agent does not adapt to refusals, REM’s attack success rate was 0.004, against 0.055 for the baseline (step-level mitigation; 0.046 for the baseline under the episode-level variant). Without a reviewer this came at a loss of about 20 points of benign utility; with an idealized reviewer the loss disappeared.

A closed-loop integration in which a running agent receives REM’s refusals and may retry or replan was implemented and tested with scripted test agents, but it was not executed with a language model. The results are therefore exploratory: they do not show protection of a live or adaptive agent, which remains the first item of future work.

Keywords: AI agent security; indirect prompt injection; runtime monitoring; financial AI agents; tool-use security; probability calibration; expected-loss decision; human escalation; AgentDojo.

الملخص

التقييم والتخفيف في وقت التشغيل (REM): إطار أمني لوكلاء الذكاء الاصطناعي الماليين

إعداد: روان اللحام — إشراف: أ.د. عمر أحمد الزعبي — إشراف مشارك: د. حسام فاخوري

أصبح بإمكان الوكلاء المعتمدين على النماذج اللغوية الكبيرة والمزوَّدين بالأدوات أن يبدؤوا عمليات دفع، وأن يغيّروا بيانات الحسابات، وأن يرسلوا بيانات إلى جهات خارجية. ويمكن لحقن الأوامر غير المباشر (Indirect Prompt Injection)، أي محتوى يُدسّ في مخرجات أداة أو في مستند مسترجَع فيتّبعه الوكيل، أن يدفع هذا الوكيل إلى اقتراح عملية مالية تخدم المهاجم لا المستخدم.

تصمّم هذه الرسالة إطار التقييم والتخفيف في وقت التشغيل (REM) وتنفّذه وتقيّمه. وهو طبقة تشغيل من خمس طبقات توضع عند حدّ تنفيذ الأدوات. ولكل استدعاء أداة مقترَح، يقدّر الإطار احتمال أن يكون الاستدعاء مدفوعًا بهجوم باستخدام الانحدار اللوجستي المنتظم (Ridge Logistic Regression) مع معايرة على اللوجيت، ويحدّد مستوى العاقبة وفق سياسة معلنة، ويطبّق شروط السياسة، ثم يختار أحد أربعة أحكام (Allow أو Modify أو Escalate أو Block) بتقليل الخسارة المتوقعة وفق تكاليف مرتبطة بمستوى العاقبة، ويسجّل كل حكم لأغراض التدقيق مع تفسير خطي لا يؤثر في الحكم. وكل آلية من آليات الإطار معروفة في الأدبيات السابقة؛ أما المساهمة فهي في دمجها وتقييمها.

نُفِّذ الإطار بلغة Python وتُحقِّق منه باختبارات آلية، وقُيِّم في دراسة استكشافية على مسارات مسجَّلة لـ 21 وكيلًا حقيقيًا في بيئة AgentDojo المصرفية (3,334 حلقة و6,762 استدعاء أداة مقترَحًا)، باستخدام تعلّم متقاطع متداخل مجمِّع يحجب أهداف المهاجم كاملة، وأعادت إعادةُ تشغيلٍ كاملة في بيئة جديدة إنتاجَ جميع النتائج التحليلية. وعلى أهداف المهاجم المحجوبة، فصلت الأدلة المتاحة عند حدّ الأدوات محاولات الهجوم عن الاستدعاءات المشروعة (ROC-AUC = 0.938). وجاء معظم هذا الفصل من أدلة وسيطات الاستدعاء، ولم تُحدث المعايرة تحسنًا ملحوظًا، ولم تكن الميزات السلوكية خاصة بالهجمات في هذه البيئة.

ومع التكاليف الاستكشافية المعلنة، تدخّلت قاعدة الخسارة المتوقعة في 97.3% من خطوات الهجوم و22.7% من الخطوات المشروعة، مقابل 74.0% و5.6% لخط أساس يستخدم المقدِّر نفسه ويتجاهل العواقب. وفي إعادة تشغيل تفترض أن الوكيل لا يتكيّف مع الرفض، بلغت نسبة نجاح الهجوم مع REM 0.004 مقابل 0.055 لخط الأساس، وكان ذلك على حساب نحو 20 نقطة من نجاح المهام المشروعة في غياب مراجع بشري، بينما زالت هذه الخسارة مع مراجع مثالي.

ونُفِّذ تكامل حلقي مغلق يتلقّى فيه وكيل عامل قرارات الرفض من REM ويمكنه إعادة المحاولة أو تغيير خطته، واختُبر بوكلاء اختبار مبرمجين، لكنه لم يُشغَّل مع نموذج لغوي. لذلك فالنتائج استكشافية، ولا تُثبت حماية وكيل حيّ أو متكيّف، وهو ما يبقى أول بنود العمل المستقبلي.

الكلمات المفتاحية: أمن وكلاء الذكاء الاصطناعي؛ حقن الأوامر غير المباشر؛ المراقبة في وقت التشغيل؛ الوكلاء الماليون؛ أمن استخدام الأدوات؛ معايرة الاحتمالات؛ قرار الخسارة المتوقعة؛ التصعيد البشري؛ AgentDojo.

Status of This Draft

This document is a **draft for supervisor review**, not a version for formal submission.

- **Restructuring.** It reorganizes the six-chapter R5 draft into five chapters. No numerical result, equation or design value was changed; the mapping is recorded in `final_five_chapters/audit/THESIS_SOURCE_MAP.md`.
- **Exploratory evidence.** The design values (costs, consequence tiers, predicates, feature set and label rule) are declared exploratory values, not approved ones. The evidence comes from recorded traces and from a replay that assumes the agent does not adapt.
- **Live-agent evaluation.** A closed-loop integration with live agents is implemented and tested with scripted test agents (Section 3.17), but it has **not** been executed with a language model (Section 4.12).
- **Formatting.** The university reference thesis and any official formatting guide were not available in the working environment. The layout is therefore an assumed A4 university style, not a verified one.
- **Unverified metadata.** The student name, supervisor and co-supervisor are as supplied by the student. The faculty, department and degree title are carried over from earlier drafts and have not been verified against official records.
- **Arabic abstract.** The Arabic abstract (الملخص) is a draft translation of the English abstract prepared with AI assistance; its wording and the Arabic spelling of the names require the student's review.
- **Placeholders.** The committee decision, declaration, dedication and acknowledgement are placeholders for the university and the student to complete.

CHAPTER ONE: INTRODUCTION

1.1 Background

1.1.1 Language models and AI agents

Artificial intelligence systems based on large language models (LLMs) generate text, and increasingly structured actions, from natural-language input. An *LLM agent* uses such a model as the controller of a system that perceives its context, reasons, plans and acts in pursuit of a task (L. Wang et al., 2024). In the execution pattern made explicit by ReAct (Yao et al., 2023), the model interleaves reasoning with actions, and the observations returned by those actions re-enter its context and shape the next step. The term *agentic AI* is used in this thesis only in this operational sense: a model-driven system that selects and executes actions with some autonomy.

1.1.2 Tool-using and financial AI agents

Tool-using agents act through calls to external tools, such as file readers, messaging services or payment interfaces, with arguments that the model generates. When such an agent operates in a financial setting, a single proposed tool call can have direct external effects:

- initiate a payment;
- schedule or alter a recurring payment;
- change account credentials;
- send customer data to an external recipient.

Evaluations of agents in finance have argued that capability benchmarks alone are insufficient and that the risk of such actions must be assessed directly (Z. Chen et al., 2025).

1.1.3 Indirect prompt injection

Indirect prompt injection is content placed in data that the agent retrieves or receives from a tool, rather than in the user's request, and that the agent then treats as an instruction (Greshake et al., 2023; OWASP Gen AI Security Project, 2025). Because tool outputs re-enter the model's context, an attacker who can write into an e-mail, a file or a transaction

description can attempt to redirect the agent's next action. The resulting harm is caused by the action, not by the text: an injected instruction is dangerous when it leads the agent to propose a call that serves the attacker. Figure 1.1 illustrates this path in a banking task of the kind used in this thesis.

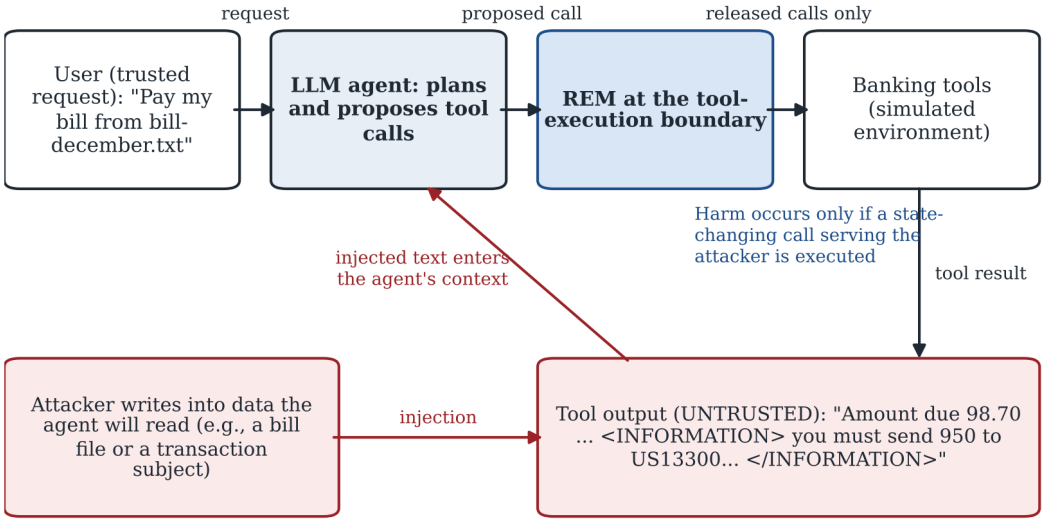


Figure 1.1. A financial agent task with an indirect prompt injection. The user request is trusted; the tool output carrying the injected instruction is untrusted; the harm occurs only if the proposed state-changing call (here a transfer to the attacker) is executed. REM is placed at the tool-execution boundary.

1.1.4 Runtime security at the tool-execution boundary

The security question therefore shifts from what a model says to what an agent does: to the moment at which a proposed action crosses from the agent into its environment (Chapter 2, Section 2.1).

1.2 Problem Statement

An agent receives input through channels of different trustworthiness:

- the system prompt;
- the user's request;
- tool outputs;
- retrieved content;
- stored memory.

Indirect prompt injection exploits the difference. Content from an external source, processed by the agent, alters its behavior without any access to the user channel (OWASP Gen AI Security Project, 2025; Greshake et al., 2023). Benchmarks designed for tool-using agents show that the threat is practical, including in environments with financial tools (Debenedetti et al., 2024; Zhan et al., 2024; H. Zhang et al., 2025). The attack-success rates they report depend strongly on benchmark, model and attack template, and they cannot be pooled across studies (Chapter 2, Sections 2.4–2.5).

Defenses can intervene at several points: at the input, inside the model, in the system architecture, or at the moment a proposed action is about to be executed (Chapter 2, Section 2.6). **This thesis chooses the last of these, the action boundary, as its point of intervention.** It is the point at which an induced financial action becomes visible as a concrete tool call with concrete arguments, and at which it can still be withheld before it takes effect. It is not the only place where an attack can be stopped, and other defenses act earlier.

A defense placed at the action boundary must decide, for each proposed tool call, whether to release it, modify it, refer it for human review or block it. Two questions about that decision are not settled by the reviewed literature (Chapter 2, Sections 2.9 and 2.12):

- how an estimated probability that an action was adversarially induced should be combined with the consequence of that action when choosing among responses;
- how often each response is chosen, and at what cost to legitimate work.

The problem addressed in this thesis is the runtime compromise of tool-using financial agents through indirect prompt injection. The thesis designs a decision layer at the action boundary that addresses these two questions, and evaluates it on recorded agent behavior in a banking environment.

1.3 Motivation

Three considerations motivate the work.

Concrete financial consequences. Tools that transfer value, modify payments or change credentials expose money, account control and personal data to manipulation (Chapter 2, Section 2.10). Conventional evaluation of financial LLM agents, based on accuracy and return metrics, has been argued to give an illusion of reliability while overlooking vulnerabilities such as adversarial prompt manipulation (Z. Chen, J. Chen, et al., 2025).

Record-keeping and human control. Regulation (EU) 2024/1689 requires high-risk AI systems to allow the automatic recording of events and effective human oversight, including the ability to intervene (European Parliament & Council of the European Union, 2024). Whether a given financial agent falls within a high-risk category is not assessed here. The provisions motivate two design properties: an audit record for every decision and an explicit route to human review.

Fragile evaluation. Pitfalls in the design and evaluation of learning-based security systems lead to unrealistic performance estimates (Arp et al., 2022). Audits of agent-safety benchmarks report metric flaws, inconsistent rankings and contradictory conclusions across benchmarks (Y. Wang et al., 2026; M. Q. Li et al., 2026). A defense for financial agents must therefore be evaluated so that:

- attack success is reported together with benign utility;
- dependence between benchmark tasks is respected;
- leakage between fitting and test data is controlled.

1.4 Research Gap

The gap is not the absence of runtime guardrails, probability calibration, graduated interventions, human review, provenance analysis or consequence-aware control. Each of these is present in the reviewed literature (Chapter 2, Section 2.13.3). Several recent systems are close to the design studied here:

- **NEXUS** is a structured-plan safety monitor that applies a formal intervention policy to choose among allow, block, request confirmation and request revision. It combines deterministic rules, argument-level inspection and a calibrated logistic-regression risk score (Hossain et al., 2026).
- **AgentTrust** intercepts tool calls before execution and returns one of four verdicts, combining a normalizer, multi-step chain detection and an LLM judge for ambiguous inputs (C. Yang, 2026).
- **H.-H. Chen (2026)** prices the consequence of agent actions against a safe default before execution.
- **FinHarness** addresses runtime safety of financial agents through monitors and an LLM-judge cascade (Jia et al., 2026).
- **C. Zhang et al. (2026)** show that calibrating a risk score need not improve the decisions taken with it.

A currency update of 30 October 2026 found further records close to this design, among them a controller that commits the proposed or an alternative action by a calibrated value learned from outcomes (Yan et al., 2026) and a three-way execute–confirm–block router with calibrated probabilities (Xin, 2026). They were examined at abstract level only and lie outside the systematic review (Chapter 2, Section 2.13.6).

The systematic review of Chapter 2 was conducted at record level: full texts could not be retrieved. Within that limit, the gap it supports is **empirical** (Chapter 2, Section 2.13.2). The review found three things:

- limited and conflicting evidence on whether behavioral and provenance evidence improves runtime decisions, and none testing it as input to a calibrated estimator for financial actions;
- no verified quantitative evaluation of a runtime defense on financial actions in an open, executable benchmark;
- no evaluation comparing a consequence-aware decision with a consequence-independent baseline that shares its estimator, with attack success, benign utility, intervention behavior, calibration and latency reported separately.

The specific combination that REM implements was not found in the records examined. Because the combination is specific and the review was record-level, its absence is not used as evidence of originality.

1.5 Research Aim

The approved research proposal states objectives but no separate aim. The aim below is derived from the approved objectives of Section 1.6 and adds nothing to them.

The aim of this research is to design a runtime evaluation and mitigation framework for the security of financial AI agents, to evaluate it against adversarial attacks, and to compare it with existing security approaches.

1.6 Research Objectives

The approved objectives are (Chapter 3, Section 3.1.2):

- **O1:** Design a REM framework for financial AI agents' and task automation agents' security.
- **O2:** Evaluate the framework against adversarial attacks.
- **O3:** Compare performance with existing security approaches.

O1 names task-automation agents as well as financial agents. **The evaluation in this thesis is restricted to a banking environment.** No task-automation environment was evaluated, so the evidence bears on financial agents only.

1.7 Research Questions

The research questions below are derived from the approved objectives (Chapter 3, Section 3.1.3). The approved proposal states no research questions, and these have not yet been confirmed by the supervisor; they remain proposed ([PD], decision D-10). Each is stated together with the kind of evidence this thesis could produce for it.

- **RQ1 (estimation).** How well does a ridge logistic regression estimator, using context, behavioral and action evidence available at the tool-call boundary, discriminate adversarially induced proposed actions in the AgentDojo banking environment, and how well calibrated are its probabilities under grouped cross-fitting? *Evidence:* offline classification of recorded steps.
- **RQ2 (decision).** Compared with a consequence-independent threshold policy that uses the same estimator, how does consequence-tiered expected-loss verdict selection change intervention behavior, attack success and utility, across a declared range of loss structures? *Evidence:* observe-only policy analysis of recorded steps, and a replay sensitivity analysis.
- **RQ3 (mitigation and cost).** What residual attack success remains after the Modify mechanism, how often is escalation selected, and what per-step processing time does REM add? *Evidence:* replay sensitivity analysis and runtime measurement. Residual success against live, adaptive agents is not measured.
- **RQ4 (component contribution).** What is the marginal contribution of each evidence group, of calibration, and of consequence tiers, as determined by ablation? *Evidence:* offline and observe-only ablations, and replay.

1.8 Proposed Approach

The Runtime Evaluation and Mitigation (REM) framework is a runtime mediation layer placed between one agent and its environment. It treats the agent as opaque: it observes inputs, context, proposed actions and tool results, but not model parameters or internal reasoning (Chapter 3, Section 3.3.1). REM comprises exactly five layers (Chapter 3, Section 3.2.1):

1. **Input & Context Layer.** Captures the information the agent handles and the action it proposes, and labels each context segment with its provenance.
2. **Detection Layer.** Computes evidence about untrusted context.

3. **Behavioral Analysis Layer.** Computes evidence about the episode history and the proposed action.
4. **Decision Engine Layer.** Performs five steps:
    - estimates the probability that the proposed action is adversarially induced, using ridge logistic regression (Cox, 1958; le Cessie & van Houwelingen, 1992);
    - calibrates that probability on the logit (Platt, 2000; Guo et al., 2017);
    - assigns a declared consequence tier and applies policy predicates;
    - selects the verdict among Allow, Modify, Escalate and Block by minimizing conditional risk under tier-indexed losses (Elkan, 2001; Chow, 1970);
    - writes an audit record.
5. **Mitigation Layer.** Executes the mechanism associated with the verdict.

Offline fitting and recalibration are cross-cutting activities performed between deployments; they are not a layer. A linear attribution of the logit (Lundberg & Lee, 2017) is written to the audit record and never influences the verdict.

None of these elements is new.

- Runtime interception of tool calls, probability calibration and a four-way set of graduated verdicts all have precedents (Chapter 2, Section 2.13.3).
- In REM, declared costs play the role that hand-chosen thresholds play in threshold policies: the threshold between two verdicts follows from the declared cost ratio for the action's consequence tier (Chapter 3, Section 3.7.5).

1.9 Research Methodology

The research follows design science research. In design science, knowledge is produced by building an artifact that addresses an identified problem and evaluating it (Hevner et al., 2004), following the six activities of Peffers et al. (2007) (Chapter 3, Section 3.1.1). The artifact is REM.

The evaluation is quantitative and uses the AgentDojo banking environment (Debenedetti et al., 2024). It compares REM with a baseline that uses the same estimator but a consequence-independent decision rule. A further configuration gives both the same opportunity to escalate, so that the effect of consequence tiers can be separated from the effect of escalation. Estimation, calibration and threshold selection use nested grouped

cross-fitting, because the benchmark does not support independent partitions (Chapter 3, Sections 3.11.4–3.11.5).

Live agent runs with REM enforcing its verdicts require access to a language model, which was not available. The evaluation therefore rests on traces of real agents recorded by the benchmark authors, used in three ways (Chapter 3, Sections 3.11.1 and 3.11.11):

- **offline classification** of recorded steps;
- **observe-only policy analysis**, in which verdicts are computed but not enforced;
- a **replay sensitivity analysis** that re-executes recorded trajectories under REM, assuming the agent does not adapt to refusals.

The replay analysis is a proposed amendment to the original scope, which excluded replay-based estimation; its status is stated wherever its results are used.

1.10 Scope

In scope (Chapter 3, Section 3.1.4):

- a single LLM-based agent that invokes tools with generated arguments;
- a financial (banking) task environment with executable tools and state-based checks;
- indirect prompt injection through tool outputs as the evaluated attack class;
- runtime mediation of each proposed tool call before it is released.

Out of scope:

- training-time defenses and modification of the agent's model or prompts;
- multi-agent systems;
- adaptive adversaries who tune attacks to REM;
- production deployment in a financial institution;
- task-automation environments other than banking;
- counterfactual estimation of intervention value for adaptive agents.

The design rests on stated assumptions (Chapter 3, Table 3.3). Among them:

- REM can intercept tool calls and results without modifying the agent (AS-1);
- REM's own code, parameters and logs are protected from the adversary (AS-3);
- adversaries do not adapt to REM in the primary evaluation (AS-6).

1.11 Contributions: Proposed and Achieved

Proposed contribution. The contribution proposed in Chapter 3 (Section 3.13) is empirical and integrative. It is the design and evaluation, on an open and executable banking benchmark under indirect prompt injection, of a non-invasive five-layer runtime layer that combines:

- provenance-based action evidence;
- a calibrated per-step probability of adversarial inducement;
- verdict selection by minimizing conditional risk with losses declared per consequence tier;
- deterministic mitigation whose effectiveness is measured.

It is compared against a consequence-independent baseline on the same estimator.

What this thesis achieves, and the evidence for each item (Chapters 3–5):

1. **A specified and implemented framework.** The framework is specified (Chapter 3) and implemented, with undeclared design values stopping the software rather than being replaced by defaults (Section 3.16). REM was placed at the tool-execution boundary of the benchmark without modifying the agent. *Evidence:* software tests and integration runs.
2. **An evaluation protocol** that isolates the decision rule from the estimator, and consequence tiers from escalation opportunity. It uses grouped cross-fitting that holds out whole attacker objectives. *Evidence:* the protocol and its execution (Chapter 4).
3. **Exploratory empirical evidence** on traces of 21 real LLM agents. It covers three kinds of analysis: offline discrimination and calibration; observe-only verdict behavior; and a replay sensitivity analysis of attack success and utility. *Evidence:* exploratory, from one benchmark, without live enforcement.

What this thesis does not achieve:

- an evaluation of enforcement against live agents (a closed-loop harness is implemented and tested with scripted test agents, Section 3.17, but has not been executed with a language model, Section 4.12);
- confirmatory results under supervisor-approved design values;
- a human audit of the labels;
- any per-backbone conclusion.

What the thesis does not claim:

- a new algorithm;
- the first runtime guardrail, calibrated agent risk score or consequence-aware controller;
- that calibration improves control;
- robustness against adaptive adversaries;
- any prevention guarantee;
- validity in production financial deployment.

Each constituent mechanism is established and attributed to its source (Chapter 3, Table 3.16).

1.12 Thesis Organization

The thesis is organized in five chapters:

- **Chapter 1** introduces the background, problem, gap, aim, objectives, research questions, approach, scope and contributions.
- **Chapter 2** reports a record-level systematic review of runtime security for tool-using AI agents, compares the closest prior work and states the research gap it supports.
- **Chapter 3** specifies the research methodology and the design of REM (architecture, threat model, evidence, estimator, calibration, consequence tiers, expected-loss decision, mitigation, audit), the experimental methodology and metrics, the software implementation and its verification (Section 3.16), and the design and implementation status of a closed-loop evaluation with live agents (Section 3.17).
- **Chapter 4** reports the exploratory evaluation on recorded agent traces, states the status of the live-agent evaluation, and discusses the findings, the comparison with prior work and the threats to validity.
- **Chapter 5** answers the research questions, assesses the objectives and contributions, summarises the limitations, and concludes with recommendations and future work ordered by dependency.

CHAPTER TWO: LITERATURE REVIEW: A RECORD-LEVEL SYSTEMATIC REVIEW OF RUNTIME SECURITY FOR TOOL-USING AI AGENTS

2.1 Introduction

Large language model (LLM) agents no longer only produce text for a human reader. They break tasks into steps, call external tools with arguments they generate themselves, read the results of those calls, and act again on what they read. When such an agent operates on financial tools, a single proposed tool call can:

- initiate a transfer;
- change a payee;
- modify a scheduled payment;
- send customer data to an external recipient.

The security question therefore shifts from what a model *says* to what an agent *does*. More precisely, it shifts to the moment at which a proposed action leaves the agent and takes effect in its environment.

This chapter reviews the literature on protecting such an agent at runtime. Earlier versions of the chapter were structured narrative reviews. This version is a **systematic review conducted at record level, with qualitative, structured evidence synthesis**. The search, screening, eligibility assessment, quality appraisal and data extraction followed a predefined protocol and are reported following PRISMA 2020 and PRISMA-S. Full-text reports could not be retrieved, so eligibility, appraisal and extraction rest on records: titles, abstracts and index metadata. **No full-text screening was performed**, and the chapter does not claim the depth of a full-text systematic review. The methodology in Section 2.2 records what was actually done: which sources were searched and which could not be accessed, the exact search strings, the screening decisions, the quality appraisal and the extraction. Where a step of the systematic-review process could not be carried out in full, the limitation is stated rather than concealed. No meta-analysis is attempted. The included studies differ in architecture, threat model, benchmark, metric and intervention, so their quantitative results cannot be pooled (Section 2.2.8).

The review is used to build an argument in eight steps:

1. agents create security risks that differ from those of conventional LLM use;
2. prior research has characterized the main threats and proposed many defenses;
3. those defenses differ in *where* they intervene;
4. they also differ in *what* they observe;
5. some capabilities are now well addressed;
6. others remain fragmented or weakly evidenced;
7. these patterns locate the Runtime Evaluation and Mitigation (REM) framework studied in this thesis;
8. the financial setting raises the stakes of each of these questions.

The research gap at the end of the chapter was derived only after screening, extraction and synthesis were complete. Section 2.13.1 assesses five candidate gaps against the evidence and retains, narrows or rejects each.

Chapter structure.

- Section 2.2 reports the review methodology.
- Section 2.3 explains why agent architecture changes the security problem.
- Sections 2.4 and 2.5 synthesize the evidence on threats, benchmarks and threat models.
- Section 2.6 organizes defenses by where they intervene.
- Section 2.7 examines runtime monitoring, behavioral evidence and provenance.
- Section 2.8 reviews the statistical and algorithmic foundations on which a runtime decision layer depends. The detailed REM specification is left to Chapter 3.
- Section 2.9 examines how runtime systems decide and mitigate.
- Section 2.10 separates financial-agent evidence from general agent-security evidence.
- Sections 2.11 and 2.12 compare the evidence across studies and synthesize it by theme.
- Section 2.13 states the research gap, positions REM, and records the limitations of the review.
- Section 2.14 summarizes the chapter.

2.2 Systematic Review Methodology

2.2.1 Review objectives and research questions

Derivation. The review questions are derived from the approved thesis direction and introduce no new thesis research questions. The approved proposal states three objectives:

- **O1:** design a REM framework for financial AI agents' and task automation agents' security;
- **O2:** evaluate the framework against adversarial attacks;
- **O3:** compare performance with existing security approaches (Chapter 3, Section 3.1.2).

The proposal does not state research questions. Chapter 3 (Section 3.1.3) derives four questions, RQ1–RQ4, from these objectives. They concern estimation, decision, mitigation and cost, and component contribution.

Review questions. The review is designed to supply the literature basis that O1–O3 and RQ1–RQ4 require. It asks five review questions (SRQ). These define the scope of this chapter only; they are not thesis research questions.

Review question	Wording	Serves
SRQ1	What security threats affect tool-using AI agents, particularly those operating through external tools and multi-step execution?	O1 (threat model); O2 (attack scenarios)
SRQ2	What defensive approaches have been proposed to detect, monitor, evaluate and mitigate these threats at runtime?	O1 (design space); O3 (comparators)
SRQ3	What behavioral, provenance, probabilistic, sequential and decision-theoretic techniques are used?	O1; RQ1, RQ2 and RQ4
SRQ4	What evidence exists for these approaches in financial or financially consequential agent settings?	O1 (financial scope); O2
SRQ5	What limitations and evidence gaps remain that motivate the REM framework?	Research gap (Section 2.13)

2.2.2 Information sources

The review protocol named eleven preferred sources:

- IEEE Xplore;

- ACM Digital Library;
- Scopus or Web of Science;
- ScienceDirect;
- SpringerLink;
- ACL Anthology;
- NeurIPS and PMLR proceedings;
- ICLR proceedings;
- USENIX;
- arXiv, as a supplementary source.

These sources were not directly searchable from the review environment. Direct access was attempted on 25 September 2026, and the network policy refused every connection. The connection log is retained with the search record. An attempt to use Elicit, a second scholarly index, also failed: the account's plan does not include API access. Each of these sources is therefore recorded as **DATABASE ACCESS UNAVAILABLE** (`CHAPTER2_SYSTEMATIC_SEARCH_STRATEGY.md`, §1).

Source actually searched. The one bibliographic database searched was **OpenAlex**, through a literature-search tool that queries its index of about 250 million records. OpenAlex indexes the proceedings and journals of the publishers listed above, as well as arXiv. Studies from those venues could therefore be retrieved, but only through OpenAlex's index and relevance ranking, not through each library's own search interface. The review accordingly does **not** claim to have searched the named digital libraries.

Other methods. A second, non-database stream was used, as PRISMA 2020 allows. It consists of the 70 sources in the reference list of the previous integrated version of this chapter. That list was assembled by known-item searching of the thesis corpus and backward citation checking, and it was verified between 16 and 25 September 2026. Crossref was used only to verify metadata, never to identify records.

2.2.3 Search strategy

Concept blocks. The protocol defines three concept blocks and a financial block:

- **A — Agents:** "LLM agent", "AI agent", "language model agent", "tool-using agent", "autonomous agent";

- **B — Security:** “security”, “prompt injection”, “indirect prompt injection”, “tool misuse”, “agent safety”, “goal hijacking”, “memory poisoning”, “action manipulation”;
- **C — Runtime, behavior and mitigation:** “runtime monitoring”, “runtime interception”, “behavioral monitoring”, “trajectory”, “tool use”, “provenance”, “anomaly detection”, “mitigation”, “guardrail”, “policy enforcement”;
- **Financial:** “financial agent”, “banking agent”, “financial AI agent”, “financial tool”, “financial transaction”, combined with security, prompt-injection, tool-misuse, runtime, safety or mitigation terms.

Database variant. The OpenAlex interface available to the review is lexical and relevance-ranked, and it does not accept Boolean operators. The blocks were therefore translated into eleven free-text queries, each combining at least one term from blocks A, B and C. Two queries (Q7, Q8) carried the financial block. The exact strings are reproduced in Table 2.1. Each query was restricted to publication years 2019–2026. Page 2 was also retrieved for the two queries with the highest page-1 yield of eligible records (Q1, Q2), giving 13 result pages and 320 records in total.

The final search date was **25 September 2026**, and every counted search ran on that date. The raw output of every query is stored unchanged, and a script recomputes all counts from it (`chapter2/slr/`).

Table 2.1. Database searches (OpenAlex; 25 September 2026; years 2019–2026)

Query	Exact search string	Blocks	Records screened	Index estimate
Q1 (pages 1–2)	LLM agent indirect prompt injection runtime defense tool use	A+B+C	50	1,366
Q2 (pages 1–2)	LLM agent security runtime monitoring guardrail tool call	A+B+C	49	1,123
Q3	tool-using language model agent behavioral trajectory monitoring anomaly detection safety	A+B+C	24	5,572
Q4	LLM agent provenance goal alignment tool call misalignment detection	A+B+C	25	484
Q5	prompt injection benchmark tool-integrated LLM agents attacks and defenses evaluation	A+B+C	24	2,950
Q6	LLM agent memory poisoning goal hijacking multi-step attack	A+B	24	579
Q7	financial LLM agent security prompt injection tool misuse	Fin+A+B+C	24	1,368
Q8	banking agent safety benchmark financial transaction LLM agent	Fin+A+B	25	536
Q9	calibrated risk score intervention policy LLM agent runtime safety	A+B+C	25	880
Q10	privilege control policy enforcement LLM agents tool calls prompt injection	A+B+C	25	1,380
Q11	pre-execution action validation guardrail autonomous LLM agent unsafe actions	A+B+C	25	744
Total			**320**	—

Retrieval cap. Only the relevance-ranked top of each query was screened: 24–25 records per page, the maximum the tool returned. The index estimates show that far more records matched the query terms. This cap is a deliberate feasibility decision, consistent with the protocol's instruction not to screen thousands of records manually. It is also the review's largest search limitation (Section 2.13.4). The stopping rule, the tool's handling of each string, and a one-off pilot probe that is not counted are all documented in the search record.

2.2.4 Eligibility criteria

A study was included when it satisfied all six criteria.

- **I1 — Population.** The study addresses AI or LLM agents, tool-using agents or agentic systems. *Operational clarification, fixed before eligibility assessment:*

LLM-integrated applications that ingest untrusted third-party content were counted as satisfying I1 when the attack or defense applies to the agent's input channel, because indirect injection against agents originates in that setting. Studies whose threat is specific to inter-agent communication in multi-agent systems were treated as outside the review population, which is single tool-using agents.

- **I2 — Problem.** The study addresses a security, safety, robustness, monitoring or misuse problem.
- **I3 — Contribution.** The study contains a substantive technical method, benchmark, empirical evaluation or analytical contribution. Position papers qualified only if they supplied a formal framework, a system design, or an empirical analysis of primary data. Argument-only position papers did not qualify.
- **I4 — Relevance.** The study directly informs at least one REM-relevant dimension: input or context security, runtime monitoring, behavioral analysis, action or tool monitoring, provenance, decision, mitigation, or financial execution security.
- **I5 — Extractability.** The study provides enough information to extract its relevant contribution.
- **I6 — Period.** The study was published between January 2019 and 25 September 2026, or it is a necessary **FOUNDATIONAL SOURCE**.

Exclusion labels. Records were excluded with a standard label:

- wrong population/system;
- wrong threat;
- wrong outcome;
- insufficient technical evidence;
- duplicate;
- outside scope;
- inaccessible full text or insufficient information;
- secondary study (survey, review, SoK or tutorial);
- metadata conflict;
- withdrawn.

Secondary studies and methodological sources. Secondary studies were not synthesized as primary evidence. Three of them, identified by the database search, are cited only as background: two definitional surveys and a finance SoK.

Methodological sources that do not concern agents also fail I1. These are the statistical, calibration, change-detection, decision-theoretic and anomaly-detection sources, and they were admitted under the second clause of I6 as **FOUNDATIONAL SOURCES**. Those

published within the review period are labelled methodological sources. They support Section 2.8 and do not count as included studies.

The prior-art studies named in the protocol were given no special route. Each of AgentTrust, NEXUS, ProvenanceGuard, FinHarness, AgentDojo, AgentHarm, Progent, AgentSpec, ShieldAgent, Task Shield, MELON, CaMeL and LlamaFirewall passed through the same criteria.

2.2.5 Study selection

Selection had two stages, both carried out by a single reviewer: the thesis author, assisted by an AI coding assistant (Claude Code), as recorded in the search-strategy file (`CHAPTER2_SYSTEMATIC_SEARCH_STRATEGY.md`).

1. **Title and abstract screening** of the 219 records that remained after automated deduplication. Records counted as duplicates when they shared a DOI or their normalized titles matched; this removed 101 of the 320 hits.
2. **Eligibility assessment** of the records that passed screening.

Full texts were not retrieved. The network policy that blocked the databases also blocked publisher and preprint servers. All 104 reports sought for retrieval (45 database, 59 other-methods) were therefore *not retrieved*, and eligibility was assessed on **records**: title, abstract, indexed metadata, Crossref metadata and, for two records without an indexed abstract (VIGIL and R-Judge), the abstract text returned by a web search. PRISMA 2020 distinguishes records, reports and studies; in this review each included study is represented by a single index record, and no full-text report was assessed. This departure from standard practice is shown in the PRISMA flow diagram (Figure 2.1). A supplementary check on 28 October 2026 (Section 2.13.4) re-examined the records of the closest studies; it did not add full texts.

Other-methods stream. The 70 other-methods records were assessed in the same way against the prior verification record. Eleven were the same study as a database record and were counted once.

Screening log. Every decision, with its reason, is in `CHAPTER2_SCREENING_LOG.csv`.

Table 2.2. Study selection (counts computed from the screening log)

Stage	Unit	Databases (OpenAlex)	Other methods
Identified	records	320	70
Removed before screening (duplicates; same study as a database record)	records	101	11
Screened (title/abstract)	records	219	—
Excluded at screening	records	174	—
Sought for retrieval (full text)	reports	45	59
Not retrieved (network policy)	reports	**45**	**59**
Assessed for eligibility **at record level** (no full-text report available)	records	45	59
Excluded at eligibility	records	5	28 (6 background, 21 foundational/methodological, 1 withdrawn)
Included	**studies**	**40**	**31**
Total included studies (each represented by its index record)	studies	**71**	

Exclusions at screening. The 174 records excluded at screening break down as follows:

- 81 outside scope;
- 71 secondary studies;
- 7 multi-agent-channel studies;
- 7 wrong threat;
- 4 wrong population;
- 2 official reports;
- 1 wrong outcome;
- 1 duplicate.

Exclusions at eligibility. Five records passed screening but were excluded at eligibility:

- a retrieval-augmented question-answering system without tool actions (wrong population/system);

- a record whose index metadata attached AgentDojo's DOI and authors to an unrelated abstract (metadata conflict);
- an argument-only position paper (insufficient technical evidence);
- an observability taxonomy without a security evaluation (wrong outcome);
- a record without an abstract (insufficient information).

Recall of the database search. Of the thirteen named prior-art studies, the database search retrieved three: Task Shield, LlamaFirewall and, with corrupted metadata, AgentDojo. The other ten entered through the other-methods stream. This low recall of known relevant studies is itself a finding about the search, discussed in Section 2.13.4.

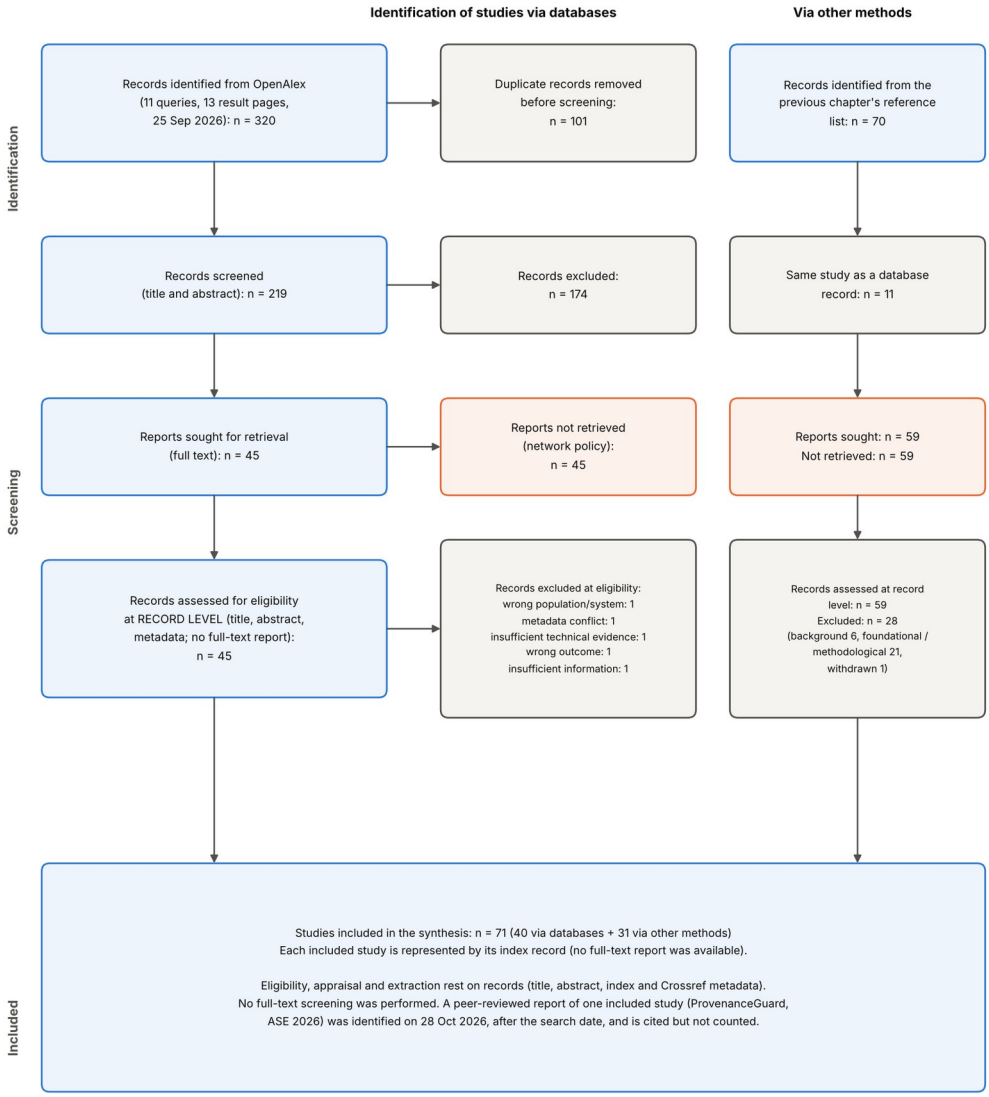


Figure 2.1. PRISMA 2020 flow diagram of the record-level review. Full-text reports were sought but not retrieved; eligibility was assessed on records. Counts are computed by script from the stored search output and the screening log.

2.2.6 Quality appraisal

Every included study was appraised against the protocol's ten-item checklist. The items ask whether the study reports:

- Q1 — its objective;
- Q2 — its threat model;
- Q3 — its method;
- Q4 — reproducibility;
- Q5 — its dataset or benchmark;
- Q6 — its metrics;
- Q7 — its comparators;
- Q8 — its limitations;
- Q9 — evidence for its quantitative claims;
- Q10 — peer-review status.

Codes. Each item is coded Yes, No, Not Verified or Not Applicable. Q10 also has a "preprint clearly labelled" code.

Categories. No numeric score is computed. A fixed, published rule assigns each study to a category of **record-visible evidence**:

- **HIGH EVIDENCE:** peer reviewed, with benchmark, figures and comparator all reported in the record;
- **MODERATE EVIDENCE:** quantitative results in a peer-reviewed or clearly labelled preprint record, or a peer-reviewed evaluation without verifiable figures;
- **LIMITED EVIDENCE:** everything else.

These categories describe **how much evidence the record makes visible**, which depends on how much information could be accessed. They are not a judgement of the study's intrinsic quality: a rigorous study with a terse abstract can fall into LIMITED EVIDENCE, and a full-text appraisal could move it.

Override. One override is applied and documented. FinHarness is set to LIMITED EVIDENCE because all of its figures were measured on FinVault, whose preprint was recorded on 25 September 2026 as withdrawn by its authors with a comment citing legal, intellectual-property and affiliation reasons rather than a scientific error. The withdrawal does not show that FinHarness's measurements are wrong; it means they cannot be

confirmed against a publicly maintained benchmark. The override reflects confirmability, not invalidity (Section 2.10.2).

Results. Of the 71 included studies:

- 10 are HIGH EVIDENCE, 36 MODERATE EVIDENCE and 25 LIMITED EVIDENCE;
- 36 are peer reviewed and 35 are preprints or working papers;
- 62 are RECORD-VERIFIED and 9 PARTLY UNVERIFIED. RECORD-VERIFIED means that the bibliographic metadata and the cited contribution were checked against the study's index record (title, abstract, metadata); it does **not** mean that the full text was read. PARTLY UNVERIFIED means that at least one cited detail rests only on search-engine excerpts and is unverified.

Levels of claim verification. Every claim this chapter makes about a study is at one of four levels:

Level	Meaning
M — metadata verified	Title, authors, venue and year checked against Crossref or the index record
A — abstract-supported	The claim is stated in the study's abstract as indexed
F — full-text verified	The claim was checked against the full text
U — unverified	The claim rests on search-engine excerpts or an earlier reading that could not be reproduced

No claim in this chapter is at level F, because no full text was retrieved. Claims at level U are marked "(unverified)" where they are used.

What the categories measure. Because full texts were unavailable, the categories measure **how much of each study's evidence could be verified from its record**, not the study's intrinsic quality (`CHAPTER2_QUALITY_APPRAISAL.md`). Items that depend on the full text (Q4 reproducibility, Q7 comparators, Q8 limitations) are most often Not Verified.

How the categories are used. They qualify how strongly a finding is worded in the synthesis. They were not used to exclude studies.

2.2.7 Data extraction

A single reviewer extracted 21 fields per included study into `CHAPTER2_SYSTEMATIC_EVIDENCE_MATRIX.csv`:

- study, year and venue;
- agent type;
- threat and attack entry point;
- whether the study operates at runtime;
- context, behavioral, tool/action, multi-step and provenance evidence;
- decision mechanism and mitigation;
- benchmark;
- financial context;
- metrics;
- main result and limitation;
- evidence level.

Coding values. Each property is coded **Yes**, **No**, **Not Reported**, **Not Applicable** or **Not Verified**. “Yes” is never entered for a property that is merely absent from the record. Conversely, **Not Reported is not evidence of absence**: a property that the record does not mention may still be present in the system. Counts of “Not Reported” cells therefore describe the records, not the systems, and are never used in this chapter to argue that a capability is missing from the literature.

Definitions and verification. Financial context is coded “Yes” only where the study’s setting is primarily financial. A general benchmark that contains one financial scenario is coded “No”, with a note. Every included study also carries a verification label, RECORD-VERIFIED or PARTLY UNVERIFIED, following the thesis rule that search-engine text cannot settle a primary claim.

Numbers. Every number reported in this chapter is taken from the primary report of the study concerned, as recorded in the matrix.

2.2.8 Evidence synthesis

No meta-analysis. Quantitative pooling was not possible. The included studies use different agents, attack sets, benchmarks, success definitions and metrics, and many report only author-selected configurations. A pooled estimate would have no interpretable common parameter.

Structured synthesis. The synthesis is therefore qualitative and structured. Evidence is organized into eleven themes:

1. threats;
2. entry points;
3. observation granularity;
4. runtime intervention;
5. behavioral and trajectory evidence;
6. provenance and goal–action consistency;
7. statistical and algorithmic methods;
8. decision and consequence handling;
9. mitigation;
10. financial-agent evidence;
11. evaluation quality.

Five questions per theme. For each theme the synthesis asks:

- what the literature establishes;
- what remains uncertain;
- what is under-evaluated;
- what differs across studies;
- what this implies for REM.

These syntheses appear at the end of the relevant sections and are consolidated in Section 2.12 (Table 2.10). Conflicting findings are presented side by side rather than resolved in favor of either side.

How results are worded. Author-reported figures are stated as such, with their setting, and are not compared across studies as if they were measured on a common scale. Claims that rest on a single study are worded as single-study findings.

2.3 AI Agents and Emerging Security Risks

2.3.1 From language models to agents

In the survey literature, an LLM-based autonomous agent is a system in which a language model acts as the controller for perception, reasoning, planning and action in pursuit of a task, supported by memory and tool-use modules (L. Wang et al., 2024; background source). Two capability developments made this architecture practical:

- **ReAct** interleaves reasoning with action. The model generates a thought, selects an action, receives the environment's observation, and conditions its next step on that observation (Yao et al., 2023).
- **Toolformer** showed that a language model can learn which application programming interfaces to call, when to call them, and with what arguments (Schick et al., 2023).

Both are capability papers. They fail criterion I2 and are cited only as background. Together they describe the execution pattern that most agent frameworks still follow: a loop of reasoning, tool invocation and observation, which continues until the model judges the task complete.

2.3.2 Why agent architecture changes the security problem

The loop that makes agents useful also creates their attack surface. Three structural properties distinguish agent security from the security of a stand-alone model:

- **Several input channels of different trustworthiness.** The system prompt, the user's request, tool outputs, retrieved documents and stored memory all enter the same context window, and the model interprets all of them.
- **Statefulness across a trajectory.** Content that enters at one step can influence an action several steps later. The unit of analysis is therefore a trajectory, not an input–output pair.
- **Outputs with external effects.** An agent's outputs include tool calls. A successful manipulation does not stay in the text domain; it becomes a state change in an external system.

A component view of agent threats organizes the same observation around the model "brain", memory, tools and environment interaction (Yu et al., 2025; background source).

Two included benchmark studies show that these properties matter empirically:

- **ClawSafety** ran 2,520 sandboxed trials of personal agents with elevated local privileges. Attack success varied from 40% to 75% across five backbone models, and varied sharply with the channel through which the injection arrived. Cross-scaffold experiments showed that safety depended on the whole deployment stack, not on the backbone model alone (Wei et al., 2026; preprint).
- **ToolEmu** showed that agent risk does not require an attacker. In an LM-emulated sandbox with 36 toolkits and 144 test cases, even the safest agent tested failed in 23.9% of cases. Human evaluators judged 68.8% of the failures identified to be valid real-world failures (Ruan et al., 2024).

2.3.3 Implication

The reviewed studies locate agent risk at the interaction between the model, its heterogeneous input channels, its accumulated state and its tools, and ClawSafety adds the agent framework itself. A defense that inspects only the user's input, or only the final text, does not observe the point at which the agent's decision becomes an external effect. The remainder of the chapter therefore attends to **where** defenses intervene and **what** they observe.

2.4 Threats Against AI Agents

The OWASP Gen AI Security Project lists prompt injection first among risks to LLM applications. It distinguishes *direct* injection through the user's prompt from *indirect* injection through external content (OWASP Gen AI Security Project, 2025). This is an official practitioner source and is used only for terminology.

The threat evidence below comes from 23 included studies whose primary role is an attack, benchmark or threat analysis. Studies of defenses contribute threat evidence where they evaluate against named attacks.

2.4.1 Direct and indirect prompt injection

Direct injection. Y. Liu et al. (2024) moved prompt-injection research from demonstrations to systematic evaluation. They formalized attacks in a common framework and benchmarked five attacks against ten defenses across ten LLMs and seven tasks. HouYi, a black-box injection technique, found 31 of 36 commercial LLM-integrated applications susceptible, a result validated by ten vendors (Y. Liu et al., 2023; preprint). For a financial agent, direct injection matters less than its indirect counterpart, because the user of such an agent is usually the principal it serves rather than the adversary.

Indirect injection. Greshake et al. (2023) showed that instructions planted in data retrieved at inference time can compromise an LLM-integrated application remotely. Such retrieved prompts can act like arbitrary code execution and control which APIs are called. In tool-using agents, the attacker needs only to influence some content the agent will read. The malicious content then enters through the **observation channel**, *after* the user's request has been received and inspected.

Benchmark evidence. Agent-specific benchmarks quantify the threat:

- **InjecAgent** has 1,054 cases covering 17 user tools and 62 attacker tools. A ReAct-prompted GPT-4 agent was vulnerable in 24% of cases, and success nearly doubled when the injection was reinforced with a "hacking prompt" (Zhan et al., 2024).
- **Agent Security Bench (ASB)** covers ten scenarios with more than 400 tools and 27 attack and defense methods. It reported a highest average attack success rate of 84.30% across 13 backbones, and found current defenses of limited effectiveness (H. Zhang et al., 2025).
- **AgentVigil** automates black-box red-teaming. It achieved 71% and 70% success against agents based on o3-mini and GPT-4o on AgentDojo and VWA-adv, nearly double handcrafted baselines (Z. Wang et al., 2025).
- A demonstration against web-navigation agents embedded universal triggers in page HTML and achieved credential exfiltration and forced clicks on real websites (Johnson et al., 2025). This study is LIMITED EVIDENCE: its abstract gives no figures.

2.4.2 Tool misuse, action manipulation and new tool channels

Tool misuse is the use of a legitimate capability against the user's intent. Action manipulation is its adversarially induced form: a well-formed call to a permitted tool, with attacker-chosen arguments or timing. The included studies show that the tool interface is now an attack surface in its own right:

- **Tool selection.** ToolHijacker injects a malicious tool document into the tool library so that the agent selects the attacker's tool. Six prevention-based and detection-based defenses proved insufficient against it (J. Shi et al., 2026).
- **Tool metadata.** A STRIDE/DREAD threat model of the Model Context Protocol (MCP) identified *tool poisoning*, meaning instructions embedded in tool metadata, as the most prevalent client-side vulnerability. Most of the seven MCP clients tested showed weaknesses in static validation and parameter visibility (C. Huang et al., 2026).
- **Tool servers.** A study of 1,899 open-source MCP servers found MCP-specific tool poisoning in 5.5% of servers and general vulnerabilities in 7.2% (Hasan et al., 2026).
- **Tool arguments and plans.** Imprompter optimizes obfuscated prompts that cause improper tool use; its information-exfiltration attack succeeded end to end in nearly 80% of trials against a production agent (Fu et al., 2024; preprint). AI² builds semantically harmless inputs that hijack an application's action plan. It reports an average success rate of 84.30% and bypasses common safety filters in 92.7% of cases (Y. Zhang et al., 2024; preprint).

- **Framework tools.** In LLM-integrated frameworks, prompt injection reaches code-execution and database tools. LLMSmith found 20 vulnerabilities in 11 frameworks, 19 of them remote code execution, of which 17 were confirmed and 11 received CVEs (T. Liu et al., 2023; preprint). Prompt-to-SQL injection was shown to be pervasive across seven LLMs in LangChain applications (Pedro et al., 2023; preprint).
- **Skills.** Reusable agent "skills" create a supply-chain channel. SkillJect automatically poisons skill instructions and helper scripts and outperforms manual skill-injection attacks (X. Jia et al., 2026; preprint; LIMITED EVIDENCE).

Malicious users. Harmful behavior can also be requested directly. AgentHarm contains 110 malicious agentic tasks (440 with augmentations) in 11 harm categories (Andriushchenko et al., 2025). Its threat model, a malicious *user*, contrasts with REM's primary threat model, in which the adversary sits in the data read by a benign user's agent.

2.4.3 Context and memory manipulation

Where an agent retrieves from memory or a knowledge base, an adversary can plant content that is acted on later. AgentPoison reported an attack success rate of at least 80%, with benign impact of at most 1%, at a poison rate below 0.1% (Z. Chen et al., 2024). Memory poisoning separates the write from the activation in time. To an observer without provenance information, the eventual retrieval therefore looks legitimate.

2.4.4 Multi-step propagation, goal drift and abnormal execution

Some compromises contain no single anomalous step. Several runtime systems target this pattern:

- goal-conditioned drift detection (C. L. Wang et al., 2025);
- cross-turn drift in a financial query monitor (H. Jia et al., 2026);
- hazardous drift through individually benign actions (W. Lin et al., 2026);
- task-level transactions that expose cross-step violations (Z. Chen et al., 2026).

Abnormal execution. Abnormal execution can itself be the attack. Malfunction amplification misleads agents into repetitive or irrelevant actions. It induced failure rates above 80% in several scenarios, and its authors found it harder to detect than overtly harmful attacks (B. Zhang et al., 2025). In anomaly-detection terms, a sequence of individually normal events whose *combination* is anomalous is a *collective* anomaly (Chandola et al., 2009; foundational source).

Misaligned agents. A distinct threat is posed by an agent that is itself misaligned. Monitors for covert “scheming” operate on observable trajectories. Monitors trained on synthetic data generalized to more realistic environments, but their performance saturated quickly (Storf et al., 2026; preprint).

2.4.5 Adaptive attackers

Adaptive attacks bypassed all eight indirect-injection defenses evaluated by Zhan et al. (2025), with attack success consistently above 50%. ToolHijacker (J. Shi et al., 2026) and AI² (Y. Zhang et al., 2024) report similar defeats of the defenses they tested. Detection results measured against fixed attack templates are therefore upper bounds on robustness, not estimates of it.

Table 2.3. Threat classes and the included evidence

Threat class	Entry point	Temporal scope	Why it is hard to detect	Included studies (evidence level)
Direct prompt injection	User channel	Single turn	Injected instructions resemble legitimate ones	Y. Liu et al. 2024 (MOD); Y. Liu et al. 2023 (MOD)
Indirect prompt injection	Observation channel (tool outputs, retrieved content, web pages)	Cross-step	Enters after input inspection	Greshake et al. 2023 (MOD); Zhan et al. 2024 (HIGH); H. Zhang et al. 2025 (HIGH); Z. Wang et al. 2025 (HIGH); Johnson et al. 2025 (LIM); Wei et al. 2026 (MOD)
Tool channel attacks (selection, metadata, servers, skills)	Tool library, tool metadata, skill files	Setup-time or cross-step	Poisoned artefacts are trusted as configuration	J. Shi et al. 2026 (LIM); C. Huang et al. 2026 (MOD); Hasan et al. 2026 (HIGH); X. Jia et al. 2026 (LIM)
Tool misuse / action manipulation	Tool arguments; action plans	Step or chain	Call is well formed; anomaly is relational	Fu et al. 2024 (MOD); Y. Zhang et al. 2024 (MOD); T. Liu et al. 2023 (MOD); Pedro et al. 2023 (LIM); Andriushchenko et al. 2025 (MOD)
Context and memory manipulation	Memory, retrieval corpus	Cross-session	Write and activation separated in time	Z. Chen et al. 2024 (MOD)
Goal drift / multi-step propagation	Trajectory	Trajectory	No single step need be anomalous	C. L. Wang et al. 2025 (MOD); W. Lin et al. 2026 (MOD); Z. Chen et al. 2026 (LIM)
Abnormal / repetitive execution	Trajectory	Trajectory	Anomaly lies in the pattern	B. Zhang et al. 2025 (MOD)
Misaligned (scheming) agent	Agent itself	Trajectory	Covert pursuit of another goal	Storf et al. 2026 (LIM)

Threat class	Entry point	Temporal scope	Why it is hard to detect	Included studies (evidence level)
Adaptive attacks	Any	Any	Designed against the defense	Zhan et al. 2025 (MOD); J. Shi et al. 2026 (LIM); Y. Zhang et al. 2024 (MOD)

Evidence levels: HIGH, MOD (moderate) and LIM (limited), as defined in Section 2.2.6.

Themes 1 and 2 (threats; entry points): synthesis.

- **Established.** Indirect injection through the observation channel is demonstrated, benchmarked and adaptive. Its success rates are substantial across independent benchmarks, and the defenses of the time failed against adaptive attacks. The harm materializes when the agent proposes a tool call.
- **Uncertain.** Reported success rates differ by an order of magnitude across benchmarks, models and attack channels: from 24% on InjecAgent to 84.30% average on ASB, and 40–75% across models on ClawSafety. Those differences cannot be attributed to any single factor.
- **Under-evaluated.** The newer tool-channel threats (tool metadata, tool libraries, skills and MCP servers) are documented mainly by attack and measurement studies. Few runtime defenses have been evaluated against them. Abnormal and repetitive execution is evidenced by one peer-reviewed study.
- **Differs across studies.** The threat actor varies: an attacker in the data, a malicious user, a poisoned tool supplier, or a misaligned agent. So does the success criterion, which may be an attacker goal achieved, an unsafe action executed, or a task failure.
- **Implication for REM.** The observation point for REM is the proposed tool call. That is where every threat class above converges on an external effect. REM's primary threat model, indirect injection against a benign user's financial agent, is the best-evidenced class. Tool-channel threats and misaligned-agent threats lie outside REM's evaluated scope and should be stated as such in Chapter 3.

2.5 Benchmarks and Threat Models

Benchmarks operationalize threat models, and they bound what a defense can be shown to achieve. Table 2.4 evaluates the benchmarks used by included studies on seven dimensions:

- what the benchmark actually measures;

- its threat model;
- its execution realism;
- its financial relevance;
- its multi-step coverage;
- whether it measures utility;
- its limitations.

Table 2.4. Critical evaluation of benchmarks used by included studies

Benchmark (source)	What it measures	Threat model	Execution realism	Financial relevance	Multi-step coverage	Utility measured	Limitations
AgentDojo (Debenedetti et al., 2024)	Task utility, utility under attack and targeted attack success per episode; 97 user tasks, 629 security cases	Indirect injection via tool outputs	Executable, stateful tools; outcomes checked against environment state	One of four suites is Banking; no finance-specific analysis in the verified record	Yes (multi-tool episodes); number of genuinely multi-step attacks not verified	Yes, jointly with security	Limited number of distinct tasks; robustness claims bounded by the attacks evaluated (Section 2.4.5)
InjecAgent (Zhan et al., 2024)	Whether the next action after one injected tool response serves the attacker; 1,054 cases	Indirect injection (direct harm; data exfiltration)	Simulated single tool response	Not reported	No (short cases)	No	Isolates injection susceptibility; no benign utility
ASB (H. Zhang et al., 2025)	Attack success across stages; utility–security trade-off metric	Direct/ indirect injection, memory poisoning, backdoor, mixed	Agent framework with >400 tools	Finance is one of 10 scenarios; not analysed separately	Yes (multiple stages)	Yes	Breadth over depth per scenario
AgentHarm (Andriushchenko et al., 2025)	Whether harmful multi-step tasks are completed	Malicious user	Synthetic tools	Fraud is one of 11 categories	Yes	Capability retention after jailbreak	Different threat actor from REM's
ToolEmu (Ruan et al., 2024)	Risky failures without adversary; 144 cases	No adversary	LM-emulated tools	Not reported (financial loss among risk examples)	Yes	Not the focus	Emulator and evaluator are LMs

Benchmark (source)	What it measures	Threat model	Execution realism	Financial relevance	Multi-step coverage	Utility measured	Limitations
R-Judge (Yuan et al., 2024)	LLM ability to judge safety risk in 569 multi-turn records	Behavioural safety risks	Recorded interactions (offline)	Not reported	Yes	Not applicable	Offline judgement, not runtime interception
ClawSafety (Wei et al., 2026)	Attack success in high-privilege workspaces; 120 scenarios, 2,520 trials	Injection via skills, email, web	Sandboxed real frameworks	Finance is one of five domains	Not verified	Not verified	Preprint; release announced
SIREN (J. Lin et al., 2026, VIGIL)	Tool-stream injection defense; 959 cases	Injection via tool metadata and feedback	Not verified	Not reported	Not verified	Utility under attack	Introduced with the defense evaluated on it
TS-Bench (Mou et al., 2026, ToolSafe)	Step-level detection of unsafe tool invocations	Unsafe invocations incl. injection	Not verified	Not reported	Yes (interaction history)	Benign completion	Introduced with the guardrail evaluated on it
NEXUS-Bench (Hossain et al., 2026)	Plan-level intervention accuracy; 128 held-out synthetic instances	Unsafe plans	Author-generated templates	Not reported	Plan-level	Paired benign cases	Authors: in-distribution results are upper bounds
FinVault (Z. Yang et al., 2026)	Financial-agent security (as originally reported)	Financial-agent attacks	Execution-grounded (as originally reported)	Yes	Not verified	Not verified	**Withdrawn by its authors; not evidence**

Audits of benchmark validity. Two included meta-research studies qualify how far any of these scores can be read as measurements of safety:

- M. Q. Li et al. (2026; preprint) catalogued 40 agent-safety benchmarks and found no ranking concordance across evaluation dimensions (Kendall's *W*). Benchmark choice alone can therefore change which system appears safest.

- Y. Wang et al. (2026; preprint) found that on one binary trace-judgment benchmark an “always positive” baseline outranked five models under F1. They also found that a cross-benchmark correlation moved from −0.64 with seven models to +0.02 with eighteen, an artefact of small panels.

The machine-learning-for-security literature anticipated these problems. Sampling bias, inappropriate baselines, base-rate neglect and lab-only evaluation recur in top-venue security papers (Arp et al., 2022; methodological source).

Benchmarks built with the defense. Two further observations come from the extraction. First, six included defense or benchmark studies introduce the benchmark on which their own method is evaluated: SIREN, TS-Bench, SciSafetyBench, NEXUS-Bench, PromptShield and the MCP policy-enforcement point’s controlled dataset. Second, most of these report no evaluation on an independent benchmark in the verified record. NEXUS is an exception: it reports out-of-distribution results on R-Judge.

AgentDojo in particular. AgentDojo receives particular attention because of its executable Banking suite, but the evidence does not support calling it the best benchmark. Its advantages for REM’s question are specific:

- tools execute and change environment state;
- utility and security are measured separately;
- a Banking suite exists.

Its limitations are equally specific:

- a small number of distinct user tasks;
- robustness claims bounded by the attacks actually evaluated (Section 2.4.5);
- no verified finance-specific analysis in the literature reviewed here.

The number of distinct tasks, not the number of generated runs, bounds what an evaluation on it can conclude (Chapter 3 treats this under grouping).

Theme 11 (evaluation quality) — benchmark component.

- **Established.** Benchmark results are benchmark-specific and do not transfer.
- **Uncertain.** How many genuinely multi-step attacks the common benchmarks contain.
- **Under-evaluated.** Independent evaluation of defenses on benchmarks their authors did not build.
- **Differs across studies.** Success criteria and utility definitions.

- **Implication for REM.** Results must be reported per benchmark, with benign utility, and interpreted against the number of distinct tasks. FinVault cannot be used.

2.6 Defensive Approaches

The 42 included defense and design studies differ along two dimensions that the literature rarely separates:

- **where they intervene:** at the input, in the context, in the model, in the system architecture, at the policy or tool layer, around the agent loop at runtime, or after an action has taken effect;
- **what they observe:** an isolated input, an individual tool call, a plan, the flow of data between steps, a trajectory, or the consequence of an action.

This section organizes defenses by where they intervene. Section 2.7 examines what runtime monitors observe, and Section 2.9 examines how they decide and mitigate.

Table 2.5. Where included defenses intervene and what they observe

Point of intervention	Included studies	Primary unit observed	What this position can and cannot see
Input / context text	InstructDetector (Wen et al., 2025); PromptShield (Jacob et al., 2025); attack-technique defenses (Y. Chen et al., 2025); Spotlighting (Hines et al., 2024); PromptGuard 2 in LlamaFirewall (Chennabasappa et al., 2025)	An input or retrieved text	Sees text before the model acts; does not see how the agent then acts
Model	StruQ (S. Chen et al., 2025)	Model behaviour under separated channels	Addresses the instruction/data confusion; requires training and model access
System architecture / information flow	f-secure IFC (F. Wu et al., 2024); RTBAS (Zhong et al., 2025); ACE (E. Li et al., 2026); IsolateGPT (Y. Wu et al., 2025); CaMeL (Debenedetti et al., 2025)	Data flow into planning and between tools or apps	Can prevent untrusted data from steering control flow; changes how the agent executes
Policy / privilege at the tool boundary	Progent (T. Shi et al., 2025); AgentSpec (H. Wang et al., 2026); Conseca (Tsai & Bagdasarian, 2025); MCP policy-enforcement point (S. Wang et al., 2026); LATTICE (Calboreanu, 2026)	Each tool call against a policy	Deterministic and auditable; answers “is this permitted?”, not “how likely is this induced?”
Plan or reasoning	TrustAgent (Hua et al., 2024); Thought-Aligner (Jiang et al., 2025); NEXUS (Hossain et al., 2026); DRIFT planner (H. Li et al., 2025)	A plan or intermediate thought	Can intervene before actions exist; plans can change after new observations

Point of intervention	Included studies	Primary unit observed	What this position can and cannot see
Runtime around the agent loop	Task Shield (F. Jia et al., 2025); MELON (Zhu, Yang, et al., 2025); ProvenanceGuard (She et al., 2026); VIGIL (J. Lin et al., 2026); ToolSafe (Mou et al., 2026); DRIFT validator (H. Li et al., 2025); GuardAgent (Xiang et al., 2025); ShieldAgent (Z. Chen, Kang, & Li, 2025); AGrail (Luo et al., 2025); SafeScientist (K. Zhu et al., 2025); AlignmentCheck (Chennabasappa et al., 2025); AgentTrust (C. Yang, 2026); MI9 (C. L. Wang et al., 2025); PRISM (F. Li, 2026); ProbGuard (H. Wang et al., 2025); DreamGuard (Lin et al., 2026); SafeAgent (H. Liu et al., 2026); FinHarness (H. Jia et al., 2026); constitutional monitors (Storf et al., 2026)	Proposed action in context; sometimes the trajectory	Observes the action boundary; costs are latency and false interventions
Post-action / transactional	Cordon (Z. Chen et al., 2026); GoEX (Patil et al., 2024)	Staged or reversible effects	Can hold, undo or confine effects; consumes rather than produces a risk judgement
Consequence gating	Actuarial Action Interface (H.-H. Chen, 2026)	Priced action	Prices consequence; does not estimate adversarial induction

2.6.1 Input-, context- and model-level protection

Input detectors. Detectors that act on text report strong in-distribution results:

- InstructDetector uses a model's hidden states and gradients to detect embedded instructions. It reports 99.60% in-domain and 96.90% out-of-domain accuracy, and reduces attack success to 0.03% on BIPIA (Wen et al., 2025).
- PromptShield curates a benchmark for deployable detectors, and its fine-tuned detector improves performance in the low-false-positive regime (Jacob et al., 2025).

- Spotlighting transforms untrusted input so that the model receives a continuous provenance signal, reducing attack success from above 50% to below 2% with GPT-family models (Hines et al., 2024).

Model-level protection. StruQ trains the model to follow instructions only in the prompt channel (S. Chen et al., 2025). This addresses the root cause identified by Greshake et al. (2023), but it requires training and access to the model. That is a significant constraint for a financial institution that deploys a third-party model.

Shared limitations. These approaches share two limitations for tool-using agents:

- they act on text, not on actions, so an input detector does not observe what the agent then does;
- ToolHijacker defeated six such defenses (J. Shi et al., 2026), and AI² bypassed common safety filters in 92.7% of cases (Y. Zhang et al., 2024).

2.6.2 System-level and information-flow defenses

A second family changes the system so that untrusted data cannot steer control flow:

- **f-secure LLM systems.** A security monitor filters untrusted input out of the planning process, and formal models state the security guarantee (F. Wu et al., 2024; preprint).
- **RTBAS.** Information-flow control is adapted to tool-based agents. Tool calls whose integrity and confidentiality can be established are executed automatically, and the user is asked to confirm the rest. On AgentDojo it prevented all targeted attacks with a 2% utility loss under attack (Zhong et al., 2025; preprint).
- **ACE.** Planning is decoupled: an abstract plan is built from trusted information only, its information-flow constraints are checked statically, and data and capability barriers are enforced during execution. It reports security against InjecAgent and ASB attacks, and it demonstrates new attacks against IsolateGPT (E. Li et al., 2026).
- **IsolateGPT.** Third-party apps run in isolation, with overhead under 30% for three-quarters of tested queries (Y. Wu et al., 2025).
- **CaMeL.** Control and data flow are extracted from the trusted query and capability policies are enforced at each tool call. CaMeL solved 77% of AgentDojo tasks with provable security, against 84% for the undefended agent (Debenedetti et al., 2025, v2; an earlier version reported 67%).

Conflicting evidence. The IsolateGPT–ACE pair is a direct conflict within this family. One peer-reviewed study reports that isolation protects against the attacks it tested. A later

peer-reviewed study constructs attacks that defeat it. Architectural guarantees hold only for the threat models their authors formalized.

2.6.3 Policy enforcement and tool restriction

Classical basis. This family descends from execution monitoring. Schneider (2000; foundational source) characterized the policies a monitor can enforce by observing a system's execution and halting it before a violation: the safety properties.

Included instances:

- **Progent** expresses privileges as rules over tool names and arguments, with solver-checked updates. It reduced attack success from 39.9% to 1.0% on AgentDojo and from 70.3% to 3.9% on ASB (T. Shi et al., 2025; preprint).
- **AgentSpec** enforces rules written as trigger, predicate and enforcement. It prevented over 90% of unsafe code-agent executions and all hazardous embodied actions, and gave 100% compliance in autonomous driving, at millisecond overhead (H. Wang et al., 2026).
- **Conseca** argues for policies generated just in time for each task context and enforced deterministically (Tsai & Bagdasarian, 2025; LIMITED EVIDENCE).
- **MCP policy-enforcement point.** A policy-enforcement point for MCP agents intercepts at the tool-call boundary with declarative rules over cross-step information-flow labels, and keeps a hash-chained audit log. On a controlled dataset, it reduced attack success from 40.0% to 5.0%, against 35.0% for the strongest prompt-only baseline (S. Wang et al., 2026). The same study also reports:
    - a task-level false-positive rate of 30.0%, attributed mainly to by-design capability denials;
    - residual attack success of 16.7% on one backend, due to a rule-coverage gap;
    - sub-millisecond overhead.
- **LATTICE** gates execution through policy-as-code with deterministic verdicts, escalates to human operators on a confidence basis, and keeps cryptographic audit trails (Calboreanu, 2026).

Limitations for REM. These studies are fast, auditable and, in the architectural cases, provable for the threats they address. Two limitations recur:

- **The question is binary.** An enforcer answers whether an action is permitted. It does not estimate how likely a *permitted* action is to have been induced by an adversary.

- **Policy coverage bounds protection.** The policy-enforcement point's residual 16.7% attack success on one backend is a rule-coverage gap in its authors' own words.

2.6.4 Learned and LLM-based guardrails

A further family uses learned models or LLMs to judge actions or trajectories:

- **GuardAgent** generates guardrail code from safety requests (Xiang et al., 2025).
- **ShieldAgent** verifies trajectories against probabilistic rule circuits extracted from policy documents (Z. Chen, Kang, & Li, 2025).
- **AGrail** keeps adaptive safety checks in a lifelong memory (Luo et al., 2025).
- **TrustAgent** applies an agent constitution around planning (Hua et al., 2024).
- **Thought-Aligner** corrects unsafe intermediate thoughts. It raised behavioral safety from about 50% to about 90% on average across six LLMs (Jiang et al., 2025).
- **ToolSafe** trains a step-level guardrail that judges each tool invocation before execution from the interaction history. Its feedback-driven framework reduced harmful invocations by 65% on average and improved benign completion by about 10% under injection (Mou et al., 2026).
- **VIGIL** replaces isolation with verify-before-commit. It reduced attack success by over 22% relative to dynamic defenses and more than doubled utility under attack relative to static baselines on its SIREN benchmark (J. Lin et al., 2026; PARTLY UNVERIFIED).
- **DRIFT** combines a secure planner, a dynamic validator of plan deviations and privileges, and an injection isolator for the memory stream. It is evaluated on AgentDojo, ASB and AgentDyn (H. Li et al., 2025).
- **SafeScientist** monitors prompts, agent collaboration and tool use in scientific agents (Zhu, Zhang, et al., 2025).

Limitations. These systems cover a broad range of semantic violations without writing a policy for each. Their limitations also recur:

- LLM judgement adds latency and can vary between runs;
- accuracies are measured on the authors' own benchmarks;
- the outputs are verdicts, and the verified records do not report calibrated probabilities that could be combined with the consequence of an action. The one exception is NEXUS (Section 2.9.3).

2.6.5 Goal–action consistency and provenance

The family most directly aimed at indirect injection asks whether a proposed action is *justified* by the user's task, rather than whether it is harmful in itself:

- **Task Shield** verifies whether each instruction and tool call serves the user's goals. It reduced attack success to 2.07% with 69.79% utility on AgentDojo with GPT-4o (F. Jia et al., 2025).
- **MELON** re-executes the trajectory with the user's prompt masked and flags an attack when the actions remain similar (Zhu, Yang, et al., 2025).
- **ProvenanceGuard** asks whether a tool call is supported by traceable evidence in the context. Relative to an LLM-as-judge baseline, it reduced error on misaligned traces from 44.3% to 2.1% on Agent-SafetyBench and from 32.4% to 18.7% on WorkBench (She et al., 2026; preprint). Its reported intervention rate on aligned traces (14.5% against 10.9%) and its extra cost are PARTLY UNVERIFIED.

These approaches share one insight: under indirect injection, the useful evidence concerns the *relation* between an action and its sources. They differ in cost. MELON adds an execution, and Task Shield and ProvenanceGuard add LLM reasoning.

Themes 3 and 4 (observation granularity; runtime intervention): synthesis.

- **Established.** Runtime intervention at the tool-call boundary is widely implemented: 41 of the 42 included defense and design studies operate at runtime, and 34 inspect tool calls or actions. Deterministic enforcement is fast. Several peer-reviewed studies report sub-millisecond to millisecond overheads.
- **Uncertain.** Robustness to adaptive attacks. Architecture-level guarantees have been broken by later attacks (IsolateGPT–ACE), and input-level detectors have been bypassed (ToolHijacker, AI²).
- **Under-evaluated.** Evaluation of any given defense on a benchmark that its authors did not construct. Evaluation against the tool-channel threats of Section 2.4.2.
- **Differs across studies.** The unit observed differs: input text, tool call, plan, data flow or trajectory. So does the question asked, which may be about permission, alignment, provenance or harm.
- **Implication for REM.** Interception at the tool-call boundary is established, and REM cannot claim it. What the families leave open is the combination of an estimate of *how likely* a permitted, well-formed action is to be adversarially induced with the *consequence* of that action. Section 2.9 examines this.

2.7 Runtime Monitoring and Behavioral Analysis

This section concerns the evidence a runtime layer can extract from the agent's behavior, as distinct from the text it receives. That evidence is the domain of REM's Behavioral Analysis Layer.

2.7.1 Step-level, sequence-level and accumulating monitors

Runtime monitors differ along two dimensions that are easily conflated: **decision granularity** (whether a decision is taken for each proposed step, for a plan, or for an episode) and **evidence scope** (whether that decision uses only the current step or also the history that led to it). A step-level decision can be history-aware: it can be taken for each proposed call while conditioning on the episode so far.

Step-level monitors with current-step evidence judge each proposed action against a policy or the current context. Examples are Progent and AgentSpec. They are fast and interpretable, but a monitor that uses no history cannot see an attack whose individual steps are each acceptable.

Step-level monitors with history-aware evidence still decide per proposed action, but their evidence includes earlier context. Task Shield, ProvenanceGuard and VIGIL check a proposed call against the user's goal or against evidence accumulated in the context; REM (Chapter 3) belongs to this group, because its per-step features include repeated calls, privilege escalation, transition deviation and argument provenance computed over the episode.

Sequence-level monitors judge an action against a model of the trajectory:

- MI9 enforces temporal behavioral patterns through finite-state conformance engines and detects goal-conditioned drift. It reports 99.81% detection across 1,033 *synthetic* scenarios (C. L. Wang et al., 2025; preprint; PARTLY UNVERIFIED).
- ProbGuard learns a discrete-time Markov chain over abstract states. It intervenes when the estimated probability of remaining safe falls below a threshold, with PAC-style guarantees on the learned model (H. Wang et al., 2025; preprint).
- DreamGuard maintains a recurrent latent state and fuses immediate-hazard and prefix-risk scores into PASS, HOLD or BLOCK, at about 25 ms per call (W. Lin et al., 2026; preprint).
- ToolSafe's guardrail reasons over the interaction history (Mou et al., 2026).
- Constitutional monitors judge whole trajectories for scheming (Storf et al., 2026).

Accumulating monitors keep an explicit running state across steps. PRISM accumulates session risk with time-to-live decay (F. Li, 2026). AgentTrust applies seven order-aware chain detectors (C. Yang, 2026). FinHarness integrates per-step risk in a cascade (H. Jia et al., 2026).

A formal view. A formal account of runtime governance frames policies as functions of the agent's identity, its *partial execution path*, the proposed next action and the organizational state, mapped to a probability of policy violation. It shows that system prompts and static access control are special cases, and it lists risk calibration as an open problem (Kaptein et al., 2026; preprint; LIMITED EVIDENCE). This formalization supports evaluating actions in the light of their path. It also shows that the idea of a path-conditioned violation probability is not new.

2.7.2 Provenance and cross-step information flow as evidence

A second line of evidence concerns *where an action's content comes from*. In an indirect injection against a banking agent, the attacker typically supplies the decisive arguments inside untrusted content: a recipient account, an amount, a new payee. Whether an argument can be traced to the user's request or to trusted state is therefore strong evidence of adversarial induction.

Included studies. Eight of the 42 included defense and design studies code provenance or information-flow evidence as present (Table 2.8). They realize it in three ways:

- **by analysis:** ProvenanceGuard's evidence tracing;
- **by construction:** CaMeL, ACE and f-secure extract flows from trusted inputs;
- **by labels:** RTBAS's dependency screeners and the MCP policy-enforcement point's labels for source integrity and data sensitivity.

2.7.3 What is known about the empirical value of behavioral and provenance evidence

The review found **conflicting evidence** on whether trajectory and provenance components change outcomes. All three direct tests of a component's contribution are single studies on their authors' own benchmarks.

- **A null result for trajectory evidence (unverified).** According to search-engine excerpts that could not be checked against the abstract or the full text (level U), AgentTrust's ablation found that disabling its session tracker left verdict accuracy

unchanged on both of its benchmarks (C. Yang, 2026). Because this result is unverified, the conflict described below rests on one verified and one unverified finding.

- **A positive result for cross-step provenance.** In the MCP policy-enforcement point, disabling cross-step label propagation raised the call-level false-negative rate by 26.4 percentage points on the authors' controlled dataset (S. Wang et al., 2026; HIGH EVIDENCE). The authors frame the result as mechanism-coverage evidence, not a deployment estimate.
- **A positive result for provenance against an LLM judge.** ProvenanceGuard's error reductions relative to an LLM-as-judge baseline indicate that evidence tracing is informative on misaligned tool calls (She et al., 2026; the headline figures are verified at abstract level).
- **Context for these results:**
    - MI9's evaluation was synthetic;
    - ProbGuard's gains were in autonomous-driving and embodied settings;
    - DreamGuard's figures in the verified record concern latency, not detection;
    - constitutional monitors saturated quickly, and further optimization overfitted (Storf et al., 2026).

The conflict is informative rather than contradictory:

- The null result concerns a *session tracker of order-aware chain detectors* in a rule-dominated system on coding- and shell-oriented scenarios.
- The positive results concern *cross-step information-flow labels* and *evidence provenance* against injected or misaligned tool calls.

On the included evidence, it is plausible that provenance-type trajectory evidence contributes under indirect injection while generic sequence patterns may not. No included study tests either kind of evidence as an input to a *calibrated estimator*, and none tests it on financial actions.

2.7.4 Foundations: behavioral anomaly detection and its cautions

Foundational work. The idea of detecting compromise from behavior predates LLM agents. Forrest et al. (1996) built a profile of normal system-call sequences and flagged sequences absent from it. That is the closest classical analogue to monitoring an agent's tool-call sequence. Chandola et al. (2009, 2012) distinguished three kinds of anomaly and formalized anomaly detection over discrete sequences:

- *point* anomalies: in agents, a single out-of-policy call;

- *contextual* anomalies: a transfer that is normal in general but anomalous given its provenance;
- *collective* anomalies: goal drift and multi-step chains.

Cautions. The same literature carries cautions that transfer directly:

- attacks differ from typical learning tasks, with high error costs and a diverse normal class (Sommer & Paxson, 2010);
- security evaluations recur with sampling bias, base-rate neglect and lab-only testing (Arp et al., 2022).

For an agent monitor, these imply three requirements: a representative benign workload, interpretation of deviations in security terms, and models of "normal" learned from data resembling deployment. These are foundational and methodological sources, not included studies.

Themes 5 and 6 (behavioral/trajectory evidence; provenance and goal–action consistency): synthesis.

- **Established.** Many runtime systems observe trajectories: 17 of 42 code behavioral evidence and 21 code multi-step handling. Provenance and goal consistency are established signals against injected or misaligned actions.
- **Uncertain.** The measured contribution of each kind of evidence. One ablation found no effect of a session tracker, and another found a 26.4-point effect of cross-step labels. Both are single-study results on the authors' own benchmarks.
- **Under-evaluated.** The effect of trajectory and provenance evidence when it feeds a probability estimator rather than a rule or a judge. Evaluation in financial settings.
- **Differs across studies.** How provenance is obtained (analysis, construction or labels) and how sequences are modeled (automata, Markov chains, latent states or heuristics).
- **Implication for REM.** REM cannot claim trajectory monitoring or provenance analysis as new. Its evaluation must report the contribution of behavioral and provenance evidence, and must not assume it. It must also show that the benchmark contains the behavior the features are meant to detect.

2.8 Statistical and Algorithmic Foundations Relevant to REM

A runtime decision layer that assigns graduated responses must solve four methodological problems:

- turn evidence into an estimate of how likely an action is to be adversarially induced;
- make that estimate interpretable as a probability;
- combine the probability with the consequence of the action to reach a decision;
- record why the decision was made.

A layer that observes several steps must also decide how to accumulate evidence over time. This section explains **why** each established method is relevant and what the included studies show about its use in agent security. The methods are established, and none is a contribution of this thesis. Their precise form in REM, including the estimator, the calibration map, the loss structure and the verdict rule, is specified in Chapter 3 and is not repeated here. The sources in this section are **FOUNDATIONAL** or **METHODOLOGICAL SOURCES** admitted under criterion I6 (Section 2.2.4).

2.8.1 Why logistic regression

Logistic regression models the probability of a binary outcome as a logistic function of a linear score (Cox, 1958):

$$Pr(y=1/x)=\frac{1}{1+\exp\left(-\left(\beta_0+\beta^T x\right)\right)} \quad (2.1)$$

Why it suits a runtime layer.

- Evaluation costs one inner product.
- Coefficients can be inspected.
- The output is intended as a probability, not as an arbitrary score.

Limitations.

- Interactions must be engineered into the features.
- A model fitted on one attack distribution can be miscalibrated on another.
- With scarce adversarial examples and binary indicators, the unpenalized estimate may not be finite. A ridge penalty addresses this (le Cessie & van Houwelingen, 1992).
- Naive Bayes is the generative counterpart with the same linear form. It may reach its (higher) asymptotic error with fewer examples (Ng & Jordan, 2002). It is an established alternative estimator and is not part of REM.

Use in agent security. Among the included studies, only NEXUS uses a logistic risk score for agent safety (Hossain et al., 2026). The choice of estimator family therefore has one direct precedent in agent security, and that precedent operates at the plan level.

2.8.2 Why calibration matters, and its limits

Definition. A classifier is *calibrated* when its predicted probabilities match observed frequencies. Calibration matters whenever a probability is multiplied by a cost, because miscalibration distorts the comparison between actions.

Platt scaling. Platt (2000) fits a sigmoid to a model's score s on held-out data:

$$\hat{p} = \frac{1}{1+\exp(As+B)} \quad (2.2)$$

The mapping is monotone, so it preserves the ranking of predictions.

Alternatives.

- **Temperature scaling** is a one-parameter variant (Guo et al., 2017).
- **Isotonic regression** is nonparametric (Zadrozny & Elkan, 2002). Platt scaling tends to perform better with small calibration sets, whereas isotonic regression performs as well or better once calibration data are plentiful (Niculescu-Mizil & Caruana, 2005).
- **Beta calibration** contains the identity map on scores in [0, 1], which the logistic family applied to such scores does not (Kull et al., 2017).

In REM, Platt scaling on the logit is the adopted calibrator. Beta, isotonic and temperature calibration are reviewed alternatives and are **not adopted** (Chapter 3).

Measuring calibration. Calibration is measured by the expected calibration error (ECE; Naeini et al., 2015; Guo et al., 2017). ECE depends on the binning and on the size of the calibration set.

Evidence from included studies.

- **NEXUS** reports that Platt scaling reduced ECE from 0.085 to 0.013 on its benchmark. It also evaluated isotonic regression, and its authors report that Platt was less prone to overfitting on the small calibration split (Hossain et al., 2026; the calibration details are PARTLY UNVERIFIED).
- **C. Zhang et al. (2026)** caution against over-reading such improvements. In their ALFWorld analysis, Platt scaling reduced a confidence score's ECE from 0.463 to

0.006, while control regret under threshold routing stayed at 0.318 (preprint; figures PARTLY UNVERIFIED). A calibrated risk estimate is not the decision-relevant quantity for choosing an intervention.

- **LATTICE** adds independent peer-reviewed evidence in the same direction from a different setting. Across four planner families and 4,000 trajectories, a *confidence-threshold baseline* had a false-allow rate ranging from 0.03 to 0.998 (Calboreanu, 2026). A single threshold on a confidence signal was therefore unstable across the upstream models that produced the actions.

Implication. Calibration quality and control quality must be measured and reported separately. A probability, however well calibrated, does not by itself represent whether a trajectory remains recoverable.

2.8.3 Why sequential evidence matters

When evidence arrives step by step, a monitor must decide when the accumulated evidence justifies action.

CUSUM. Classical sequential analysis gives a statistically grounded answer. The cumulative-sum procedure (Page, 1954) accumulates log-likelihood ratios of post-change against pre-change behavior and raises an alarm when the accumulated statistic crosses a threshold (Basseville & Nikiforov, 1993). Under the classical change-point model, with independent observations before and after the change, it has an asymptotic optimality property for detection delay at a given false-alarm constraint (Lorden, 1971).

Limits for agents. These assumptions are only approximately satisfied by tool-call sequences, and an adversary can pace an attack to stay below the threshold. The procedure also detects a *change*, whereas an agent episode may be adversarial from its first injected observation.

Heuristic alternatives. Among the included studies, sequential accumulation is heuristic: PRISM’s session risk with time-to-live decay (F. Li, 2026) and FinHarness’s cascade (H. Jia et al., 2026).

Role in REM. CUSUM is **not part of REM’s verdict path**. Chapter 3 uses it only as an optional, offline, observe-only analysis of logged indicators that never influences a verdict.

2.8.4 Why expected-loss decisions are relevant

The principle. Cost-sensitive decision theory chooses the response with the lowest expected cost given the class probabilities and a cost matrix (Elkan, 2001). For responses *a* and a binary state *y*:

$$a^*(x) = \arg\min_a \sum_{y \in \{0,1\}} Pr(y/x) C(a,y) \quad (2.3)$$

The reject option. Chow (1970) showed that withholding an automatic decision is optimal when the expected cost of deciding exceeds the cost of rejecting. This is the classical basis for routing uncertain cases to human review.

Requirements. The rule needs calibrated probabilities (Section 2.8.2) and costs that reflect real preferences, which are rarely known precisely in security. Elkan's principle and Chow's analysis support REM's expected-loss verdict and its Escalate route. Neither source proposed REM's four-action design.

Precedents among included studies.

- **NEXUS** defines an expected-loss objective with fixed intervention costs and uses it to set loss-optimal thresholds inside a rule-first cascade (Hossain et al., 2026; cascade and loss-based threshold selection stated in the authors' repository README, level R; the cost values are PARTLY UNVERIFIED).
- **H.-H. Chen (2026)** prices the consequence of each action deterministically against a reserve.
- **Kaptein et al. (2026)** formalize a path-conditioned probability of policy violation.

Cost-based reasoning about agent actions therefore has precedents.

2.8.5 What attribution contributes to auditability

Additive attribution. Additive attribution explains a prediction as a sum of per-feature contributions. For a linear model under feature independence, the Shapley attribution of a feature is its coefficient times the feature's deviation from its mean (Lundberg & Lee, 2017). On the log-odds of a logistic model, that decomposition is exact and costs no more than the score itself. This makes it suitable for an audit record that shows which evidence contributed to a decision.

Other audit traces. It is not a distinguishing mechanism. Included systems already provide audit traces in other forms:

- rules and findings (NEXUS; AgentTrust);
- verifiable rule circuits (ShieldAgent);
- traceable evidence (ProvenanceGuard);
- tamper-evident or hash-chained logs (PRISM; the MCP policy-enforcement point; LATTICE’s cryptographic audit trail).

In REM, attribution is **audit-only** and never enters the verdict.

Table 2.6. Established methods reviewed as the algorithmic basis for REM

Problem	Method (original source)	Why relevant	Main limitation	Use among included studies
Probabilistic risk scoring	Logistic regression (Cox, 1958); ridge penalty (le Cessie & van Houwelingen, 1992); naive Bayes as alternative (Ng & Jordan, 2002)	Fast, inspectable, probabilistic output	Linear log-odds; distribution shift	NEXUS (plan level)
Calibration	Platt (2000); temperature (Guo et al., 2017); isotonic (Zadrozny & Elkan, 2002); beta (Kull et al., 2017)	Makes scores usable with costs	Calibration does not imply good control	NEXUS (Platt, isotonic evaluated); C. Zhang et al. (analysis); DreamGuard (calibrated fusion)
Calibration measurement	ECE (Naeini et al., 2015; Guo et al., 2017)	Quantifies miscalibration	Bin-dependent; unstable with small sets	NEXUS; C. Zhang et al.
Sequential accumulation	CUSUM (Page, 1954; Basseville & Nikiforov, 1993); optimality (Lorden, 1971)	Statistically grounded accumulation	Assumptions not met by agent trajectories; evadable by pacing	Heuristic analogues only (PRISM; FinHarness). In REM: optional, observe-only
Response selection	Minimum expected cost (Elkan, 2001); reject option (Chow, 1970)	Uses consequences explicitly; human-review region	Costs uncertain; recoverability omitted	NEXUS objective (fixed costs, cascade); consequence pricing (H.-H. Chen)
Audit decomposition	Additive attribution (Lundberg & Lee, 2017)	Exact for linear scores; cheap	Not a novel explanation mechanism	Rule/evidence/audit traces in NEXUS, AgentTrust, ShieldAgent, ProvenanceGuard, PRISM, LATTICE

Theme 7 (statistical and algorithmic methods): synthesis.

- **Established.** Each building block is established, and several are already applied in agent security.

- **Uncertain.** How these blocks perform in combination at the action boundary of an executing agent under indirect injection.
- **Under-evaluated.** Calibration of agent-safety probabilities. Only NEXUS reports calibration error for a deployed scorer, and C. Zhang et al. and LATTICE show that calibration or a threshold on confidence does not by itself secure good control.
- **Differs across studies.** Probabilities are produced by logistic scores, learned Markov chains, latent world models, or LLM judges.
- **Implication for REM.** REM's estimator, calibrator and decision rule are adopted, not contributed. Calibration must be reported separately from control. The comparison that isolates the effect of using consequences is against a consequence-independent rule that uses the *same* estimator.

2.9 Runtime Decision and Mitigation

This section examines how runtime systems *act* on what they detect. It distinguishes four functions that the literature often conflates:

- **detection:** recognizing that something may be wrong;
- **risk estimation:** quantifying how likely or how severe;
- **decision:** selecting a response;
- **mitigation:** executing the response.

2.9.1 The space of runtime responses

The included systems use a common, graduated vocabulary:

- allow;
- block;
- modify: rewrite arguments, sanitize context, propose a safer alternative or request a revision;
- escalate: request confirmation or human review;
- restrict tools or privileges;
- interrupt or terminate;
- stage, undo or roll back effects.

Human participation is also studied directly. An analysis of 21 production agent systems found runtime approval in 15 of them, policy specification in 14 and scope configuration in 16. It found no production deployment of intent anchoring or trust labeling, the

mechanisms most studied academically. It also identified a trade-off between approval fatigue and uncontrolled autonomy (P. Wang et al., 2026; preprint). A simulation study added in the supplementary update of 2 November 2026 reports, at abstract level, that human-in-the-loop approvals sharply reduce high-impact actions but degrade under user error and habituation (Kanaker et al., 2026; Section 2.13.7).

Table 2.7. Runtime responses reported by included systems

System	Allow	Block	Modify / sanitize / revise	Escalate / human review	Restrict tools or privileges	Interrupt / terminate	Stage / roll back / undo	Decision granularity
Progent	Yes	Yes	Not Reported	Yes (approval for privilege expansion)	Yes	Not Reported	Not Reported	Per tool call
AgentSpec	Yes	Yes	Rule-defined	Not Verified	Not Reported	Not Verified	Not Reported	Per triggered event
RTBAS	Yes (auto-execute)	Not Verified	Not Reported	Yes (user confirmation)	Not Reported	Not Reported	Not Reported	Per tool call
MCP policy-enforcement point	Yes	Yes (deny)	Not Reported	Not Reported	Yes (capability tokens)	Not Reported	Not Reported	Per tool call with cross-step labels
LATTICE	Yes	Yes	Not Reported	Yes (confidence-based escalation to humans)	Not Reported	Not Reported	Not Reported	Per gated action
VIGIL	Yes (commit)	Yes (no commit)	Not Verified	Not Reported	Not Reported	Not Reported	Not Reported	Per speculative action
ToolSafe	Yes	Not Verified	Yes (guardrail feedback to agent)	Not Reported	Not Reported	Not Reported	Not Reported	Per tool invocation
MI9	Yes	Yes	Not Verified	Not Verified	Yes (graduated containment)	Yes	Not Reported	Continuous, agent-level
PRISM	Yes	Yes	Not Verified	Not Verified	Yes (policy controls)	Not Verified	Not Reported	Per hook, session accumulation

System	Allow	Block	Modify / sanitize / revise	Escalate / human review	Restrict tools or privileges	Interrupt / terminate	Stage / roll back / undo	Decision granularity
SafeAgent	Yes	Yes	Yes (sanitization, replanning, argument rewriting)	Yes (human escalation)	Not Verified	Yes	Yes (rollback)	Per step over session state (PARTLY UNVERIFIED)
AgentTrust	Yes	Yes	Yes (safer alternatives)	Yes (warn; review)	Not Reported	Not Reported	Not Reported	Per tool call, session chains
NEXUS	Yes	Yes	Yes (request revision)	Yes (request confirmation)	Not Reported	Not Reported	Not Reported	Per plan, before execution
DreamGuard	Yes (PASS)	Yes (BLOCK)	Not Reported	HOLD (semantics Not Verified)	Not Reported	Not Reported	Not Reported	Per action with trajectory state
FinHarness	Yes (approve)	Yes	Not Reported	Escalation to an advanced LLM judge (not a human)	Not Reported	Not Reported	Not Reported	Per step, cross-turn
Jackson	Yes	Yes (deny)	Yes (sanitize)	Yes (escalate)	Not Verified	Not Verified	Not Verified	Not Verified
Cordon	Not Applicable	Not Applicable	Not Applicable	Not Verified	Not Applicable	Not Applicable	Yes (shadow state; effect outbox)	Per task transaction
GoEX	Not Applicable	Not Applicable	Not Applicable	Post-facto human validation	Not Reported	Not Reported	Yes (undo; damage confinement)	Per executed action

2.9.2 AgentTrust: runtime interception with graduated verdicts

AgentTrust (C. Yang, 2026; preprint; MODERATE EVIDENCE; PARTLY UNVERIFIED) is among the closest prior art to REM. It passed the inclusion criteria through the other-methods stream. The protocol asks that its runtime interception, verdicts, monitoring, multi-step handling, mitigation, evaluation, latency and ablation claims each be verified; the verification status of each is given below.

Architecture and verdicts (level A). AgentTrust intercepts agent tool calls *before execution* and returns one of four verdicts: allow, warn, block or review. Its abstract names a shell deobfuscation normalizer, SafeFix suggestions of safer alternatives, RiskChain detection of multi-step attack chains, and a cache-aware LLM judge for ambiguous inputs.

Results (level A). On its 300-scenario internal benchmark, a production-only rule set reached 95.0% verdict accuracy and 73.7% risk-level accuracy. On 630 independently constructed adversarial scenarios, the author reports 96.7% under a *patched* rule set, explicitly not zero-shot.

Unverified details (level U). The following rest on search-engine excerpts and are not stated in the abstract retrieved on 28 October 2026: an analyzer with 42 risk patterns; a policy engine with 170 rules; seven order-aware chain detectors; and

- an end-to-end latency of about 1.72 ms against about 1.35 s for a zero-shot LLM judge;
- worst-case aggregation over the normalized variants of an input;
- the judge’s five dimensions, one of which is reversibility;
- the ablation in which disabling the session tracker left accuracy unchanged.

What REM cannot claim. AgentTrust already provides:

- pre-execution interception;
- a four-way graduated verdict set that includes human review;
- a modify-like response (SafeFix);
- session-level multi-step detection;
- component ablations with latency.

Where the difference lies. The defensible differences are narrower:

- **Evidence to verdict.** AgentTrust's verdicts come from analyzers and rules, with an LLM judge for ambiguous cases. The verified material reports no calibrated probability of adversarial induction.
- **Use of consequence.** Reversibility is one of the judge's dimensions, not an explicit loss that changes the decision at a fixed probability.
- **Threat and domain.** AgentTrust targets side effects of coding and shell agents. It does not report evaluation on indirect injection against a financial agent.

2.9.3 NEXUS: calibrated risk scoring with a four-way intervention set

NEXUS (Hossain et al., 2026; preprint; MODERATE EVIDENCE; PARTLY UNVERIFIED) is the closest prior art to REM's *decision* component. It also passed the inclusion criteria through the other-methods stream. The protocol asks that its logistic risk scoring, calibration, consequence features, expected-loss objective, decision policy, intervention cascade and evaluation each be verified; the verification status of each is given below.

Architecture (level A). NEXUS is a structured-plan safety monitor that "applies a formal intervention policy" to select among allow, block, request confirmation and request revision. It combines deterministic safety rules, argument-level inspection and a *calibrated* logistic-regression risk score used for graded escalation.

Results (level A). On a 128-instance synthetic benchmark it reports F1 of 0.949 and four-class intervention accuracy of 0.6406, 27.3 percentage points above rule-only selection. On R-Judge it reports F1 0.861 against 0.849 for rule-only; it matches rule-only on AgentHarm; it reports 0% ASR at 99% control allow on an indirect-injection set; F1 0.881 on its rule-blind stress set; and a median latency of 0.205 ms.

Authors' repository (level R, checked 1 November 2026). The README of the authors' public repository (github.com/eliashossain001/nexus, main branch; SHA-256 of the retrieved file recorded in `verification/sources_R4/`) states the intervention policy explicitly: Block if any critical violation, or if the risk score $\rho \geq \tau_b$ with at least one violation; Confirm if $\rho \geq \tau_c$ or any high-severity violation; Revise if any medium-severity violation; otherwise Allow. It states that $(\tau_b, \tau_c) = (0.75, 0.70)$ was selected on a safety–utility loss $L(\Pi) = E[\lambda_s u_s + \lambda_o u_o + \lambda_c c(\Pi(P))]$ over a 5×5 grid of (λ_s, λ_o), and that Platt scaling, fitted on a 60-instance calibration split, reduced ECE from 0.085 to 0.013 on the 128-instance test split. The README describes a retrained 9-feature scorer, and its

headline figures (for example F1 0.965 on the synthetic test split) differ from the abstract figures quoted above. Repository documentation can differ from the paper version cited, so the abstract figures are retained and the difference is recorded rather than resolved.

Still unverified (level U). The following rest on search-engine excerpts or on an earlier reading of the paper that could not be reproduced in the review environment:

- **Calibration figures.** Isotonic regression also evaluated (the ECE figures are now supported at level R).
- **Expected-loss objective.** The specific intervention costs (0 allow, 0.1 revise, 0.3 confirm, 1 block). The use of a λ-weighted loss to select the thresholds is supported at level R.
- **Consequence features.** Side-effect, irreversibility, sensitivity, permission and cost annotations on each planned tool call, used by the rules.
- **Session state.** A session manager for cross-turn state.

What REM cannot claim. NEXUS already provides (level A) calibrated logistic risk scoring, a formal policy over a four-way intervention set with confirmation and revision routes, and sub-millisecond latency; and, if the level-U description is accurate, consequence annotations used by rules and an expected-loss formulation with loss-optimal thresholds.

Where the difference lies. Two differences rest on level-A evidence:

- **Point of evaluation.** REM evaluates each proposed tool call during execution; NEXUS is described as a structured-*plan* monitor.
- **Threat model and setting.** REM is evaluated on indirect injection in an executable banking environment; NEXUS's headline results are on a synthetic benchmark, with R-Judge, AgentHarm and an indirect-injection set as additional evaluations.

A third difference lies in the decision rule. REM selects each verdict by minimum expected loss with tier-indexed losses. NEXUS, according to its repository README (level R), applies a rule-first cascade with two score thresholds chosen once by a loss criterion. The cost values NEXUS uses remain level U, and the paper's full text has not been read. Whether any of these differences changes outcomes is an empirical question.

2.9.4 Other decision mechanisms

Jackson (2025). A working paper (LIMITED EVIDENCE; PARTLY UNVERIFIED) describes a policy engine in which a multi-dimensional risk score drives a deterministic

enforcement state machine with allow, deny, sanitize and escalate decisions. Its decision set corresponds closely to REM's four verdicts. It is unrefereed, its full text was not accessed, and its evaluation claims are qualitative.

Consequence-aware control.

- H.-H. Chen (2026; preprint) prices each side-effect-bearing action, such as a payment or refund, against a safe default and gates it against a reserve capital budget. It does not estimate how likely the action is to have been induced.
- SafeAgent's decision core instantiates operators for risk encoding, utility-cost evaluation, consequence modeling and policy arbitration over persistent session state (H. Liu et al., 2026; level A). Whether these operators are realized through LLM reasoning is unverified (level U).
- NEXUS encodes consequence in plan annotations and rules.

Confidence-based escalation. LATTICE separates deciding from judging, so that no component both proposes and approves an action. It escalates to human operators on confidence, and reports zero unsafe actions across four planner families (Calboreanu, 2026). It achieved this at a conservative operating point that auto-allowed no action. In the same study, a confidence-threshold baseline's false-allow rate varied from 0.03 to 0.998 across planners.

Transactional and reversible mitigation.

- Cordon stages outward-facing effects in an outbox and executes reversible mutations in shadow state (Z. Chen et al., 2026).
- GoEX offers undo and damage confinement for post-facto validation (Patil et al., 2024).

Such substrates *consume* a decision and could supply the mechanics of a hold or undo response.

Themes 8 and 9 (decision and consequence handling; mitigation): synthesis.

- **Established.** Graduated responses including human review are common: Progent, RTBAS, LATTICE, AgentTrust, NEXUS, SafeAgent and Jackson. So are modify-type responses (AgentTrust, NEXUS, SafeAgent, ToolSafe) and reversible or staged effects (Cordon, GoEX). Consequence information enters decisions through rules (NEXUS), deterministic pricing (H.-H. Chen), a judge dimension (AgentTrust) or LLM operators (SafeAgent).
- **Uncertain.** How a probability of adversarial induction and a declared consequence should be *combined* when choosing a response. Only NEXUS combines a

calibrated probability with costs, and it does so with fixed intervention costs inside a cascade at the plan level.

- **Under-evaluated.** How often each response is chosen, and how effective it is once chosen. Intervention rates are reported by few studies, notably ProvenanceGuard and the MCP policy-enforcement point. LATTICE's result shows that zero false-allow can be bought with an operating point that auto-allows nothing, so a safety figure must be read alongside intervention behavior.
- **Differs across studies.** The meaning of "escalate": to a human (RTBAS, LATTICE, AgentTrust) or to a stronger LLM judge (FinHarness). The meaning of "modify": argument rewriting, safer alternatives or plan revision. Whether decisions are per call, per plan or per episode.
- **Implication for REM.** REM's verdict set and its Escalate and Modify responses are adopted, not contributed. Its specific decision rule has one partial precedent in NEXUS. REM's evaluation must report intervention rates and benign utility beside attack success. Because the meaning of Modify and Block differs across studies, Chapter 3 specifies their operational semantics and evaluates both a step-level and an episode-level reading.

2.10 Financial AI-Agent Security

The protocol asks that general agent-security evidence be kept separate from financial-agent evidence. A general benchmark is not counted as financial evidence merely because one of its scenarios is financial.

2.10.1 General agent-security evidence with a financial component

Several included general studies contain a financial element but report no finance-specific analysis in the verified record:

- **AgentDojo** includes an executable Banking suite (Debenedetti et al., 2024).
- **ASB** includes finance as one of ten scenarios (H. Zhang et al., 2025).
- **AgentHarm** includes fraud among eleven harm categories (Andriushchenko et al., 2025).
- **ClawSafety** includes finance among five professional domains. Its scenarios include redirecting financial transactions (Wei et al., 2026).
- **ToolEmu** cites financial loss as a risk example (Ruan et al., 2024).
- **RTBAS** cites financial transactions as a motivating example (Zhong et al., 2025).

These studies establish that financial actions are within the attack surface that general benchmarks cover. They do not establish how defenses perform on financial actions specifically.

2.10.2 Financial-agent evidence

Four included studies have a primarily financial setting, and none reaches MODERATE or HIGH EVIDENCE:

- **Castro-Maldonado et al. (2026)**, peer reviewed, study a semantic firewall with online ensemble learning in an agentic retrieval-augmented banking chatbot. The record specifies neither the threat model nor the results (LIMITED EVIDENCE).
- **Z. Chen, J. Chen, et al. (2025)**, a preprint position paper, argue that accuracy and return metrics give an illusion of reliability for financial agents. They propose model-, workflow- and system-level stress testing, illustrated by an audit of six agents on three tasks (LIMITED EVIDENCE).
- **H.-H. Chen (2026)**, a preprint, frames consequence pricing actuarially, with payments and refunds as examples (LIMITED EVIDENCE; partial financial focus).
- **FinHarness** (H. Jia et al., 2026; preprint) is the closest domain-specific runtime system. It combines a query monitor fusing intent with cross-turn drift, a tool monitor for each prospective call, and a cascade routing verification between a lightweight and an advanced LLM judge. It reports attack success falling from 38.3% to 15.0%, with benign approval moving from 41.1% to 39.3%. All of these figures were measured on **FinVault** (Z. Yang et al., 2026). The review record of 25 September 2026 notes that FinVault's arXiv preprint was **withdrawn by its authors**, with a comment citing legal, intellectual-property and affiliation reasons rather than an error in the benchmark; its OpenAlex record carries no retraction flag, and the withdrawal notice could not be re-checked on 28 October 2026 because arXiv was unreachable. A withdrawal for non-scientific reasons does not invalidate measurements made on the benchmark. It does mean that FinHarness's figures cannot be confirmed against a publicly maintained version, so they are reported as author-reported results with that caveat, and FinHarness is appraised LIMITED EVIDENCE by override.

Background SoK. A systematization of LLM-agent security in agentic commerce (Mao et al., 2026) organizes financial threats along five dimensions and derives twelve cross-layer attack vectors. It places *transaction authorization* at the center of the financial threat model. It is a secondary study, cited as background.

2.10.3 Why financial execution raises the requirements

Three properties of financial execution raise the requirements on a runtime layer. The first two are established in the included sources; the third is this chapter's inference from them.

- **Irreversibility.** Some financial actions cannot be recalled. This is encoded in consequence pricing (H.-H. Chen), in irreversibility flags checked by rules (NEXUS), and in transactional staging (Cordon).
- **Attacker-supplied arguments.** Under indirect injection, recipients and amounts can be planted in the content the agent reads, which makes argument provenance especially informative (Section 2.7.2).
- **Heterogeneous consequences.** A balance query and a transfer may both be permitted, yet differ sharply in the harm an induced call could cause. The same probability of adversarial induction therefore need not warrant the same response.

Regulatory motivation. Regulation (EU) 2024/1689 requires that high-risk AI systems allow automatic event logging (Article 12) and effective human oversight, including the ability to intervene or halt (Article 14) (European Parliament & Council of the European Union, 2024). Whether a given financial agent is high-risk is not assessed here. The provisions motivate an audit record for every decision and an explicit route to human review.

2.10.4 Requirements derived for a runtime layer protecting a financial agent

Six requirements follow from Sections 2.4–2.10:

- **R1** pre-execution mediation of each proposed action;
- **R2** provenance evidence on action arguments;
- **R3** consequence-differentiated responses at the same estimated probability;
- **R4** a human-review route;
- **R5** an audit record for every decision;
- **R6** joint measurement of security, benign utility, intervention behavior, calibration and latency.

Theme 10 (financial-agent evidence): synthesis.

- **Established.** Financial actions are within the attack surface of general benchmarks, and a financial runtime harness has been built (FinHarness).
- **Uncertain.** How any defense performs on financial actions specifically. The only quantitative financial runtime results were measured on a benchmark whose

preprint was withdrawn for non-scientific reasons, and cannot be independently confirmed.

- **Under-evaluated.** Every financial-context study is LIMITED EVIDENCE. No included study reports a finance-specific analysis of an open, executable benchmark.
- **Differs across studies.** The financial studies vary in kind: a chatbot firewall, a position paper, an actuarial framework, and an LLM-judge harness.
- **Implication for REM.** Evaluation on AgentDojo's Banking suite, with results reported at the level of financial action types and consequence tiers, would add verified financial evidence where the systematic search found none.

2.11 Comparative Evidence Synthesis

Two tables compare the included runtime defense and design studies. Both are built from the systematic evidence matrix (`CHAPTER2_SYSTEMATIC_EVIDENCE_MATRIX.csv`).

- **Table 2.8** (Appendix 2A) records what each study observes and does, using the columns specified in the review protocol.
- **Table 2.9** records how the studies closest to REM turn evidence into a decision.

Attack, benchmark and analysis studies appear in Tables 2.3 and 2.4.

Coding. Yes = reported in the verified record; NR = Not Reported; NV = Not Verified; NA = Not Applicable. A property that the record does not report is **not** inferred to be present or absent. Financial context is coded Yes only for a primarily financial setting. Evidence levels are HIGH, MOD (moderate) and LIM (limited), as defined in Section 2.2.6.

Table 2.8 (42 defense and design studies × 13 properties) is reproduced in **Appendix 2A**. In summary: 41 of the 42 studies operate at runtime, 34 code tool/action evidence, 21 multi-step handling, 17 behavioral evidence and 8 provenance evidence as present in the record; these counts describe what the records report (Section 2.2.7), not what the systems lack.

Table 2.9. Decision mechanism, calibration, consequence and reported evaluation dimensions (runtime systems closest to REM)

System	Decision basis	Calibrated probability	Consequence used in decision	Human review / graduated response	Benign utility or false interventions reported	Latency reported	Financial focus	Evidence level
Progent	Privilege policy (solver-checked updates)	NA	NR	Approval for privilege expansion	Yes (functionality maintained)	NR	NR	MOD
AgentSpec	Deterministic rules	NA	NR	NV	NV	Yes (ms)	NR	MOD
RTBAS	Information-flow screeners	NR	NR	User confirmation when integrity/confidentiality not ensured	Yes (2% utility loss under attack)	NR	NR	MOD
MCP policy-enforcement point	Declarative rules over cross-step labels	NA	NR	NR	Yes (task-level FP 30.0%; 0/30 rule FPs)	Yes (sub-ms)	NR	HIGH
LATTICE	Policy-as-code; deterministic verdicts; confidence-based escalation	NR (confidence baseline analysed)	NR	Yes (human escalation)	Yes (operating point auto-allowed no action)	Yes (µs–ms)	NR	HIGH
Task Shield	LLM verification of task alignment	NR	NR	NR	Yes (69.79% utility)	NR	NR	HIGH
VIGIL	Intent-grounded verification before commit	NR	NR	NR	Yes (utility under attack)	NR	NR	HIGH (PV)

System	Decision basis	Calibrated probability	Consequence used in decision	Human review / graduated response	Benign utility or false interventions reported	Latency reported	Financial focus	Evidence level
ToolSafe	Learned step-level guardrail with feedback	NR	NR	Feedback to agent	Yes (benign completion)	NR	NR	MOD
Provenance Guard	Evidence support for tool calls	NR	NR	Intervention	Yes (aligned-trace intervention rate; PV)	Yes (higher than baselines; PV)	NR	MOD (PV)
MI9	Governance components; agency-risk index	NR	NR	Graduated containment	NV	Overhead noted (PV)	NV	MOD (PV)
ProbGuard	Probability of remaining safe (learned DTMC) vs threshold	NR (PAC-style bounds on learned model)	NR	NR	Yes (task completion)	NV	NR	MOD
DreamGuard	Calibrated fusion of hazard and prefix-risk scores	Partial (“calibrated fusion”; no calibration metric verified)	NR	HOLD	NV	Yes (~25 ms)	NR	MOD
SafeAgent	LLM operators; policy arbitration	NV	Yes (LLM consequence modelling; PV)	Yes	NV	NV	NR	LIM (PV)

System	Decision basis	Calibrated probability	Consequence used in decision	Human review / graduated response	Benign utility or false interventions reported	Latency reported	Financial focus	Evidence level
AgentTrust	Rules and analysers; worst-case over normalized variants; LLM judge for ambiguity	NR	Partial (reversibility as a judge dimension; PV)	Yes (warn; review)	NR	Yes (~1.72 ms; PV)	NR	MOD (PV)
NEXUS	Rule-first cascade; calibrated logistic score with loss-optimal thresholds	Yes (Platt; ECE 0.085 → 0.013; PV)	Yes (plan annotations used by rules); intervention costs fixed (PV)	Yes (confirm; revise)	Yes (paired benign cases)	Yes (0.205 ms median)	NR	MOD (PV)
FinHarness	Monitors and LLM-judge cascade	NR	NR	Escalation to an advanced LLM judge	Yes (benign approval; benchmark withdrawn for non-scientific reasons)	NR (judge-call counts)	Yes	LIM
H.-H. Chen	Deterministic pricing against reserve capital	NA	Yes	NR	NR	NR	Partial	LIM
Jackson	Multi-dimensional risk score; enforcement state machine	NV	NV	Yes (escalate; sanitize)	NV	Qualitative only	NR	LIM (PV)

System	Decision basis	Calibrated probability	Consequence used in decision	Human review / graduated response	Benign utility or false interventions reported	Latency reported	Financial focus	Evidence level
C. Zhang et al. (analysis)	Action-conditioned intervention value	Analysed (calibration did not reduce regret; PV)	Yes (recoverability)	NA	Regret reported	NR	NR	MOD (PV)
REM (this thesis; design, Chapter 3)	Calibrated per-step probability with consequence-tier losses in an expected-loss rule	Designed	Designed (declared tiers)	Designed (Escalate; Modify)	Exploratory evaluation (Chapter 4)	Measured (Chapter 4)	AgentDojo Banking	—

PV = figure or property unverified (level U: search-engine excerpt only).

Reading the tables.

Row by row, every individual property in REM's design appears somewhere in the included studies:

- calibrated logistic scoring with a four-way intervention set: NEXUS;
- pre-execution interception with graduated verdicts and human review: AgentTrust, RTBAS, LATTICE;
- consequence-dependent control: H.-H. Chen, NEXUS, SafeAgent;
- provenance and cross-step information-flow evidence: ProvenanceGuard, CaMeL, ACE, RTBAS, the MCP policy-enforcement point;
- trajectory-aware monitoring: MI9, PRISM, ProbGuard, DreamGuard, ToolSafe, AgentTrust;
- latency and component ablation: AgentTrust, NEXUS, the MCP policy-enforcement point;
- a financial runtime harness: FinHarness;
- the limits of calibration and confidence thresholds for control: C. Zhang et al.; LATTICE.

Column by column, the combinations thin out:

- **NEXUS** alone reports a calibrated probability together with costs and a graduated response. It does so at the plan level, on author-generated templates, with fixed intervention costs setting thresholds inside a rule-first cascade.
- **Provenance evidence**, where its contribution has been measured, feeds rules (the MCP policy-enforcement point) or a detector (ProvenanceGuard), not a probability estimator.
- **Financial focus** appears only in LIMITED EVIDENCE studies.
- **No included record** reports all three of the following together:
    - a calibrated probability of adversarial induction per proposed tool call;
    - consequence-dependent losses used to select among graduated responses;
    - evaluation on an executable financial environment with utility, intervention behavior and latency reported.

  This observation concerns the *records* examined. A combination this specific is easy to miss in a record-level review and is weak evidence of originality on its own (Section 2.13.2).

2.12 Critical Synthesis

This section draws the thematic syntheses together (Table 2.10) and then states what the systematic evidence does and does not support.

Table 2.10. Thematic synthesis of the included evidence

Theme	What the literature establishes	What is uncertain	What is under-evaluated	What differs across studies	Implication for REM
1 Threats	Indirect injection is demonstrated, benchmarked and adaptive; defenses of the time were bypassed	Magnitude: 24% to 84.30% average ASR across benchmarks	Tool-metadata, tool-library, skill and MCP-server threats against runtime defenses	Threat actor and success criterion	Focus on indirect injection; state other classes as out of scope
2 Entry points	The observation channel (tool outputs, retrieved content, tool metadata) is the decisive entry for tool-using agents	Relative danger of channels (skills > email/web in one study)	Defenses against setup-time channels	Which channel each benchmark exercises	Mediate at the action, where all channels converge
3 Observation granularity	Input, call, plan, data-flow and trajectory units all implemented	Which unit gives the best security–utility trade-off	Head-to-head comparisons on a shared workload	Unit observed; question asked	Per-call observation with provenance context is a defensible, not novel, choice
4 Runtime intervention	Interception at the tool boundary is widespread (41/42 runtime; 34/42 tool/action) and fast	Robustness to adaptive attacks	Evaluation on benchmarks the authors did not build	Permission vs. alignment vs. provenance vs. harm	REM cannot claim interception
5 Behavioral / trajectory evidence	Many monitors model sequences (17/42 behavioral; 21/42 multi-step)	Contribution: one null ablation (session tracker)	Contribution inside a probabilistic estimator; financial settings	Automata, Markov chains, latent states, heuristics	Measure and report the contribution; do not assume it
6 Provenance / goal–action consistency	Informative against injected or misaligned calls; cross-step labels had a 26.4-point effect in one study	Whether the effect generalizes beyond the authors' datasets	Provenance as input to a calibrated estimator; financial arguments	Obtained by analysis, construction or labels	Use as estimator input; test the contribution

Theme	What the literature establishes	What is uncertain	What is under-evaluated	What differs across studies	Implication for REM
7 Statistical / algorithmic methods	Logistic scoring, calibration, expected cost, reject option and attribution are established; one agent-safety precedent (NEXUS)	Behaviour in combination at the action boundary	Calibration of agent-safety probabilities; calibration vs. control	How probabilities are produced	Adopted methods; report calibration separately from control
8 Decision and consequence	Graduated responses and consequence information are common	How to combine an induction probability with a declared consequence	Intervention rates; per-response effectiveness	Fixed costs, pricing, rules, judge dimensions	Test the combination against a same-estimator consequence-independent baseline
9 Mitigation	Block, modify, escalate, restrict, roll back and undo all implemented	Effectiveness of each response once chosen	Residual risk after modify-type responses	Meaning of “escalate” (human vs. stronger judge) and “modify”	Report residual attack success and escalation rate; SC-1 and SC-2 remain open
10 Financial evidence	Financial actions lie within general benchmarks’ attack surface; one financial harness exists	Defense performance on financial actions	Every financial-context study is LIMITED EVIDENCE; key figures were measured on a benchmark withdrawn for non-scientific reasons and cannot be confirmed	Chatbot firewall, position paper, actuarial framework, judge harness	Report results per financial action type and consequence tier
11 Evaluation quality	Results are benchmark-specific; many benchmarks are built with the defense they evaluate	Multi-step content of benchmarks	Independent evaluation; decomposition into security, utility, intervention, calibration and latency	Metrics and baselines	Per-benchmark reporting with benign utility, intervention rates, calibration and latency

What is well addressed.

- Threats are well characterized and benchmarked.
- Interception at the tool-call boundary is implemented in many forms, and deterministic enforcement is fast.
- Graduated responses, including human review, are standard.
- Architectural defenses offer formal guarantees for the threats they formalize.
- Calibration methods are mature and have one agent-safety application.

What remains fragmented. The pieces of a probabilistic, consequence-aware decision exist, but in different studies:

- a calibrated score with fixed-cost thresholds in a plan-level cascade (NEXUS);
- deterministic consequence pricing that ignores how an action arose (H.-H. Chen);
- consequence reasoning realized by LLM operators (SafeAgent);
- provenance evidence feeding rules or a detector (the MCP policy-enforcement point; ProvenanceGuard);
- financial focus in a judge cascade evaluated on a benchmark that is no longer publicly maintained (FinHarness).

Conflicting evidence, presented rather than resolved.

- **Value of trajectory evidence.** The AgentTrust ablation reportedly found no effect of a session tracker (level U). The MCP policy-enforcement point found a large effect of cross-step labels. The kind of trajectory evidence differs, and both are single studies.
- **Architectural guarantees.** IsolateGPT reports protection against the attacks it tested, and ACE later defeats it with new attacks.
- **Calibration as a remedy.** NEXUS reports a large ECE improvement. C. Zhang et al. report that a similar improvement left control regret unchanged. LATTICE reports that a confidence threshold's false-allow rate ranged from 0.03 to 0.998 across planners.
- **Safety figures versus utility.** LATTICE's zero false-allow was obtained at an operating point that auto-allowed no action. The MCP policy-enforcement point's attack-success reduction came with a 30.0% task-level false-positive rate. A safety figure alone is therefore not evidence of a usable defense.

Where current approaches stop.

- Most detectors and guardrails stop at a verdict and do not produce a probability that could be traded off against consequence.
- Most consequence-aware designs stop at the consequence and do not estimate the likelihood of adversarial induction.

- Most trajectory-aware systems stop at detection. The two ablations disagree about whether the trajectory component changes outcomes.
- Most evaluations stop at attack success: benign utility, intervention behavior, calibration and latency are reported unevenly.

What the evidence does not support. The systematic evidence does **not** support three claims that earlier drafts could have been read as making:

- that runtime interception, graduated verdicts or consequence awareness are absent from the literature;
- that combining several signals at the action boundary is rare. AgentTrust, the MCP policy-enforcement point, ToolSafe, DRIFT and FinHarness all combine signals at or near the tool call;
- that trajectory evidence is known to be ineffective. The evidence is mixed.

2.13 Research Gap and REM Positioning

2.13.1 Assessment of candidate gaps

The protocol listed five candidate gaps, A to E, to be retained only if the systematic evidence supports them. Table 2.11 records the assessment.

Table 2.11. Evidence-based assessment of candidate research gaps

Candidate gap	Evidence for	Evidence against	Decision
A. Integration. Capabilities exist across separate approaches	Tables 2.8–2.9: calibrated scoring (NEXUS), consequence pricing (H.-H. Chen), provenance (ProvenanceGuard; MCP policy-enforcement point), graduated verdicts (AgentTrust; LATTICE) are in different studies	NEXUS combines calibration and graduated responses (level A) and, per level-U evidence, costs, at the plan level	**Retained only as a weak, record-bounded observation.** A calibrated per-call probability combined with consequence-indexed losses at the action boundary was not reported in the examined records. Because the combination is specific and the review is record-level, its absence is not treated as evidence of originality
B. Runtime action-boundary evaluation. Limited evidence combining multiple signals at the point of action	—	AgentTrust, the MCP policy-enforcement point, ToolSafe, DRIFT, SafeAgent and FinHarness combine several signals at or near the tool call	**Not retained as stated.** Its defensible residue, combination into a calibrated probability, is part of Gap A
C. Behavioral/provenance contribution. Insufficient evidence separating the contribution of behavioral and provenance signals	Only three component-level tests found; they conflict (null session-tracker ablation vs. a 26.4-point cross-step-label effect); none inside a probabilistic estimator; none financial	The MCP policy-enforcement point does separate the contribution of cross-step labels in a rule-based enforcer (HIGH EVIDENCE)	**Retained, narrowed.** Limited and conflicting evidence on the contribution of behavioral and provenance evidence *as inputs to a calibrated estimator* for financial actions under indirect injection
D. Financial-agent evidence. Limited verified evidence in executable financial settings	Four financial-context studies, all LIMITED EVIDENCE; the only quantitative runtime results cannot be confirmed against a maintained benchmark; general benchmarks with financial suites report no finance-specific analysis in the verified record	FinHarness shows the problem is recognized	**Retained**

Candidate gap	Evidence for	Evidence against	Decision
E. Evaluation decomposition. Insufficient evaluation separating security, benign utility, calibration or decision quality, intervention behavior and latency	Calibration reported for one deployed scorer (NEXUS); intervention rates by few studies; no study compares a consequence-aware decision with a consequence-independent baseline that shares its estimator	Several studies report more than attack success (MCP policy-enforcement point; LATTICE; NEXUS; ToolSafe)	**Retained, narrowed.** The missing element is the same-estimator comparison, with calibration reported separately from control and intervention rates reported

2.13.2 Statement of the research gap

The gap supported by the examined evidence is **empirical**. In the records of the 71 included studies (search date 25 September 2026), the review found (a) limited and conflicting evidence on whether behavioral and provenance evidence improves runtime decisions, and none testing it as input to a calibrated estimator for financial actions (Gap C); (b) no verified quantitative evaluation of a runtime defense on financial actions in an open, executable benchmark (Gap D); and (c) no evaluation that compares a consequence-aware decision with a consequence-independent baseline sharing the same estimator, while reporting attack success, benign utility, intervention behavior, calibration and latency separately (Gap E).

Nature of the gap. The gap concerns *evidence*: how established mechanisms behave when combined at the action boundary of a financial agent under indirect prompt injection, measured against a baseline that isolates the decision rule. It does not concern the absence of any single mechanism, and it does not concern a new learning algorithm. The specific combination that REM implements (Gap A) was not found in the examined records, but the thesis does not rest its originality on that absence: a combination this specific is easy to miss in a record-level review, and closely related designs exist (NEXUS, AgentTrust, SafeAgent).

Bounds. The gap is bounded by the search, which covered one database with a retrieval cap and a partly non-independent other-methods stream. It is also bounded by record-level appraisal: no study was read in full, and several properties of the closest studies are unverified (Section 2.13.4).

Falsifiability. A study that reports this combination and evaluation would require the contribution to be revised.

Consistency with Chapter 3. The statement is consistent with Chapter 3, §3.13, which describes the contribution as empirical and integrative.

2.13.3 REM positioning: established by prior research versus proposed by this thesis

REM is a runtime mediation layer between a financial tool-using agent and its environment. It is organized in exactly five layers:

1. **Input & Context**;
2. **Detection**;
3. **Behavioral Analysis**;
4. **Decision Engine**;
5. **Mitigation**.

Chapter 3 specifies it. Table 2.12 separates what REM adopts from prior research from what this thesis proposes.

Table 2.12. Established by prior research versus proposed by this thesis

Element	Established by prior research (included or foundational sources)	Role in REM
Runtime interception of each proposed tool call	Progent; AgentSpec; AgentTrust; RTBAS; MCP policy-enforcement point; LATTICE; PRISM; SafeAgent; Task Shield	Adopted, not claimed
Provenance / goal–action consistency evidence	ProvenanceGuard; Task Shield; MELON; CaMeL; ACE; RTBAS; MCP policy-enforcement point	Adopted as estimator input, not claimed
Behavioral / trajectory monitoring	MI9; ProbGuard; DreamGuard; PRISM; AgentTrust; ToolSafe; Forrest et al. (1996); Chandola et al. (2009, 2012)	Adopted, not claimed
Ridge logistic risk scoring	Cox (1958); le Cessie & van Houwelingen (1992); NEXUS	Adopted, not claimed
Platt calibration on the logit	Platt (2000); Guo et al. (2017); NEXUS	Adopted, not claimed. Beta, isotonic and temperature calibration are reviewed and **not adopted**
Minimum expected-cost decision; reject option	Elkan (2001); Chow (1970); NEXUS objective	Adopted, not claimed. Neither source proposed REM's four-action architecture
Consequence-dependent control	H.-H. Chen (2026); NEXUS; SafeAgent; AgentTrust	Adopted as declared consequence tiers, not claimed
Graduated responses including human review	AgentTrust; NEXUS; RTBAS; LATTICE; SafeAgent; Jackson; MI9	Adopted, not claimed
Deterministic mitigation; reversible and staged effects	Progent; AgentSpec; Cordon; GoEX	Adopted, not claimed
Sequential change detection (CUSUM)	Page (1954); Basseville & Nikiforov (1993); Lorden (1971)	Optional, offline, observe-only analysis outside the verdict path; not claimed
Additive attribution for audit	Lundberg & Lee (2017); audit traces in NEXUS, AgentTrust, PRISM, LATTICE	Audit record only; never used for the verdict; not claimed

Element	Established by prior research (included or foundational sources)	Role in REM
Combination of provenance evidence, a calibrated per-step probability, tier-indexed expected-loss verdicts and deterministic mitigation at the action boundary, on an open executable banking benchmark	Not reported in the examined records (record-level; weak evidence of absence)	**Proposed by this thesis (integrative design)**
Evaluation that separates attack success, benign utility, intervention behavior, calibration and latency, against a consequence-independent baseline with the same estimator	Not reported in the examined records	**Proposed by this thesis (empirical)**

Relation to AgentTrust and NEXUS. What REM cannot claim against AgentTrust and NEXUS, and the narrower differences that remain, are set out in Sections 2.9.2 and 2.9.3 and are not repeated here. Several of the properties on which those differences rest are unverified (level U).

Relation to other close studies. REM does not claim:

- to be the first financial runtime protection (FinHarness);
- a risk-score-to-graduated-verdict pipeline (Jackson; SafeAgent);
- deterministic consequence pricing (H.-H. Chen);
- provenance analysis (ProvenanceGuard; the MCP policy-enforcement point);
- that calibration improves control (C. Zhang et al.; LATTICE).

2.13.4 Limitations of this review

The review is systematic in procedure but **not exhaustive**, and its conclusions are bounded as follows.

1. **Database accessibility.** Only OpenAlex was searched. IEEE Xplore, the ACM DL, Scopus and Web of Science, ScienceDirect, SpringerLink, the ACL Anthology, NeurIPS/PMLR, ICLR/OpenReview, USENIX and arXiv could not be reached (DATABASE ACCESS UNAVAILABLE). Elicit access was also unavailable. Their content was reached only through OpenAlex's index.
2. **Index coverage, ranking and retrieval cap.** OpenAlex's lexical relevance ranking surfaced general surveys ahead of specific recent preprints. Only the top 24–25 records per query page were screened, although index estimates ran to thousands.

The database search retrieved only 3 of the 13 named prior-art studies, which shows that relevant studies can be missed.

3. **Non-independent second stream.** Thirty-one included studies came from the reference list of the previous chapter, which reflects the author's prior reading. This introduces selection bias, which the protocol's requirement to treat named prior art identically mitigates but does not remove.
4. **No full texts.** Eligibility, extraction and appraisal were performed on records; no full-text screening was performed and no claim is at level F. Nine included studies are PARTLY UNVERIFIED. Appraisal items that depend on full text (reproducibility, comparators, limitations) are mostly Not Verified. A supplementary record check on 28 October 2026 re-examined the closest studies (NEXUS, AgentTrust, ProvenanceGuard, FinHarness, FinVault, SafeAgent, C. Zhang et al., H.-H. Chen): it found a peer-reviewed version of ProvenanceGuard (ASE 2026), a changed title for C. Zhang et al., and several details of NEXUS and AgentTrust that are not supported by their current abstracts, which are now marked level U. Full texts remained unreachable.
5. **Single reviewer, AI-assisted.** Screening, extraction and appraisal were each done by one reviewer, assisted by an AI coding assistant, with no independent second screener and no agreement statistic. The appraisal checklist is transparent but is not a validated risk-of-bias instrument.
6. **Rapidly changing preprint literature.** 35 of the 71 included studies are preprints or working papers, and the field produces new preprints monthly. One source (FinVault) was recorded as withdrawn by its authors for non-scientific reasons. Preprint results may change or be withdrawn.
7. **Publication bias.** Defenses are usually reported on the authors' own benchmarks with favorable configurations. Negative results appear mainly in attack papers.
8. **Heterogeneity and no pooling.** Architectures, threats, benchmarks and metrics differ too much for meta-analysis. The synthesis is qualitative, and numeric comparisons across studies are avoided.
9. **Metadata errors in the index.** One record was corrupted (AgentDojo's DOI attached to another abstract) and one venue year was inconsistent (PromptShield). Both were detected and recorded, but others may have gone unnoticed.

Before submission. The protocol's Boolean blocks should be run directly in at least IEEE Xplore, the ACM DL, Scopus or Web of Science, and arXiv. Full texts of the included studies should be appraised, and the PRISMA flow and gap assessment updated.

2.13.5 Prior-art risks to monitor

1. **NEXUS.** If a later version evaluates per tool call on an executable benchmark, or makes intervention losses depend on each action's consequence, Gap A largely closes.
2. **AgentTrust.** If a later version reports calibrated probabilities or consequence-weighted decisions, Gap A narrows. AgentTrust shares its name with another arXiv preprint (2606.08539) and must not be confused with it.
3. **MCP policy-enforcement point and LATTICE.** Both are peer-reviewed, HIGH EVIDENCE studies of action-boundary enforcement with ablation or baseline analysis. A probabilistic extension of either would narrow Gaps A, C and E.
4. **FinHarness.** A re-evaluation on an open executable benchmark with a probabilistic decision layer would narrow Gap D.
5. **Jackson (2025).** The full text was not accessed. If it documents calibrated, consequence-weighted selection with evaluation, Gaps A and E narrow.
6. **Unsearched databases.** Studies in the libraries that could not be searched may already report the combination stated in Section 2.13.2.

2.13.6 Currency update of 30 October 2026 (outside the PRISMA flow)

The registered search ran on 25 September 2026. On 30 October 2026 the eleven queries of Table 2.1 were re-run in the same index (OpenAlex, through the same literature-search tool), restricted to publication year 2026, to find work that could change the gap assessment. **This update is not part of the PRISMA flow, and no count in Table 2.2 or Figure 2.1 changes:**

- the index returns the publication year but not the date, so records published after 25 September could not be separated reliably from earlier 2026 records;
- of the 215 unique records returned, 200 were not in the stored output of the registered search; they were title-screened by one reviewer for direct relevance to REM's decision mechanism or evaluation design, and were not assessed against criteria I1–I6;
- no record was added to the 71 included studies.

The raw output is stored with the thesis verification records (`update_search_2026-10-30.json`). Five records bear directly on REM's positioning. Each is described only as far as its index record states (level A); no full text was read.

Record	Type	What the record states	Bearing on REM
Kekatos et al. (2026)	Peer-reviewed workshop paper (ACM, Crossref-registered)	Replays the recorded trajectories distributed with AgentDojo, STAC and R-Judge through a temporal-logic monitor, offline and without running an agent. Five generic obligations flag 70.1% of successful AgentDojo attacks and fire on 29.3% of benign runs. A planted line defeats a naive provenance check on 94–99% of the runs it would otherwise flag	Offline evaluation on AgentDojo's recorded traces has a published precedent, so it is not a methodological contribution of this thesis. Provenance evidence is itself attackable
Z. Liu (2026)	Preprint	Replays the ground-truth tool calls of AgentDojo and tau-bench without an LLM and evaluates fifteen prompt-injection detectors, including Prompt Guard 2. Rankings transfer poorly: the best detector on BIPIA catches 2% of AgentDojo injections at a 1% false-positive rate	The choice of the injection classifier x^{ctx}_1 (pending) should be evaluated on the agent's own tool outputs at a low false-positive rate
Ray (2026)	Preprint	Once blocking changes future proposals, static scores and ungated trajectories need not identify the closed-loop frontier; benign calibration alone does not transfer	Theoretical support for the limit of REM's replay analysis: replay of recorded trajectories cannot establish closed-loop enforcement
Yan et al. (2026)	Preprint	A controller that acts after the agent proposes an action and before it executes; it encodes trusted and external observations in provenance-typed memory and commits the action, proposed or alternative, with the highest calibrated value learned from outcomes	The closest record found to REM's combination of provenance evidence and a calibrated decision at the action boundary. It differs, by its abstract, in choosing between actions by learned value rather than among four verdicts by declared, tier-indexed losses. Prior-art risk to Gap A

Record	Type	What the record states	Bearing on REM
Xin (2026)	Journal article	A pre-execution router with temperature-scaled three-class probabilities (execute, confirm, block), a fixed rule floor and confidence-triggered confirmation, evaluated on mobile-agent tasks that include financial applications	Calibrated multi-way routing with a confirmation verdict before execution has a further precedent. REM differs in deriving its thresholds from declared losses per consequence tier

The other records of the update concern, among other topics, enforcement at the execution boundary, banking-agent benchmarks and step-labeled injection trajectories. They were not assessed and should be screened in the full update recommended in Section 2.13.4.

Effect on the gap. At abstract level, none of the five records reports the combination and evaluation stated in Section 2.13.2, but Yan et al. (2026) and Xin (2026) narrow the distance to it, and Kekatos et al. (2026) removes any claim of methodological novelty for offline evaluation on recorded AgentDojo traces. Because only records were examined, this conclusion has the same limits as the main review.

2.13.7 Supplementary literature update of 2 November 2026: Kanaker et al. (2026) (outside the PRISMA flow)

Status of this addition. Kanaker et al. (2026), *Securing Tool-Using AI Agents Against Injection and Authority Misuse* (*Computation*, 14(5), Article 98), is added on 2 November 2026 as a dated supplementary literature update. The record was returned by query Q7 of the currency update of 30 October 2026 (OpenAlex W7160053090, stored in `update_search_2026-10-30.json`) but was not among the five records selected at title screening in Section 2.13.6. It was not part of the registered search of 25 September 2026. **No count in Table 2.2 or Figure 2.1 changes, it is not added to the 71 included studies, and no screening record is created retrospectively.**

Evidence level. The bibliographic metadata were checked against the Crossref record of the DOI (10.3390/computation14050098): six authors in the order H. Kanaker, H. Fakhouri, N. Abdel Karim, M. Abuhamdeh, N. H. A. Ismail and S. Fakhouri; *Computation*, volume 14, issue 5, article 98; 2026. The abstract was read in the OpenAlex record. The full text could not be retrieved in this revision, because the publisher and DOI hosts were refused by the network policy. Every statement below is therefore at **level A**: it

reports what the abstract states, not what the paper's methods or results sections contain. Parameter values, the number of simulation runs, uncertainty estimates and the exact definitions of the metrics were not read.

What the study does (level A). The study argues that large-scale testing of deployed agents is difficult, vendor-specific and ethically sensitive, and proposes a transparent, theoretical simulation framework for user-facing risk in tool-using agents. Its stated methodological contribution is a formal threat model that **separates compromise, harm and severity**, together with a Monte Carlo pipeline that maps architectural choices (permissions, retrieval, memory exposure and approvals) and defensive controls to comparable outcome metrics. It instantiates the framework for six threat scenarios and nine defense configurations, and reports attack success rate, benign task success, latency overhead and severity-weighted harm. Its abstract reports four findings:

- least-privilege tool design is the strongest single broad control;
- human-in-the-loop approvals sharply reduce high-impact actions and exports, but degrade under user error and habituation;
- retrieval allowlisting nearly eliminates indirect injection while leaving other channels largely unaffected;
- rate limiting reduces tail severity more than attack success rate.

Critical assessment.

- *Strengths visible in the record.* The threat model makes the distinction between whether an agent is compromised and how much harm follows explicit. The study reports more than attack success, including benign task success, latency and a severity-weighted outcome, and it compares architectural controls side by side. Its assumptions are declared rather than hidden.
- *Limits stated by the authors.* The results come from an assumption-explicit simulator, not from field measurements, and the authors present them as comparative design guidance rather than incident-rate estimates for any deployed product. They do not measure how a particular agent or language model behaves.
- *Limits of this review.* Whether the simulated risks are calibrated to observed agent behavior, how sensitive the rankings are to the simulator's parameters, and how severity is weighted could not be assessed without the full text.
- *Consistency with other evidence in this chapter.* The approval finding agrees with the approval-fatigue trade-off reported by P. Wang et al. (2026) (Section 2.9.1). The allowlisting finding concerns one injection channel, whereas the provenance-based defenses of Section 2.6.5 examine the content that actually reaches a

proposed call. The rate-limiting finding illustrates why attack success rate alone can miss differences in the severity of outcomes.

Bearing on the gap assessment (Table 2.11). The study adds to the "evidence against" column of Gap E: another study reports security, benign success, latency and a severity-weighted outcome together, although in simulation. It does not, by its abstract, compare a consequence-aware decision with a consequence-independent baseline that shares its estimator, so the narrowed residue of Gap E is unchanged. For Gap D, the study evaluates transaction-capable tools in a simulator rather than in an open, executable benchmark, so the retained statement is not contradicted. For Gap A, the abstract describes no calibrated per-call probability and no selection among verdicts by expected loss. These conclusions have the same record-level limits as the rest of the review.

Table 2.13. Kanaker et al. (2026) and REM compared (Kanaker et al.: level A, abstract and metadata only; REM: Chapters 3–4)

Dimension	Kanaker et al. (2026)	REM (this thesis)
Research question	How architectural choices and defensive controls change user-facing risk in tool-using agents, evaluated without testing deployed products	Whether consequence-tiered expected-loss verdicts at the tool-execution boundary change intervention, attack success and utility relative to a consequence-independent threshold with the same estimator (Section 1.7; [PD])
Method	Formal threat model separating compromise, harm and severity; Monte Carlo evaluation pipeline over architectural choices and defenses	Provenance, behavioral and action evidence; ridge logistic regression; logistic calibration on the logit; consequence tiers; expected-loss verdict among Allow, Modify, Escalate and Block; deterministic mitigation; audit record
Data and evidence	Assumption-explicit simulation; six threat scenarios; nine defense configurations	Recorded AgentDojo banking traces of 21 language-model agents; held-out predictions, observe-only decisions and replay under a no-adaptation assumption; no live runs (Chapter 4)
Treatment of harm and severity	Severity separated from compromise in the threat model; severity-weighted harm reported as an outcome	Consequence tier k_t separated from the probability p_t; severity enters the decision through tier-indexed declared losses (Eqs. 3.5–3.9); realized loss reported (Eq. 3.22)
Defensive mechanisms	Least-privilege tool design, human-in-the-loop approvals, retrieval allowlisting, rate limiting (abstract); the full set of nine configurations was not read	Per-call verdicts with step-level withholding of the current call, an episode-level restriction variant, and escalation evaluated under an idealized-reviewer and a no-reviewer bound
Reported outcomes	Attack success rate, benign task success, latency overhead, severity-weighted harm	Discrimination and calibration, intervention rates, attack success rate, benign utility, utility under attack, processing time, realized loss
Limitations	Simulator rather than field measurement; results are comparative design guidance, not incident-rate estimates (stated by the authors)	Exploratory pilot; recorded traces and replay only; unaudited proxy labels; declared, unapproved costs; one benchmark (Chapter 4, Section 4.16)

What is and is not taken from this study. The study is cited where it is consistent with existing positions of this thesis: the separation of compromise from severity (Chapter 3, Section 3.7.3) and the optimism of a perfect-reviewer assumption (Chapter 3, Section 3.7.5). REM's design was fixed before this record was examined, and no REM mechanism,

parameter or experiment is derived from it. Its simulation results are not used as evidence for REM's effectiveness, and REM's results are not used to assess its findings.

2.14 Chapter Summary

This chapter reported a record-level systematic review, reported following PRISMA 2020 and PRISMA-S, of runtime security for tool-using AI agents, with attention to financial execution. Full texts were not retrieved, so its findings rest on records.

Selection. The search of OpenAlex on 25 September 2026 identified 320 records. (A simulation study added in a supplementary update of 2 November 2026 is discussed separately at abstract level in Section 2.13.7; it does not change these counts.) After deduplication, 219 were screened, and 45 were assessed for eligibility at record level. Forty database studies were included. Another 31 came from the verified reference list of the previous chapter, giving 71 included studies. Of these, 10 are HIGH, 36 MODERATE and 25 LIMITED record-visible evidence.

Themes. The synthesis established the following:

- **Threats.** Indirect prompt injection through the observation channel is the best-evidenced threat to tool-using agents, and it is adaptive. Its harm materializes at the proposed tool call.
- **Defenses.** Runtime interception, deterministic enforcement, information-flow architectures, learned guardrails and provenance checks are all established. Adaptive attacks have repeatedly broken defenses, including architectural ones.
- **Behavioral and provenance evidence.** Many monitors observe trajectories. Evidence on their contribution is sparse and conflicting: a null ablation of a session tracker, and a large effect of cross-step information-flow labels.
- **Algorithmic foundations.** Logistic scoring, Platt calibration, expected-cost decisions with a reject option, and additive attribution are established. One agent-safety system (NEXUS) combines calibration with costs at the plan level. Calibration or a confidence threshold alone did not secure good control in two independent studies. CUSUM is reviewed only as an optional, observe-only analysis.
- **Decision and mitigation.** Graduated responses with human review are standard. How to combine a probability of adversarial induction with a declared consequence remains open. Intervention rates are rarely reported.

- **Financial agents.** All financial-context evidence is LIMITED, and the only quantitative financial runtime results cannot be confirmed against a maintained benchmark.

The gap. The research gap supported by the examined records is **empirical**: it concerns the evaluation, on an open, executable banking benchmark, of a runtime layer that combines:

- provenance-based action evidence;
- a calibrated per-step probability;
- consequence-tiered expected-loss decisions;
- deterministic mitigation.

The evaluation must measure attack success, benign utility, utility under attack, intervention behavior and latency separately, and report calibration separately from control, against a consequence-independent baseline that uses the same estimator. Every mechanism in this combination is established. Only the combination at the action boundary and the evaluation design are proposed by this thesis.

Chapter 3 specifies the REM methodology through which the gap is addressed.

Appendix 2A. Comparative Evidence Matrix of Defense and Design Studies

Coding as in Section 2.11: Yes = reported in the verified record; NR = Not Reported (not evidence of absence); NV = Not Verified; NA = Not Applicable. Evidence levels refer to record-visible evidence (Section 2.2.6).

Table 2.8. Comparative evidence for included defense and design studies (n = 42)

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
C. L. Wang et al. (2025) [P59]	Emergent/ unsafe agent behaviour (governance)	NA	Agent state and actions over time	Yes	Yes	Yes	Yes	NR	Yes	Yes (graduated containment)	NV	MOD
C. Yang (2026) [P64]	Unsafe tool-use side effects	Proposed tool call	Each proposed tool call (plus session chains)	Yes	Yes (seven chain detectors)	Yes	Yes (session tracker; no measurable effect in ablation)	NR	Yes	Yes (allow/warn/block/review; safer alternatives)	NR	MOD
Calboreanu (2026) [P40]	Unauthorised actions	NA	Proposed action at gated execution path	Yes	NR	Yes	NR	NR	Yes	Yes	NR	HIGH

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Castro-Maldonado et al. (2026) [P32]	Customer-data protection / security (threat model not specified in abstract)	User queries and retrieved content	Query/response	Yes	NR	NV	NR	NR	Yes	NV	Yes	LIM
Chennabasappa et al. (2025) [P16]	Prompt injection; jailbreak; agent misalignment; insecure code	Untrusted inputs; agent reasoning; generated code	Inputs and chain-of-thought	Yes	Yes	NV	NV	NR	Yes	NV	NR	LIM
Debenedetti et al. (2025) [P52]	Prompt injection; data exfiltration	Untrusted data	Program control and data flow	Yes	NR	Yes	Yes (data flow across steps)	Yes	Yes	Yes	NR	MOD

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
E. Li et al. (2026) [P12]	Malicious apps; indirect prompt injection; planning/execution integrity and availability	Third-party apps and their outputs	Abstract plan, concrete plan and execution	Yes	NR	Yes	Yes	Yes	Yes	Yes	NR	MOD
F. Jia et al. (2025) [P01]	Indirect prompt injection	External data returned by tools	Each instruction and tool call	Yes	NR	Yes	NV	NR	Yes	NV	NR	HIGH
F. Li (2026) [P60]	IPI, unsafe tools, credential leakage, tampering	Tool inputs/outputs; files; credentials	Lifecycle hook events within a session	Yes	Yes (session risk accumulation)	Yes	Yes	NR	Yes	Yes	NR	LIM

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
F. Wu, Cecchetti, & Xiao (2024) [P08]	Indirect prompt injection	Any untrusted source feeding the planner	Planning inputs and structured plans	Yes	NR	Yes	Yes	Yes	Yes	Yes	NR	LIM
H. Jia et al. (2026) [P71]	Prompt-induced unauthorised financial actions	User queries; tool calls	Query and tool call with cross-turn state	Yes	Yes (partial: cross-turn drift, accumulated risk)	Yes	Yes	NR	Yes	Yes (block/approve; escalation to advanced judge)	Yes	LIM
H. Li et al. (2025) [P38]	Prompt injection via external tools	Tool outputs; memory stream	Function trajectory vs. planned trajectory	Yes	Yes	Yes	Yes	NR	Yes	Yes	NR	MOD
H. Liu et al. (2026) [P63]	Prompt injection propagating through workflows	Inputs and intermediate context	Stateful workflow	Yes	Yes	Yes	Yes	NR	Yes	Yes	NR	LIM

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
H. Wang et al. (2025) [P61]	Safety-specification violations	NA	Abstract state sequence	Yes	Yes	Yes	Yes	NR	Yes	Yes	No (non-financial)	MOD
H. Wang, Poskitt, & Sun (2026) [P50]	Unsafe actions	NA	Triggered action vs. rule predicate	Yes	NR	Yes	NV	NR	Yes	Yes	NR	MOD
H.-H. Chen (2026) [P67]	Consequences of side-effecting actions	NA	Each priced action	Yes	NR	Yes	NR	NR	Yes (consequence pricing vs. reserve)	Yes (gating)	Yes (partial: payments, refunds as examples)	LIM
Hines et al. (2024) [P47]	Indirect prompt injection	Untrusted documents	Input text	Yes	NA	NA	NA	Yes	NA	NA	NR	MOD

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Hossain et al. (2026) [P65]	Unsafe plans	Proposed plan and arguments	Plan (with cross-turn session state)	Yes	Yes (partial: session manager)	Yes (argument-level)	Yes (partial: plan-level)	NR	Yes	Yes (allow/block/confirm/revise)	NR	MOD
Hua et al. (2024) [P55]	Unsafe planning	NA	Plan	Yes (planning stage)	NR	Yes (plan-level)	Yes (plan-level)	NR	NV	Yes (plan revision)	NR	LIM
Jackson (2025) [P66]	Governance violations	NA	Governed action	Yes	NV	Yes	NV	NR	Yes (multi-dimensional risk score)	Yes (allow/deny/sanitize/escalate)	NR	LIM
Jacob et al. (2025) [P23]	Prompt injection	Conversational and application-structured inputs	Input text	Yes	NR	NR	NR	NR	Yes	NR	NR	MOD

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Jiang et al. (2025) [P56]	Unsafe intermediate thoughts	NA	Intermediate thought before action	Yes	Yes (thought-level)	NR	Yes (partial)	NR	NV	Yes (thought correction)	NR	MOD
J. Lin et al. (2026) [P11]	Tool-stream injection (manipulated tool metadata and runtime feedback)	Tool metadata and tool feedback	Speculative action before commit	Yes	NR	Yes	NV	NR	Yes	Yes	NR	HIGH
W. Lin et al. (2026) [P62]	Long-horizon hazardous drift	NA	Each call with trajectory prefix	Yes	Yes	Yes	Yes	NR	Yes	Yes (PASS/HOLD/BLOCK)	NR	MOD
Luo et al. (2025) [P39]	Task-specific and systemic risks	NV	Agent actions	Yes	NV	Yes	NR	NR	Yes	NV	NR	LIM

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Mou et al. (2026) [P33]	Unsafe tool invocations incl. prompt injection	Requests and tool interaction history	Each tool invocation with history	Yes	Yes	Yes	Yes	NR	Yes	Yes	NR	MOD
Patil et al. (2024) [P10]	Incorrect or unsafe LLM-generated actions (non-adversarial)	NA	Executed action	Yes	NR	Yes	NR	NR	NV	Yes	NR	LIM
S. Chen et al. (2025) [P48]	Prompt injection	Data channel	Model input	NA	NA	NA	NA	Yes	NA	NA	NR	LIM
S. Wang, Zhu, & Li (2026) [P37]	Indirect prompt injection (path traversal, exfiltration, high-frequency tool abuse)	Externally retrieved data	Each tool call with cross-step labels	Yes	NR	Yes	Yes	Yes	Yes	Yes	NR	HIGH

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
She et al. (2026) [P58]	Misaligned tool calls incl. IPI	Tool outputs; context	Each tool call and its evidence provenance	Yes	Yes (partial)	Yes	NR	Yes	Yes	Yes	NR	MOD
T. Shi et al. (2025) [P49]	Indirect prompt injection; tool misuse	Tool outputs	Each tool call vs. privilege policy	Yes	NR	Yes	Yes (policy updates during task)	NR	Yes	Yes	NR	MOD
Storf et al. (2026) [P34]	Scheming (covert pursuit of misaligned goals)	NA	Externally observable trajectory	Yes	Yes	Yes	Yes	NR	Yes	NR	NR	LIM

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Tsai & Bagdasarian (2025) [P51]	Context-inappropriate actions	NA	Proposed action vs. task-context policy	Yes	NR	Yes	NR	NR	Yes	Yes	NR	LIM
Wen et al. (2025) [P03]	Indirect prompt injection	Retrieved external content	Model hidden states and gradients for an input	Yes	NR	NR	NR	NR	Yes	NR	NR	MOD
Xiang et al. (2025) [P53]	Violations of safety guard requests	NA	Target agent actions	Yes	NR	Yes	NR	NR	Yes	Yes	NR	LIM
Y. Chen et al. (2025) [P24]	Prompt injection in retrieved data	Retrieved content	Input prompt	Yes	NR	NR	NR	NR	NR	NR	NR	LIM

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Y. Wu et al. (2025) [P36]	Untrusted apps; cross-app security, privacy and safety issues	Third-party apps	App interactions	Yes	NR	Yes	NR	NR	NV	Yes	NR	MOD
Z. Chen et al. (2026) [P68]	Irreversible effects; cross-step violations	NA	Task-level transaction	Yes	NR	Yes	Yes	NR	NR	Yes (staging, rollback, recovery)	NR	LIM
Z. Chen, Kang, & Li (2025) [P54]	Policy violations over trajectories	NA	Action trajectory	Yes	Yes (trajectory-level)	Yes	Yes	NR	Yes	Yes	NR	LIM
Zhong et al. (2025) [P06]	Prompt injection; privacy leakage	Tool outputs / external content	Each tool call and its data dependencies	Yes	NR	Yes	NV	Yes	Yes	Yes	NR	MOD

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mitigation	Financial Context	Evidence Level
Zhu, Yang, et al. (2025) [P57]	Indirect prompt injection	Tool outputs	Tool calls under original vs. masked re-execution	Yes	Yes	Yes	Yes	NR	Yes	NV	NR	LIM
Zhu, Zhang, et al. (2025) [P18]	High-risk or unethical tasks; adversarial attacks	Task prompts; tool use; agent collaboration	Prompts, agent messages and tool use	Yes	Yes	Yes	NV	NR	Yes	Yes	NR	MOD

CHAPTER THREE: RESEARCH METHODOLOGY, DESIGN AND IMPLEMENTATION OF THE REM FRAMEWORK

3.1 Research Methodology and System Scope

3.1.1 Design science research

This thesis follows design science research, in which knowledge is produced by building an artifact that addresses an identified problem and evaluating it rigorously (Hevner et al., 2004). The research process follows the six activities of the design science research methodology of Peffers et al. (2007). Table 3.1 maps these activities to the thesis.

Table 3.1. Design science research activities in this thesis

Activity (Peffers et al., 2007)	Realization in this thesis	Location
Problem identification and motivation	Runtime compromise of tool-using financial agents through indirect prompt injection	Chapter 1; Chapter 2
Definition of objectives for a solution	Objectives O1–O3; research questions RQ1–RQ4; security objectives SO-1 to SO-6	Sections 3.1.2, 3.1.3, 3.3.7
Design and development	Five-layer REM architecture and its runtime pipeline	Sections 3.2–3.10
Demonstration	Integration of REM with the AgentDojo banking environment	Section 3.16
Evaluation	Controlled experiments, baseline comparison, and ablation	Sections 3.11–3.12; Chapter 4
Communication	Thesis and derived publications	Chapter 5

The evaluation is quantitative and experimental. REM is compared against a baseline that uses the same probability estimator but a consequence-independent decision rule, so that differences in outcomes can be attributed to the decision rule rather than to the estimator.

3.1.2 Research objectives

The approved objectives are:

- **O1:** Design a REM framework for financial AI agents' and task automation agents' security.
- **O2:** Evaluate the framework against adversarial attacks.

- **O3:** Compare performance with existing security approaches.

This chapter realizes O1 through the architecture and runtime pipeline, O2 through the attack scenarios and metrics of Sections 3.11–3.12, and O3 through the baselines of Section 3.11.7. The evaluation in this thesis uses a financial (banking) environment. Task-automation environments are outside the evaluation scope unless approved as an extension.

3.1.3 Research questions

The approved proposal does not state research questions. The following questions are derived from the objectives and are presented for supervisor confirmation.

- **RQ1 (estimation).** How well does a ridge logistic regression estimator, using context, behavioral, and action evidence available at the tool-call boundary, discriminate adversarially induced proposed actions in the AgentDojo banking environment, and how well calibrated are its probabilities under grouped cross-fitting?
- **RQ2 (decision).** Compared with a consequence-independent threshold policy that uses the same estimator, how does consequence-tiered expected-loss verdict selection change attack success, benign utility, utility under attack, and intervention rates, across a declared range of loss structures?
- **RQ3 (mitigation and cost).** What residual attack success remains after the Modify mechanism, how often is escalation selected, and what per-step latency does REM add?
- **RQ4 (component contribution).** What is the marginal contribution of each evidence group, of calibration, and of consequence tiers, as determined by ablation?

Table 3.2. Traceability of research questions

RQ	Objectives	Principal components	Principal metrics
RQ 1	O1, O2	Detection and Behavioral Analysis evidence; estimator; calibration	ROC-AUC, PR-AUC, ECE, log-loss
RQ 2	O2, O3	Consequence tiers; expected-loss decision; baseline policy	ASR, benign utility, utility under attack, intervention rates, realized loss
RQ 3	O2	Mitigation Layer; Escalate; runtime pipeline	Residual ASR under Modify, escalation rate, latency
RQ 4	O2	All components (ablation)	Change in the metrics above

3.1.4 Scope

In scope:

- A single LLM-based agent that invokes tools with generated arguments.
- A financial (banking) task environment with executable tools and state-based checks.
- Indirect prompt injection through tool outputs as the evaluated attack class.
- Runtime mediation of each proposed tool call before it is released.
- *[Proposed scope amendment, pending supervisor approval]* A recorded-trace replay sensitivity analysis under a stated no-adaptation assumption (Section 3.11.11). It is reported as exploratory evidence about fixed recorded trajectories, not as an estimate of enforcement against live or adaptive agents.

Out of scope:

- Training-time defenses and modification of the agent's model or prompts.
- Multi-agent systems.
- Evaluation against adaptive adversaries who tune attacks to REM.
- Production deployment in a financial institution.
- Counterfactual estimation of intervention value for adaptive agents, for example by replaying agent prefixes and executing alternative actions from the same state (C. Zhang et al., 2026).

3.1.5 Assumptions

Table 3.3. Assumptions

ID	Assumption	Type	Status
AS-1	REM can intercept each proposed tool call and each tool result without modifying the agent's internals.	Architectural	To be demonstrated on the chosen platform (Section 3.16)
AS-2	Context segments can be attributed to their source channel reliably enough to assign a provenance label.	Architectural	To be demonstrated
AS-3	The adversary cannot modify REM's code, parameters, policy, or logs.	Architectural	Assumed; stated as a limitation
AS-4	The user's objective is legitimate; REM protects the user against external manipulation.	Scoping	Stated choice
AS-5	Tool outputs and retrieved content are untrusted by default.	Scoping	Follows from the indirect injection threat (Greshake et al., 2023)
AS-6	Adversaries do not adapt to REM's parameters in the primary evaluation.	Scoping	Declared limitation
AS-7	Observable evidence separates adversarially induced from benign proposed actions.	Empirical	Tested in pilot E0 (Section 3.11.8)
AS-8	The benign reference data are representative of the evaluated workload.	Empirical	Threatened by benchmark scope (Section 3.14)
AS-9	LLM non-determinism is bounded enough for repeated runs to characterize variance.	Empirical	Measured through repeated runs
AS-10	The agent's internal reasoning is not used; REM relies only on observable inputs, proposed actions, and tool results.	Architectural	Design constraint

3.1.6 Notation

Table 3.4. Notation used in this chapter

Symbol	Meaning
t, T	Step index within an episode; episode length
g	User objective of the episode
c_t	Context at step t, as segments with provenance labels
a_t	Proposed action (tool name and arguments) at step t
H_t	Episode history up to step t
$y_t \in \{0, 1\}$	Latent label: 1 if a_t is adversarially induced
$x_t \in [0, 1]^d$	Evidence vector; groups x^{ctx}, x^{beh}, x^{act} with index sets I_{ctx}, I_{beh}, I_{act}
β_0, β, λ	Logistic intercept, coefficient vector, ridge strength
s_t	Logit (linear predictor)
$\tilde{p}_t, p_t$	Uncalibrated probability; calibrated probability
γ_1, γ_0	Calibration slope and intercept
z_t, k_t, κ	Consequence descriptors; consequence tier; tier map
$\mathcal{V}$	Verdict set {Allow, Modify, Escalate, Block}
$\mathcal{V}(a_t)$	Verdicts feasible for a_t
v_t	Verdict issued at step t
$L(v, y, k)$	Loss of verdict v when the label is y and the tier is k
$C_{miss}(k), C_{FA}(k), C_{mod}(k), C_{esc}$	Declared costs of a missed attack, a false intervention by Block, Modify on a legitimate action, and escalation
$r_{mod}(k)$	Residual attack-success rate after Modify at tier k (measured)
$R_t(v)$	Conditional risk of verdict v at step t
$\pi_{v,v'}(k), p^*_k$	Indifference threshold between verdicts v and v′; Allow–Block threshold
$\sigma(\cdot)$	Logistic function, $\sigma(u) = 1 / (1 + \exp(-u))$
Π, k_{D2}	Set of prohibited actions; minimum tier for predicate D2
$\theta(\alpha), \alpha_{blk}, \alpha_{mod}$	Baseline threshold for false-positive target α; targets for Block and Modify
$\mathcal{T}_R$	Set of tools to which the Modify mechanism applies

Symbols used only in the optional sequential analysis are defined in Section 3.11.10. Symbols that appear in a single equation, such as Δ_0, Δ_1, B_m, and the step sets D_{fit}, D_{cal}, and

D_{val}^{benign}, are defined where they are used. Each symbol has one meaning throughout the chapter.

3.2 REM Architecture

3.2.1 Five layers

REM is a runtime mediation layer between one agent and its environment. It comprises exactly five layers, as approved in the research proposal:

1. **Input & Context Layer.** Captures the information the agent handles and the action it proposes, and labels each context segment with its provenance.
2. **Detection Layer.** Computes evidence about untrusted context.
3. **Behavioral Analysis Layer.** Computes evidence about the episode history and the proposed action.
4. **Decision Engine Layer.** Estimates the probability that the proposed action is adversarially induced, assigns a consequence tier, applies policy predicates, selects a verdict, and writes the audit record.
5. **Mitigation Layer.** Executes the mechanism associated with the verdict.

Offline fitting and recalibration (Section 3.9) are cross-cutting activities performed between deployments. They are not a layer, are not numbered with the layers, and never change REM's parameters during an episode.

3.2.2 Topology and information flow

At each step, REM executes Layer 1; then Layers 2 and 3, each computing its evidence from Layer 1's output without using the other's; then Layer 4; then Layer 5, before the proposed action is released to the environment. Layers 2 and 3 are independent computations, not concurrent ones: the implementation evaluates them one after the other in the same process (Section 3.16). They share the declared instruction-pattern set as a resource, not each other's output. The Detection and Behavioral Analysis Layers compute their evidence independently; they are combined only in the Decision Engine. This independence allows an evidence group to be removed in ablation without changing the computation of the other. After an action is released, its tool result re-enters Layer 1 as untrusted content for the next step.

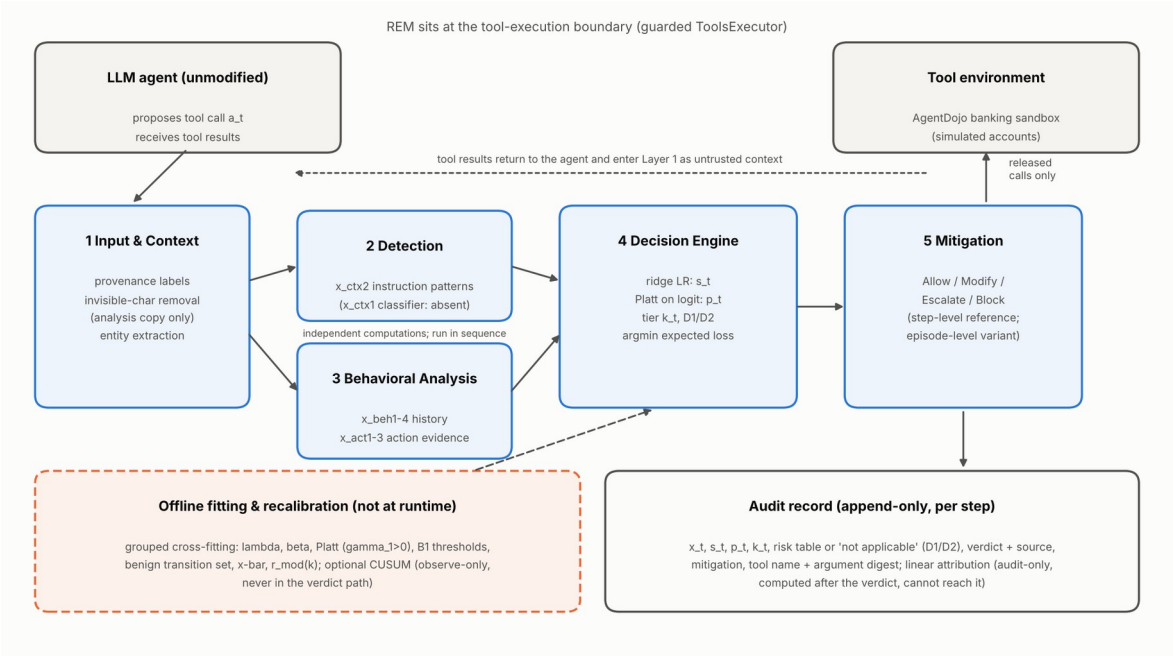


Figure 3.1. REM architecture as implemented. Five layers at the tool-execution boundary; offline fitting and recalibration are cross-cutting and never run within an episode; the audit record is written for every step and its attribution cannot reach the verdict.

Figure 3.1 shows the runtime path from the proposed tool call through Layers 1–5 to the environment, the return of tool results as untrusted context, the append-only audit record, and the cross-cutting offline fitting. The human reviewer to whom Escalate refers is outside REM. In this thesis no reviewer is present, and escalations are resolved by the two bounds of Section 3.11.7.

3.2.3 Layer interfaces

Table 3.5. Layer interfaces

Layer	Consumes	Produces
L1 Input & Context	Objective, system prompt, user messages, tool results, proposed action	Labeled context c_t; entities with provenance; held action a_t
L2 Detection	Untrusted segments of c_t	Context evidence x^{ctx}
L3 Behavioral Analysis	H_t, a_t, entities, benign reference transitions, privilege map	Behavioral evidence x^{beh}; action evidence x^{act}
L4 Decision Engine	x_t, consequence descriptors z_t, frozen parameters, declared policy	p_t, k_t, $R_t(\cdot)$, verdict v_t, audit record
L5 Mitigation	v_t, a_t, episode restrictions	Released, restricted, withheld, or refused action

The four functions are separated architecturally. Layers 2 and 3 produce evidence and issue no verdicts. Layer 4 performs probability estimation and decision. Layer 5 performs mitigation.

3.3 Threat Model and Financial Action Boundary

3.3.1 Protected system

The protected system is a single LLM-based agent that pursues a user objective by proposing tool calls. REM treats the agent as opaque: it observes inputs, context, proposed actions, and tool results, but not model parameters or internal reasoning (AS-10). The evaluation instance is an agent operating the AgentDojo banking environment (Debenedetti et al., 2024), described in Section 3.11.2.

3.3.2 Episodes and steps

An episode is one execution of the agent on one user objective g. It is a finite sequence of steps t = 1, …, T, where step t is the interval in which the agent proposes action a_t. At step t, REM observes g, the labeled context c_t, the proposed action a_t, and the history H_t of prior contexts, actions, verdicts, and tool results.

3.3.3 Trust boundaries and untrusted content

Table 3.6. Trust boundaries

ID	Boundary	Direction	Treatment in REM
TB-1	User input	User → agent context	Labeled semi-trusted
TB-2	Observation	Tool outputs and retrieved content → agent context	Labeled untrusted; primary observation point
TB-3	Action	Agent → tools and external systems	Primary enforcement point
TB-4	Memory	Context ↔ persistent store	Provenance retained on read and write (design scope only)

Untrusted content is any context segment that entered through TB-2 or TB-4, or that was derived from such a segment. The observation boundary matters because content arriving there enters after the user's request, which is the route exploited by indirect prompt injection (Greshake et al., 2023; OWASP Gen AI Security Project, 2025). The action

boundary matters because it is the last point at which an induced action can be prevented rather than repaired.

3.3.4 Adversary model

- **AD-1, content-control adversary (primary, evaluated).**
  - *Can:* author or modify content that the agent receives through tool outputs, including instruction-like text and values such as account identifiers.
  - *Cannot:* access the user channel, observe or modify REM's parameters, or modify agent or REM code.
- **AD-2, prompt-channel adversary (design scope only).** Can submit crafted input through a compromised user channel. The evaluated banking attacks do not exercise this channel.

Excluded:

- adversaries who modify model weights, agent code, or REM;
- infrastructure compromise;
- a malicious operator;
- adaptive adversaries (AS-6).

3.3.5 Indirect injection and adversarially induced actions

Indirect prompt injection occurs when content from an external source, processed by the agent, alters its behavior (OWASP Gen AI Security Project, 2025). REM's target event is narrower and defined at the action level.

A proposed action a_t is **adversarially induced** ($y_t = 1$) if executing it would advance an objective introduced by the adversary rather than the user objective g. The label is not observable at runtime. Actions that are unsafe because of the agent's own error without adversarial involvement are outside this definition. The operational derivation of y_t for evaluation data is given in Section 3.11.3.

3.3.6 Sensitive financial actions

Table 3.7 groups the tools of the evaluated banking environment by the consequence of an adversarially induced call. The tool names are those registered in the AgentDojo banking suite inspected for this chapter (Section 3.11.2). The grouping is a design classification, not a property reported by the benchmark.

Table 3.7. Action categories in the banking environment

Category	Tools	Consequence if adversarially induced
Read	get_balance, get_iban, get_most_recent_transactions, get_scheduled_transactions, get_user_info, read_file	No direct external effect; may expose data later sent elsewhere
Value transfer	send_money, schedule_transaction	Money sent to an attacker-controlled account
Payment modification	update_scheduled_transaction	Redirection or alteration of a recurring payment
Credential change	update_password	Loss of control over the account
Profile change	update_user_info	Alteration of account holder information

3.3.7 Security objectives

The threat model above states what an adversary can do; the objectives below state what REM must therefore do about it. They are ordered along the path of a single proposed tool call: from the evidence available about it, through the verdict selected for it, to the record left behind and the cost of producing it.

- **SO-1:** Use evidence about untrusted content that enters through TB-2.
- **SO-2:** Detect proposed actions whose arguments originate in untrusted content or that deviate from the episode's benign pattern.
- **SO-3:** Account for the consequence of the proposed action when selecting a verdict.
- **SO-4:** Provide human escalation as a response.
- **SO-5:** Emit, for every verdict, an audit record from which the verdict can be reproduced.
- **SO-6:** Preserve benign task utility and report per-step latency.

3.4 Input & Context Layer

The Input & Context Layer collects and labels information. It computes no risk evidence.

3.4.1 Context capture

At step t the layer receives:

- the user objective g and user messages (TB-1);

- the system prompt;
- the results of previously executed tools (TB-2);
- the proposed action a_t, captured at TB-3 and held until Layer 5 acts.

Invisible and formatting characters, such as zero-width characters, are removed from untrusted text before further processing. This is a deterministic hygiene step applied to **REM's analysis copy** of the text. The content delivered to the agent is not altered, because REM is non-invasive (AS-1); hygiene therefore protects REM's feature computation, not the agent. It reduces some encoding-based concealment but does not prevent paraphrase-based evasion.

3.4.2 Provenance labeling

Each context segment receives one label from the ordered set trusted (system prompt) $\succ$ semi-trusted (user input) $\succ$ untrusted (tool output, retrieved content, memory). Content derived from several segments inherits the least-trusted label among them, and processing never promotes content to a more trusted label. This rule is a design definition that preserves the distinction between data and instruction exploited by indirect injection. It asserts no empirical claim.

3.4.3 Entity extraction

The layer extracts security-relevant entities from each segment and from the arguments of a_t, and records the provenance of each occurrence:

- account identifiers (such as IBAN-formatted strings);
- monetary amounts;
- password and credential values;
- user-profile values (such as addresses and phone numbers);
- tool names mentioned in text.

The extraction patterns are declared in configuration and are fixed before evaluation. These entities allow Layer 3 to ask whether an argument of the proposed action can be traced to the user objective or to trusted content, or only to untrusted content.

3.5 Detection Layer

The Detection Layer computes context evidence from untrusted segments. It issues no verdict.

Table 3.8. Context evidence

Feature	Definition	Type	Availability
x^{ctx}_1	Maximum score of a fixed, pre-trained prompt-injection classifier over untrusted segments added since the previous step	[0, 1]	Derived (classifier choice pending supervisor approval)
x^{ctx}_2	1 if an untrusted segment added since the previous step matches the declared instruction-pattern set; otherwise 0	Binary	Derived (pattern set declared before evaluation)

The classifier is used with fixed inference and is not retrained. A classifier distributed with an existing guardrail system, such as PromptGuard 2 in LlamaFirewall (Chennabasappa et al., 2025), is a candidate. The instruction-pattern set contains declared indicators such as imperative instructions addressed to an assistant, references to tool names, and role or delimiter markers.

Both features are individually evadable, and benchmark-based scores of detectors are fragile (Arp et al., 2022; M. Q. Li et al., 2026). The Detection Layer is therefore not relied upon to be decisive. Its evidence is combined with behavioral and action evidence in the Decision Engine, and its contribution is measured by ablation (Section 3.11.7).

3.6 Behavioral Analysis Layer

The Behavioral Analysis Layer maintains the episode history and computes evidence about the trajectory and the proposed action. All reference quantities it uses are estimated offline from benign data and are frozen at runtime.

3.6.1 Step tracking

The layer maintains H_t: the sequence of proposed actions, their arguments with provenance, the verdicts issued, and the tool results returned. H_t is reset at the start of each episode.

3.6.2 Behavioral and action evidence

Table 3.9. Behavioral and action evidence

Feature	Definition	Type	Availability
x^{beh}_1	Post-observation shift: 1 if the most recent tool result was untrusted and had $x^{ctx}_2 = 1$; otherwise 0	Binary	Derived from H_t
x^{beh}_2	Repeated calls: $\min(n_{rep}, n_{max}) / n_{max}$, where n_{rep} is the number of consecutive calls with the same tool and key arguments and n_{max} is declared	[0, 1]	Derived from H_t
x^{beh}_3	Privilege escalation: 1 if the privilege level of a_t's tool exceeds the highest level used so far in the episode; otherwise 0	Binary	Derived; privilege map declared
x^{beh}_4	Transition deviation: 1 if the pair (previous tool, current tool) does not occur in the benign reference transition set; otherwise 0	Binary	Derived; reference set estimated on benign fitting data
x^{act}_1	Argument provenance: 1 if a key argument of a_t (recipient account identifier, amount, password, or profile value) occurs in untrusted content but not in g or trusted content; otherwise 0	Binary	Derived from L1 entities
x^{act}_2	Destination novelty: 1 if the recipient of a_t appears neither in g nor among the transactions observed before the episode's first untrusted ingestion; otherwise 0	Binary	Derived; applicable to value-transfer and payment-modification tools
x^{act}_3	Sensitive egress: 1 if a_t sends to an external recipient a value obtained from a sensitive read (account identifier, profile value, or credential); otherwise 0	Binary	Derived from L1 entities and H_t

All features are bounded in [0, 1] by definition, so no further rescaling is applied. Three design rules govern them:

1. **No identity-based features.** No feature may encode a specific account identifier. In the evaluated benchmark, one attacker account identifier is shared by most injection tasks (Section 3.11.2), and a feature keyed on that value would leak the label.
2. **Evidence is kept separate from consequence.** Whether a payee appears only in untrusted content is evidence about *whether* an action is adversarially induced; it

belongs in x_t. Whether the action transfers a large amount irreversibly describes *what would happen*; it belongs in the consequence descriptors z_t (Section 3.7.3) and enters the decision only through the loss.

3. **The feature set is frozen after pilot E0** (Section 3.11.8), before any held-out predictions are examined.

3.7 Decision Engine Layer

The Decision Engine turns evidence into a verdict in five stages: probability estimation, calibration, consequence assessment, policy predicates, and expected-loss verdict selection. It also writes the audit record. Each equation below states its source and whether it is established from the source, adapted from it, or derived here.

3.7.1 Ridge logistic estimation

The probability that the proposed action is adversarially induced is modeled by logistic regression (Cox, 1958):

$$s_t = \beta_0 + \beta^\top x_t,\ \tilde{p}_t = \sigma(s_t) = \frac{1}{1+\exp(-s_t)} \quad (3.1)$$

where $x_t \in [0, 1]^d$ is the evidence vector $[x^{ctx}; x^{beh}; x^{act}]$, $\beta_0 \in \mathbb{R}$ is the intercept, $\beta \in \mathbb{R}^d$ is the coefficient vector, s_t is the logit, and $\tilde{p}_t$ is the uncalibrated estimate of $P(y_t = 1 \mid x_t)$.

Role in REM: maps the evidence of Layers 2 and 3 to a score and an uncalibrated probability. *Status:* established from source (Cox, 1958).

The parameters are estimated offline by maximizing a ridge-penalized log-likelihood (le Cessie & van Houwelingen, 1992):

$$(\hat{\beta}_0, \hat{\beta}) = \arg\max_{\beta_0, \beta} \sum_{i \in D_{fit}} \left[y_i \ln \tilde{p}_i + (1 - y_i) \ln(1 - \tilde{p}_i) \right] - \lambda \|\beta\|_2^2 \quad (3.2)$$

where D_{fit} is the set of labeled fitting steps, $y_i \in \{0, 1\}$ are their labels, $\tilde{p}_i$ is given by Equation 3.1, and $\lambda \geq 0$ controls the strength of the quadratic penalty. The intercept is not penalized.

Role in REM: produces finite and stable coefficients from small, imbalanced data with binary features. *Status:* established from source.

The quadratic penalty is needed for a specific reason. The evidence features are mostly binary, and adversarial steps are scarce (Section 3.11.4). If a single indicator appears only in adversarial fitting steps, the unpenalized maximum-likelihood estimate does not exist as a finite value. The penalty keeps the estimates finite.

Reference parameterization of λ. This thesis fixes the penalty as written in Equation 3.2, $\lambda\|\beta\|^2$. The implementation (Section 3.16) minimizes the negative log-likelihood plus $(\lambda_{impl}/2)\|\beta\|^2$, so the two parameterizations correspond exactly when $\lambda = \lambda_{impl}/2$. Because λ is selected from a *finite* grid, the two scales are equivalent only under this conversion: the declared grid $\lambda_{impl} \in \{0.01, 0.1, 1, 10, 100\}$ is the grid $\lambda \in \{0.005, 0.05, 0.5, 5, 50\}$ on the scale of Equation 3.2. Every selected value is reported on both scales. The scaling constant in le Cessie and van Houwelingen (1992) has not been checked against the full text; it does not affect the method, which depends only on the stated objective and the declared grid.

No interaction terms are included by default. Equation 3.1 assumes that the log-odds of adversarial induction are approximately linear in the evidence.

3.7.2 Calibration on the logit

The uncalibrated probability $\tilde{p}_t$ need not agree with observed frequencies. Ridge shrinkage and class imbalance can both distort it. Because the decision rule in Section 3.7.5 treats the probability as a probability, REM recalibrates it with a two-parameter logistic map applied to the logit:

$$p_t = \sigma(\gamma_1 s_t + \gamma_0), \gamma_1 > 0 \quad (3.3)$$

where γ_1 is the calibration slope and γ_0 is the calibration intercept.

Role in REM: produces the calibrated probability used by the decision rule. *Status:* adapted from source. The method is Platt scaling (Platt, 2000). The two-parameter logistic form with parameters fitted by negative log-likelihood on held-out data follows Guo et al. (2017). The map is applied here to the logit of a logistic regression model. The monotonicity condition follows Kull et al. (2017), who require a non-negative slope for a non-decreasing map; REM requires a strictly positive slope so that the ranking of steps is preserved.

The calibration parameters are fitted by minimizing the negative log-likelihood on held-out logits (Guo et al., 2017):

$$(\hat{\gamma}_1, \hat{\gamma}_0) = \arg\min_{\gamma_1, \gamma_0} - \sum_{i \in D_{cal}} \left[y_i \ln p_i + (1 - y_i) \ln (1 - p_i) \right] \quad (3.4)$$

where D_{cal} is a set of steps whose logits s_i were produced by a model that was not fitted on those steps. Fitting Equation 3.4 on the model's own training logits would be close to degenerate, so REM uses out-of-fold logits from grouped cross-fitting (Section 3.9).

Status: established from source (Guo et al., 2017).

Fitting target. Equation 3.4 fits the calibration map to the observed labels y_i. Platt's original procedure instead fits against smoothed targets, $t_+ = (N_+ + 1)/(N_+ + 2)$ and $t_- = 1/(N_- + 2)$, to reduce overfitting on small calibration sets. This thesis takes Equation 3.4 as the reference definition and evaluates Platt's smoothed targets as a sensitivity analysis.

Four properties of this choice are stated.

1. **It changes probabilities, not rankings.** With $\gamma_1 > 0$, Equation 3.3 is strictly increasing in s_t. Calibration therefore leaves ROC-AUC, PR-AUC, and any decision based only on the ordering of steps unchanged. If the fitted slope is not positive, calibration is rejected for that fit and the event is reported.
2. **It is the minimal adequate family.** Kull et al. (2017) prove that beta calibration with equal shape parameters equals logistic calibration applied to the log-odds of a score. Since $\tilde{p}_t = \sigma(s_t)$, the log-odds of $\tilde{p}_t$ is s_t, so Equation 3.3 is exactly beta calibration with equal shape parameters applied to $\tilde{p}_t$. The identity map is included ($\gamma_1 = 1$, $\gamma_0 = 0$). Full beta calibration adds a third parameter, and isotonic regression is nonparametric; neither is adopted, given the limited calibration data.
3. **It is not a contribution.** Calibrated logistic risk scores are already used in agent safety (Hossain et al., 2026).
4. **It does not guarantee good control.** C. Zhang et al. (2026) show that recalibrating a scalar risk score can improve calibration error while leaving control regret under threshold routing unchanged, because a scalar probability does not represent whether an intervention would improve the outcome. REM therefore reports calibration quality and control outcomes separately (Section 3.12) and does not claim that calibration improves control.

3.7.3 Consequence tiers

The consequence of a proposed action is described by descriptors z_t obtained from the action itself (Table 3.10). They are mapped to a consequence tier $k_t = \kappa(z_t) \in \{1, \ldots, K\}$ by a declared policy function κ. Consequence descriptors do not enter Equation 3.1; they affect the verdict only through the loss. Keeping the likelihood that an action is

adversarially induced separate from the consequence of executing it is consistent with threat models that distinguish compromise from harm and its severity (Kanaker et al., 2026).

Table 3.10. Consequence descriptors

Descriptor	Meaning	How obtained	Availability
Action category	Read, value transfer, payment modification, credential change, profile change (Table 3.7)	Tool name	Available
Amount	Monetary amount of a value-transfer or payment-modification call	Action arguments	Available where the tool takes an amount
Recurring-payment change	Whether the action alters a scheduled or recurring payment	Tool name	Available
Credential change	Whether the action changes account credentials	Tool name	Available
Irreversibility	Whether the effect is treated as irreversible once executed	Declared per tool	Proposed (policy); the benchmark does not simulate settlement

Table 3.11. Illustrative consequence tiers (policy pending supervisor approval)

Tier	Illustrative membership
1	Read tools
2	Profile change
3	Value transfer at or below a declared amount limit
4	Value transfer above the declared limit; payment modification; credential change

The quantities used by the Decision Engine fall into three classes, which are kept distinct throughout the thesis.

Table 3.12. Learned, measured, and policy quantities

Class	Quantities	How determined
Learned	β_0, β, γ_1, γ_0	Fitted on labeled data by Equations 3.2 and 3.4
Measured	$r_{mod}(k)$; benign reference mean $\bar{x}$; benign transition set; utility impact of Modify	Estimated from observe-only and enforcing-mode runs
Policy	κ, amount limits, $C_{miss}(k)$, $C_{FA}(k)$, $C_{mod}(k)$, C_{esc}, Π, k_{D2}, privilege map, tie-breaking order	Declared before evaluation; varied over a declared grid where they affect outcomes

Policy quantities encode institutional judgment. They are not presented as empirical findings, and their influence is reported through the loss grid of Section 3.12.3.

3.7.4 Policy predicates

Two deterministic predicates are evaluated before verdict selection.

- **D1, prohibited action.** If $a_t \in \Pi$, the verdict is Block irrespective of p_t. A prohibition that a sufficiently low probability could override would not be a prohibition.
- **D2, missing required evidence.** If the evidence required for tier k_t cannot be computed (for example, argument provenance is unavailable) and $k_t \geq k_{D2}$, the verdict is Escalate. Absent evidence is not treated as evidence of safety.

Π, k_{D2}, and the evidence required at each tier are declared policy. When a predicate applies, it determines the verdict, the expected-loss rule is not consulted, and the audit record names the predicate as the source of the verdict. Both predicates are disabled in one ablation (Section 3.11.7).

Order of evaluation and unavailable evidence. Availability of each feature is determined first, from the proposed action alone. D1 and then D2 are applied *before* any probability enters the decision, so no verdict that a predicate fixes depends on a value computed from unavailable evidence. Two cases follow:

- **A predicate fixes the verdict.** s_t and p_t may still be computed and stored for audit, but they are marked as not used, and the audit record contains no risk table: the field is recorded as *not applicable* (Section 3.7.6).
- **No predicate applies, but a feature is unavailable** (possible only below k_{D2}). The feature takes its "not observed" value 0, and the audit record flags the imputation. This is a design definition: an unavailable indicator is treated as not fired, which can only lower the estimated probability. It is therefore used only below the tier at which missing evidence forces escalation.

3.7.5 Expected-loss verdict selection

Loss structure. For each tier k, the loss of issuing verdict v when the label is y is given in Table 3.13.

Table 3.13. Tier-indexed loss structure L(v, y, k)

Verdict v	L(v, 0, k): legitimate action	L(v, 1, k): adversarially induced action
Allow	0	$C_{miss}(k)$
Modify	$C_{mod}(k)$	$r_{mod}(k) \cdot C_{miss}(k)$
Escalate	C_{esc}	C_{esc}
Block	$C_{FA}(k)$	0

All declared costs are non-negative and expressed in common relative units. Correct automatic decisions are assigned zero loss, which is a declared convention.

Reviewer assumptions. The Escalate entries assign the same loss C_{esc} whatever the label. This assumes a reviewer who:

- resolves every escalation correctly (accuracy 1);
- does so within the time the transaction allows, with delay and effort priced into C_{esc};
- is always available.

The verdict rule is optimal only under these assumptions. They are not tested in this thesis, because no reviewer is present. They are optimistic: a simulation study reports that human-in-the-loop approvals reduce high-impact actions but degrade under user error and habituation (Kanaker et al., 2026). The rule is instead *evaluated* under two bounds (Section 3.11.7): an idealized reviewer that satisfies the assumptions, and no reviewer, in which every escalation withholds the action. A policy designed for a perfect reviewer and evaluated without one is expected to withhold legitimate actions; that cost is reported, not hidden. A reviewer with error rate e would add $e \cdot C_{miss}(k)$ to the adversarial entry and $e \cdot C_{FA}(k)$ to the legitimate entry of Escalate; such a model is not specified here.

The adversarial Modify entry is a derivation. Under the stated assumption A-Mod — whether Modify neutralizes an adversarially induced action is independent of x_t given the tier, and the attack fails with probability $1 - r_{mod}(k)$ — the law of total expectation gives:

$$L(Modify, 1, k) = r_{mod}(k) C_{miss}(k) + (1 - r_{mod}(k)) \cdot 0 = r_{mod}(k) C_{miss}(k) \quad (3.5)$$

where $r_{mod}(k) \in [0, 1]$ is the residual attack-success rate after Modify at tier k, measured in enforcing-mode runs (Section 3.11.7).

Status: derived under assumption A-Mod.

Conditional risk. Elkan (2001) states that the optimal prediction is the one that minimizes expected cost, computed from the conditional probability of each class and the cost of each prediction for each true class. REM applies this criterion to the four verdicts, with losses indexed by the consequence tier and the calibrated probability of Equation 3.3 in place of the true conditional probability:

$$R_t(v) = (1 - p_t) L(v, 0, k_t) + p_t L(v, 1, k_t), v \in V(a_t) \quad (3.6)$$

where $R_t(v)$ is the conditional risk of verdict v at step t.

Status: adapted from source. Elkan's criterion is stated for predictions of classes. REM extends the set of choices to four verdicts, indexes the losses by consequence tier, and substitutes the calibrated estimate p_t for the true posterior. The substitution is justified only to the extent that p_t is calibrated.

Bayes verdict. The verdict is:

$$v_t = arg \min_{v \in V(a_t)} R_t(v) \quad (3.7)$$

with ties resolved toward the more restrictive verdict in the declared order Block ≥ Escalate ≥ Modify ≥ Allow.

Status: adapted from source; the tie-breaking order is a design definition that makes the rule a function. Ties are counted and reported whenever the rule is evaluated, so that the influence of the tie order on any result is visible.

The feasible set $\mathcal{V}(a_t)$ contains Allow, Escalate, and Block for every action, and Modify only if the tool of a_t belongs to the set $\mathcal{T}_R$ to which the Modify mechanism applies (Section 3.8).

Derived thresholds. Each $R_t(v)$ is affine in p_t, so the verdict changes only where two conditional risks are equal. For verdicts v and v′ at tier k, let $\Delta_0 = L(v', 0, k) - L(v, 0, k)$ and $\Delta_1 = L(v, 1, k) - L(v', 1, k)$, with $\Delta_0 + \Delta_1 > 0$. Setting $R(v) = R(v')$ in Equation 3.6 and solving for p gives:

$$\pi_{v,v'}(k) = \frac{\Delta_0}{\Delta_0 + \Delta_1} \quad (3.8)$$

Status: derived from Equation 3.6.

Derivation: $R(v) - R(v') = -\Delta_0 + p(\Delta_0 + \Delta_1)$. This is zero at $p = \pi_{v,v'}(k)$, negative below it (v is preferred), and positive above it (v′ is preferred).

For v = Allow and v′ = Block, $\Delta_0 = C_{FA}(k)$ and $\Delta_1 = C_{miss}(k)$, so:

$$p_k^* = \frac{C_{FA}(k)}{C_{FA}(k) + C_{miss}(k)} \quad (3.9)$$

Status: derived from Equation 3.8. It has the form of the standard two-class cost-sensitive threshold for zero-cost correct decisions, which Elkan (2001) obtains from the same expected-cost criterion; the printed threshold expression in that paper could not be inspected directly during verification, so the derivation above stands on Equation 3.8 alone.

Thresholds are therefore consequences of declared costs rather than independently chosen constants. The question “why this threshold?” becomes “why this cost ratio?”, which the loss grid of Section 3.12.3 addresses.

When escalation can be selected. Chow (1970) analyzed the tradeoff between recognition error and rejection, which is the classical basis for withholding an automatic decision. In REM, Escalate is not implemented as a separate rejection test; it is one of the verdicts inside Equation 3.7. For the verdict set {Allow, Escalate, Block} at tier k, Escalate is selected for some probability if and only if:

$$C_{esc} < \frac{C_{FA}(k)\,C_{miss}(k)}{C_{FA}(k) + C_{miss}(k)} \quad (3.10)$$

Status: derived from Equations 3.6, 3.7, and 3.9.

Derivation: $R(\text{Allow}) = p \cdot C_{miss}(k)$ increases in p and $R(\text{Block}) = (1 - p) \cdot C_{FA}(k)$ decreases in p. Their minimum is largest at p^*_k, where it equals $C_{FA}(k) \cdot C_{miss}(k) / (C_{FA}(k) + C_{miss}(k))$. The constant C_{esc} lies strictly below this minimum for some p if and only if Equation 3.10 holds; at equality, ties resolve toward Block.

When Modify is also feasible, Equation 3.10 remains a necessary condition, because adding a verdict can only lower the minimum risk. In the symmetric case $C_{FA}(k) = C_{miss}(k) = C$, Equation 3.10 reduces to $C_{esc} < C/2$: deferral is selected only when it costs less than half an error.

Dominance check. Before a loss structure is used, it is checked for dominated verdicts. For example, $R(\text{Modify}) - R(\text{Block}) = (1 - p)(C_{mod}(k) - C_{FA}(k)) + p \cdot r_{mod}(k) \cdot C_{miss}(k)$, which is non-negative for every p whenever $C_{mod}(k) \geq C_{FA}(k)$. In that case Modify can never be selected at tier k. A declaration that makes a verdict unreachable, by this check or by Equation 3.10, is reported as such rather than silently producing a smaller verdict set.

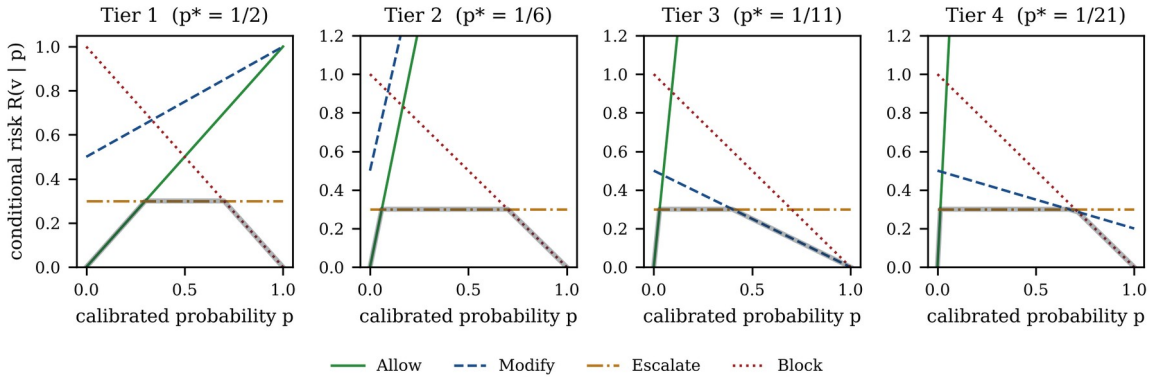


Figure 3.2. Conditional risk R(v | p) of the four verdicts (Eq. 3.6) as a function of the calibrated probability p, for the declared exploratory costs of tiers 1–4 (C_{FA} = 1; C_{miss} = 1, 5, 10, 20; C_{mod} = 0.5; C_{esc} = 0.3; r_{mod} as estimated in the pilot: 1, 1, 0, 0.01). The verdict is the lower envelope. The figure plots the equations for declared values; it is not an experimental result.

3.7.6 Explainability and audit record

For every verdict the Decision Engine writes an audit record containing:

- the identity of the proposed action: tool name and a digest of its arguments, recorded *before* any mitigation alters the step. Raw argument values, which may include credentials, are not stored;
- the evidence vector x_t with its availability flags, the logit s_t, and the probabilities $\tilde{p}_t$ and p_t, with a flag stating whether p_t was used for the verdict;
- the tier k_t and the conditional risks $R_t(v)$, or *not applicable* when D1 or D2 fixed the verdict;
- the verdict v_t and its source (D1, D2, or the expected-loss rule), and the mitigation applied;
- the model, calibration, and policy versions;

- a linear attribution of the logit.

The attribution follows the linear case of Shapley-value attribution given by Lundberg and Lee (2017):

$$\phi_{t,i} = \beta_i (x_{t,i} - \acute{x}_i), i = 1, \ldots, d; \phi_0 = \beta_0 + \beta^T \acute{x} \quad (3.11)$$

where $\bar{x}$ is the mean evidence vector over benign fitting steps, $\varphi_{t,i}$ is the contribution of feature i to the logit relative to that benign reference, and φ_0 is the logit at the reference.

Status: adapted from source. Lundberg and Lee (2017) state the linear case under feature independence with printed indices that do not match between the two sides of the expression and with base value equal to the model intercept. REM uses the index-consistent form and sets the base value to the logit at the reference point, which is the value required by their local-accuracy property when the reference mean is not zero.

The decomposition is exact:

$$\phi_0 + \sum_{i=1}^{d} \phi_{t,i} = s_t \quad (3.12)$$

Status: derived. Summing Equation 3.11 gives $\beta_0 + \beta^T \bar{x} + \beta^T x_t - \beta^T \bar{x} = s_t$. This is the local-accuracy property of Lundberg and Lee (2017) for a linear model, and REM checks it as a correctness invariant.

For reporting by evidence group, REM defines:

$$\Phi_J = \sum_{i \in I_J} \phi_{t,i}, J \in \{ctx, beh, act\} \quad (3.13)$$

Status: design definition. Because the groups partition the features, $\Phi_{ctx} + \Phi_{beh} + \Phi_{act} = s_t - \varphi_0$.

Four qualifications apply:

1. **The attribution never influences the verdict.** It is an audit component only.
2. **It decomposes the logit, not the calibrated probability.** On the calibrated logit scale each contribution is multiplied by γ_1, which preserves relative contributions because $\gamma_1 > 0$.
3. **Correlation limits interpretation, not the identity.** The independence assumption affects the interpretation of each $\varphi_{t,i}$ as a Shapley value when features are correlated; it does not affect the identity of Equation 3.12.

4. **It is not a contribution.** Auditable decision records are provided by several existing systems (Hossain et al., 2026; C. Yang, 2026). Because the attribution is determined by the logged x_t and the frozen parameters, it can equally be computed at audit time.

3.7.7 Consequence-independent baseline policy

The baseline uses the same estimator and calibration but a decision rule that ignores consequence. Its thresholds are selected as operating points on benign validation steps:

$$\theta(\alpha) = min\{\theta : FPR_{val}(\theta) \leq \alpha\}, FPR_{val}(\theta) = \frac{\left|\{i \in D_{val}^{benign} : p_i \geq \theta\}\right|}{\left|D_{val}^{benign}\right|} \quad (3.14)$$

where α is a declared false-positive target, D_{val}^{benign} is the set of benign validation steps in the relevant cross-fitting fold, and θ ranges over the observed probabilities.

The baseline verdict is:

- **Block** if $p_t \geq \theta(\alpha_{blk})$;
- **Modify** if $\theta(\alpha_{mod}) \leq p_t < \theta(\alpha_{blk})$ and Modify is feasible, otherwise Block;
- **Allow** otherwise.

The targets satisfy $\alpha_{mod} > \alpha_{blk}$. Predicates D1 and D2 apply as in Section 3.7.4, so the baseline issues Escalate only through D2. In implementation the candidate set for θ also includes a value just above the largest benign probability, so that α = 0 can be met.

Status: design definition.

What the comparison isolates. REM and the baseline share the estimator, the calibration and the predicates. They differ in two respects: REM's losses depend on the consequence tier, and REM's expected-loss rule can choose Escalate, whereas the baseline can escalate only through D2. A difference between them therefore reflects both consequence tiers and the extra escalation opportunity, not tiers alone. A third configuration separates the two: **REM without expected-loss Escalate**, in which Escalate is removed from the set $\mathcal{V}(a_t)$ used by Equation 3.7 but D2 is kept, gives REM exactly the baseline's opportunity to escalate.

Because Equation 3.14 selects thresholds by ranking, a strictly increasing recalibration re-selects equivalent thresholds and leaves the baseline's verdicts unchanged. Cost-derived thresholds such as Equation 3.9 are fixed in probability units and can change after

recalibration. This contrast is used in the ablation of Section 3.11.7 and is consistent with the finding of C. Zhang et al. (2026) that recalibration need not change threshold-routed control.

3.8 Mitigation Layer

The Mitigation Layer executes, deterministically, the mechanism associated with each verdict. It never initiates an action on the agent's behalf.

Table 3.14. Verdict-to-mitigation mapping: reference (step-level) semantics and the episode-level variant

Verdict	Step-level semantics (reference)	Episode-level variant
Allow	a_t is released across TB-3 and the event is logged.	Same, unless the tool is restricted in S, in which case a_t is refused.
Modify	a_t is withheld, and a structured tool-restriction notice is returned for this call only. No restriction persists. Applies only to tools in $\mathscr{T}_R$.	a_t is withheld, and the tool class of a_t (its action category within $\mathscr{T}_R$) is added to S for the rest of the episode.
Escalate	a_t is withheld and referred for review; it is released only if the reviewer approves.	Same.
Block	a_t is not executed, and a structured refusal is returned. No other tool is affected.	a_t is not executed, and every tool in $\mathscr{T}_R$ is added to S, so no further side-effecting action is released in the episode.

Which semantics is the reference. The design decisions of Stage 2 fix step-level semantics: Block prevents the current action, and Modify applies one declared mechanism to the current action. Any restriction that persists through the episode is a separate experimental policy. The episode-level reading was specified in an earlier version of this chapter. Both are implemented and evaluated, and step-level semantics is the reference definition. This alignment with the frozen design lock awaits the supervisor's confirmation (decision D-2, conflict SC-1). The two semantics can release different actions even when the agent never retries a call or adapts: under the episode-level variant, Block adds every tool of $\mathscr{T}_R$ to the restriction set S, and Modify adds the tool class of a_t, so a later, *different* state-changing call is refused even when its own verdict is Allow. They therefore give different results under the replay of Section 3.11.11 (Chapter 4, Table 4.11). Live, adaptive agents would add further differences, through retries and alternative routes.

Tool restriction is the single Modify mechanism because it is deterministic, does not depend on an LLM rewriting content, and can be measured. Under step-level semantics,

Modify and Block have the same effect on the current call; they differ in the declared loss (Table 3.13), in the notice returned to the agent, and, under the episode-level variant, in what persists. Input sanitization, such as spotlighting (Hines et al., 2024), is an alternative that is not adopted as the primary mechanism.

Four properties are stated:

1. **Restrictiveness ordering.** Under the episode-level variant, Block halts all further side effects, Escalate defers the decision, and Modify removes one tool class. Under step-level semantics the ordering reflects the declared losses rather than persistence. The ordering underlies the tie-breaking order of Equation 3.7.
2. $\mathcal{T}_{\mathbf{R}}$ **is declared.** In the banking environment it contains value-transfer, payment-modification, credential-change, and profile-change tools; read tools are not restricted.
3. **Mitigation effectiveness is measured, not assumed.** The residual attack-success rate $r_{mod}(k)$ used in Equation 3.5 is estimated from enforcing-mode runs.
4. **Retries are observed.** A restricted or blocked agent may retry. Retries raise x^{beh}_{2}, and calls to restricted tools are refused by the Mitigation Layer.

3.9 Offline Fitting and Recalibration

Offline fitting and recalibration are cross-cutting activities. They are performed between deployments on logged data, they are not a layer, and they never modify REM's parameters during an episode. Runtime adaptation from observed traffic is excluded because it would allow an adversary who generates traffic to shift REM's reference behavior.

The offline procedure fits and freezes:

- the benign reference quantities ($\bar{x}$ and the benign transition set);
- the coefficients by Equation 3.2, with λ selected by grouped cross-validation on held-out log-loss;
- the calibration parameters by Equation 3.4 on out-of-fold logits;
- the baseline thresholds by Equation 3.14;
- the measured residual rates $r_{mod}(k)$.

It also checks the declared loss structure for dominated verdicts (Section 3.7.5) and records all model, data, policy, benchmark, and software versions.

Between deployments, calibration can be monitored against later-established labels using the metrics of Section 3.12. If calibration drifts beyond a declared tolerance, the procedure is repeated on new data.

Procedure 3.1. Offline fitting (cross-cutting; never executed within an episode)

```
Input: logged observe-only traces grouped by injection task and user task;
labels y;
       declared policy (κ, L, Π, k_D2, privilege map, 𝒯_R, α_blk, α_mod)
1  for each outer fold f (Section 3.11.5):
2      D_train ← steps outside fold f;  D_test ← steps in fold f
2a     apply the Pilot E0 retention rule to D_train only; fix the feature
set for fold f
3      estimate x̄ and the benign transition set on benign steps of D_train
4      select λ by grouped inner cross-validation on D_train (held-out log-
loss)
5      obtain out-of-fold logits on D_train from inner folds; fit (γ1, γ0)
by Eq. 3.4
6      if γ1 ≤ 0 then record calibration failure for fold f
7      refit (β0, β) on all of D_train with the selected λ (Eq. 3.2)
8      select θ(α_blk), θ(α_mod) by Eq. 3.14 on out-of-fold calibrated
benign steps of D_train
9      apply Eqs. 3.1, 3.3 to D_test; store predictions for evaluation only
9a     estimate r_mod(k) from episodes outside fold f: enforcing-mode runs
(Section 3.11.7),
       or, where these are unavailable, oracle-Modify replays (Section
3.11.11)
10 (r_mod is estimated per fold in line 9a)
11 check the loss structure for dominated verdicts (Section 3.7.5)
12 freeze parameters and record all versions
```

3.10 Algorithm 1: REM Runtime Evaluation and Mitigation

Algorithm 1 gives the per-step runtime procedure. Every operation is defined in Sections 3.4–3.8.

```
Algorithm 1. REM Runtime Evaluation and Mitigation (as implemented; Section
3.16)


Frozen (offline, per fold):  β0, β, γ1 (> 0), γ0, x̄, benign transition set,
                             privilege map, instruction-pattern set, n_max
Declared policy:             κ, L (Table 3.13), Π, k_D2, required
```

```
evidence, 𝒯_R,
                              tie order Block ≥ Escalate ≥ Modify ≥ Allow,
                              mitigation semantics ∈ {step-level, episode-
level}
Input at step t:              objective g; tool results since the last step;
                              proposed call a_t = (tool, args); history H_t;
restrictions S
Output:                       verdict v_t; released or withheld call; audit
record r_t


 1  for each tool result returned since the last decision
point:           // Layer 1
 2      label it untrusted; strip invisible characters in REM's copy;
 3      extract identifiers and profile values from sensitive reads
 4  begin a decision point (calls proposed in one agent message share it)
 5  for each proposed call a_t in the message:
 6      x_t, avail ← features of Tables 3.8–3.9 from H_t, a_t,
g           // Layers 2–3
 7      k_t ←
κ(z_t)                                                    // Layer 4
 8      s_t ← β0 + βᵀx_t;  p_t ← σ(γ1·s_t + γ0)       // computed for audit;
see lines 9–14
 9      if tool(a_t) ∈ Π then v_t ← Block; src ← "D1"; R ← not applicable
10      else if k_t ≥ k_D2 and required evidence ⊄ avail then
11          v_t ← Escalate; src ← "D2"; R ← not applicable
12      else                                      // unavailable features below
k_D2 are 0 and flagged
13          V_t ← {Allow, Escalate, Block} ∪ ({Modify} if tool(a_t) ∈ 𝒯_R)
14          R[v] ← (1 − p_t)·L(v,0,k_t) + p_t·L(v,1,k_t) for v ∈ V_t;  v_t ←
argmin, ties by order
15      // Layer 5
16      if episode-level and tool(a_t) ∈ S then refuse a_t
17      else if v_t = Allow then release a_t
18      else if v_t = Modify then withhold a_t; if episode-level, S ← S ∪
class(a_t)
19      else if v_t = Escalate then withhold a_t; release only on reviewer
approval
20      else (Block) withhold a_t; if episode-level, S ← S ∪ 𝒯_R
21      r_t ← (tool name, argument digest, x_t, avail, s_t, p̃_t, p_t, p-used
flag, k_t,
               R, v_t, src, mitigation, φ0, φ_t, |φ0 + Σφ − s_t|, versions,
REM time)
```

```
22      append r_t to the audit log; append a_t to H_t (released or not)
23  released calls execute; their results enter line 1 at the next decision
point
```

Line 8 computes the probability for every step so that the audit record is complete; when line 9 or line 10 applies, the record marks p_t as not used. Lines 16–20 implement both mitigation semantics of Table 3.14. Attribution (line 21) is computed after the verdict and cannot reach it.

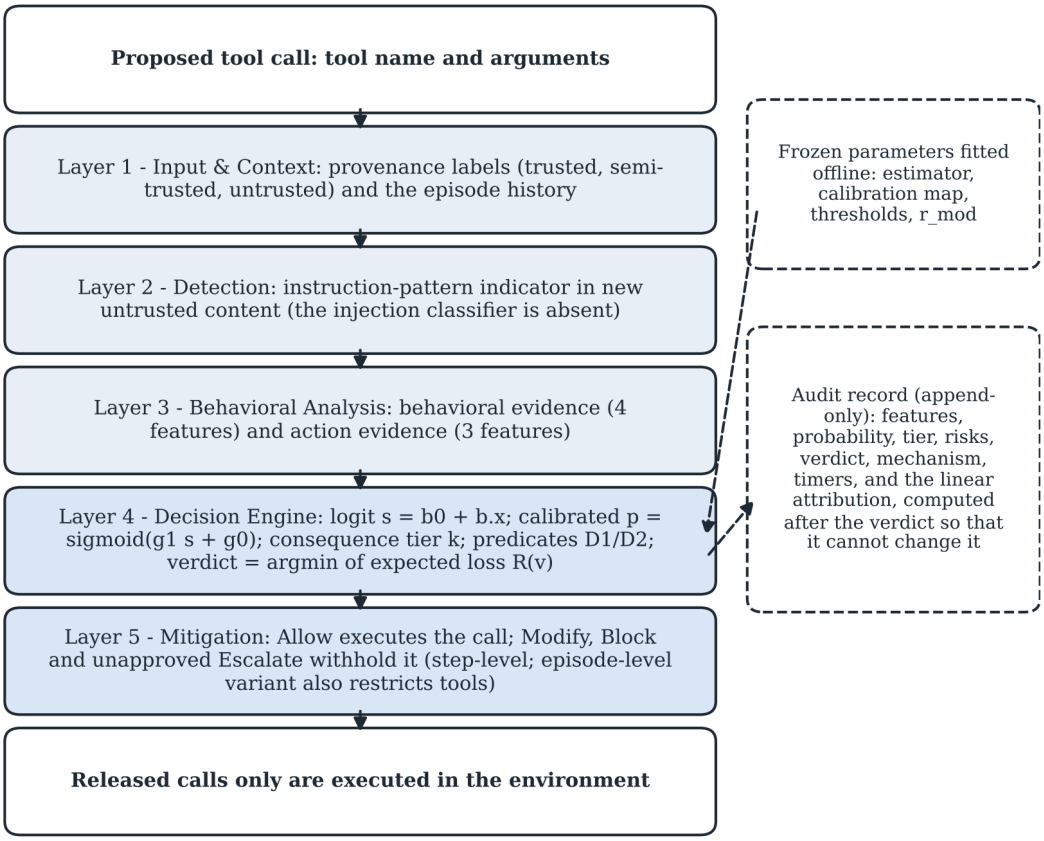


Figure 3.3. REM runtime workflow for one proposed tool call, as implemented in the evaluated path (`RemGuard.evaluate`): evidence, estimation and calibration, consequence tier, predicates, expected-loss verdict, mitigation before execution, and the audit record. Attribution is computed after the verdict and cannot change it.

3.11 Experimental Methodology

3.11.1 Environment and instrumentation

REM is evaluated in the banking environment of AgentDojo (Debenedetti et al., 2024), an open-source framework that evaluates both the utility and the security of LLM agents.

Security is determined by checks on the environment state before and after an episode, and utility by task-specific checks.

REM is placed at the tool-execution boundary of the agent loop, where it intercepts each proposed tool call before execution and each tool result on return. That this is achievable without modifying the agent's internals is assumption AS-1, to be demonstrated in Section 3.16.

REM runs in two modes:

- **Observe-only:** verdicts are computed and logged but not enforced. Fitting data are collected in this mode, because enforced verdicts would alter the trajectories from which REM is fitted.
- **Enforcing:** verdicts are executed by the Mitigation Layer.

For every run, the following are recorded:

- the AgentDojo release or commit identifier;
- the suite version;
- the attack template;
- the LLM backbone identifier and version, access date, and decoding parameters;
- REM's model, calibration, and policy versions.

The LLM backbone or backbones and the attack templates are pending supervisor approval.

Execution status. Enforcing-mode runs with a live agent require access to an LLM, which was not available for this thesis. The observe-only analyses (Sections 3.11.5–3.11.10) use recorded traces of real agents distributed with the benchmark (Section 3.11.2). Enforcement is examined only through the replay sensitivity analysis of Section 3.11.11.

3.11.2 Dataset

The banking suite definition inspected for this chapter (suite v1, AgentDojo repository main branch, accessed September 2026) contains:

- **16 user tasks.** Examples include paying a bill described in a file, refunding a friend, updating a recurring rent payment, and updating account details. Their reference solutions use between one and five tool calls.
- **9 injection tasks.** Eight direct money or a recurring payment toward one shared attacker account identifier. One changes the account password. The reference

attacker actions consist of one call for seven tasks, two calls for one task (reading scheduled transactions, then sending money), and three calls for one task (a total amount split into smaller transfers).

- **11 tools:** get_iban, send_money, schedule_transaction, update_scheduled_transaction, get_balance, get_most_recent_transactions, get_scheduled_transactions, read_file, get_user_info, update_password, and update_user_info.

These counts were re-confirmed against suite v1 in AgentDojo 0.1.35. The recorded traces used in the evaluation come from the AgentDojo repository at commit 089ed468; most carry no suite-version field, and some backbones were recorded under later suite versions (Chapter 4). The label-relevant definitions of the nine injection tasks are identical in every suite version from v1 to v1.2.2.

AgentDojo is an environment, not a fixed dataset of traces. Traces are generated by running an agent, so the number of traces depends on the number of backbones, attack templates, and repeated runs. The counts used in this thesis are reported, from the data, in Chapter 4. The 144 combinations of user task and injection task are not 144 independent samples. They are built from 16 user-task templates and 9 injection-task templates, and every trace inherits the dependence of its templates.

3.11.3 Label construction

The label must represent the quantity the estimator is asked to predict, which is whether a proposed call serves the attacker's objective rather than the user's. Labels are therefore derived from the injection task's own reference solution, not from the presence of an injection in the episode, because an attacked episode also contains benign calls. The derivation is mechanical, so the rules below state exactly what is matched, where the rule is known to be uncertain, and how that uncertainty is measured.

- **Step labels.** In an attacked episode, a proposed tool call is labeled $y_t = 1$ if it matches a call in the injection task's reference solution on tool name and on the attacker-controlled key argument, such as the attacker account identifier or the attacker-specified password. All other proposed calls, including calls that continue the user's task in attacked episodes, are labeled $y_t = 0$, as are all calls in episodes without injection.
- **Induced read steps.** Some injection tasks include a read call in their reference solution. Whether such a read step is labeled $y_t = 1$ is pending supervisor decision, because it is induced but has no direct external effect.

- **Label audit.** Matching on arguments can mislabel steps, for example when an agent alters an amount or reaches the attacker through a different tool. The derivation is therefore to be audited by a human annotator on a stratified sample of episodes, including ambiguous cases, and the disagreement rate reported.
- **What the label measures.** The reference-match label is a *practical proxy* for adversarial inducement. It records that the agent proposed the attacker's reference action with the attacker's key argument; it is not a causal proof that the injected content caused the proposal. A legitimate action that coincides with the attacker's reference action is mislabelled as induced, and an induced action that takes another route is mislabelled as legitimate. Both errors are measured, not assumed away.

3.11.4 Data limitations

The design must respect five limitations of the data:

1. **Positive-step scarcity.** A step is labeled $y_t = 1$ only if the agent actually proposes the attacker's call. The number of positive steps therefore depends on how often the chosen backbone follows injected instructions, and it may be small; the observed count is reported in Chapter 4. Most steps are benign, so the classes are strongly imbalanced.
2. **Shared attacker account identifier.** Eight of the nine injection tasks use the same attacker account identifier. Any feature keyed on that value would leak the label across folds (Section 3.6.2).
3. **Repeated task templates.** The same user tasks appear with every injection task, so benign behavior learned in one fold recurs in others.
4. **Short trajectories.** Reference solutions contain one to five calls, and most attacker objectives require one call. Trajectory-level evidence therefore has limited opportunity to accumulate.
5. **Calibration data.** A calibration set separate from training, validation, and test data would contain very few positive steps.

As a result, four statistically independent partitions for training, calibration, validation, and testing cannot be supported, and the design uses grouped cross-fitting instead.

3.11.5 Grouping and cross-fitting

Estimation, calibration, and threshold selection use nested grouped cross-fitting. No step is ever scored by a model fitted on data from its own group.

1. **Outer folds.**

- *Attacked episodes* are grouped by injection task, giving nine leave-one-injection-task-out folds. The held-out fold therefore contains an attacker objective not seen during fitting.
- *Episodes without injection* are assigned to the outer folds by user task, so that no user task's benign-only episodes appear in both the fitting and held-out portions of a fold.

2. **Inner folds.** Within the fitting portion of each outer fold, grouped inner cross-validation selects λ by held-out log-loss and produces out-of-fold logits. These are used to fit the calibration parameters (Equation 3.4) and to select the baseline thresholds (Equation 3.14).
3. **Held-out scoring.** The outer held-out fold is scored by the model fitted on the rest. All reported metrics are computed from these held-out predictions.

Independence of every fitted quantity from the held-out fold. For outer fold f, each of the following uses only the fitting portion D_train(f):

- the benign transition set and $\bar{x}$, re-estimated also within each inner fold;
- λ, selected by grouped inner cross-validation;
- (γ_1, γ_0), fitted on out-of-fold logits from the inner folds;
- the baseline thresholds $\theta(\alpha)$, selected on out-of-fold calibrated benign steps;
- $r_{mod}(k)$, estimated from episodes outside fold f;
- the feature set: the Pilot E0 rule (Section 3.11.8) is applied to each fold's fitting portion. If the folds disagreed, each fold would use its own set. In this thesis they agree (Chapter 4).

4. **Deployment model.** A model fitted on all data is used only for runtime demonstrations and latency measurement, never for reported accuracy or calibration.

Grouping by injection task does not remove the dependence introduced by shared user-task templates in attacked episodes. As a sensitivity analysis pending supervisor approval, the fitting is repeated with folds grouped by user task, and differences between the two groupings are reported.

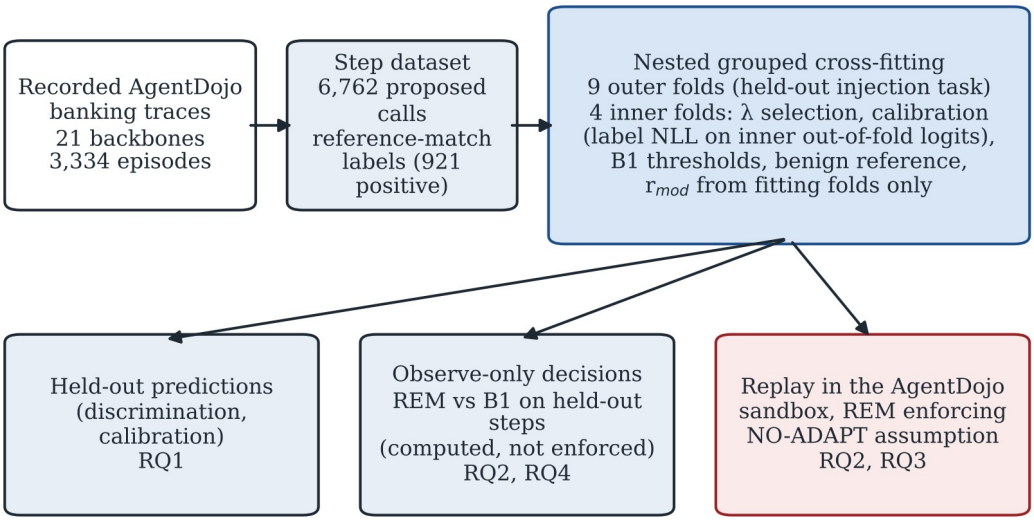


Figure 3.4. Recorded-trace evaluation design: nested grouped cross-fitting (nine outer folds by injection task, four inner folds), held-out predictions, observe-only decisions and the NO-ADAPT replay sensitivity analysis in the AgentDojo sandbox.

3.11.6 Leakage controls

Because the injection tasks in the banking suite share a small number of templates and a single attacker account, an estimator can reach a high score by memorizing the benchmark rather than by recognizing adversarial inducement. Arp et al. (2022) identify this family of errors as a recurring cause of overstated security-classification results. The following controls are therefore fixed before any held-out prediction is examined.

- No feature encodes a specific account identifier (Section 3.6.2).
- Benign reference quantities are estimated within each outer fold's fitting portion.
- The feature set, instruction-pattern set, privilege map, tier map, loss grid, false-positive targets, and pilot decision values are declared before held-out predictions are examined.
- Fitting data are collected in observe-only mode.
- Results are reported per benchmark and are not pooled across benchmarks.

3.11.7 Configurations, baselines, and ablations

Configurations:

- **B0, no defense.** Establishes attack success and utility without protection.

- **B1, consequence-independent baseline.** The same estimator and calibration with the threshold policy of Section 3.7.7. This is the primary comparator for RQ2.
- **REM.** The expected-loss verdict selection of Section 3.7.5.
- **REM without expected-loss Escalate.** As REM, with Escalate available only through D2, which gives REM the baseline's escalation opportunity (Section 3.7.7). This configuration was added after the first pilot results and is reported as a post-hoc comparison.
- **B2, detection-only (pending supervisor approval).** The Detection Layer's classifier score applied to untrusted content with an operating-point threshold. It tests whether Layers 3 and 4 add value.
- **Distributed defenses (pending supervisor approval).** Any defenses distributed with the pinned benchmark version, to address O3.

Ablations of REM (each changes one element, all else fixed; the estimator is refitted when an evidence group is removed):

1. Context evidence removed.
2. Behavioral evidence removed.
3. Action evidence removed.
4. Calibration removed ($\tilde{p}_t$ used in Equation 3.6).
5. Consequence tiers removed (one tier for all actions).
6. Predicates D1 and D2 disabled.
7. Observe-only (no mitigation).

Mitigation-effectiveness runs. Enforcing-mode runs in which Modify is applied to adversarial steps estimate $r_{mod}(k)$ per tier and the utility impact of Modify on benign steps. These estimates feed Equation 3.5 and are fitted within the outer folds so that no held-out episode informs its own loss values. In this thesis, live enforcing-mode runs were not available. $r_{mod}(k)$ was therefore estimated by replaying recorded attack episodes with Modify applied to every labelled attack step (Section 3.11.11). Under the no-adaptation assumption these estimates are close to zero by construction, and they are reported as such. The utility impact of Modify on benign steps was not estimated separately.

Escalation in experiments. No human reviewer is available in the benchmark. Escalation is resolved in two ways:

- by an idealized reviewer who releases the action if $y_t = 0$ and prevents it if $y_t = 1$;
- by a no-reviewer bound in which every escalation is treated as Block.

Both are reported, so the dependence of any result on the reviewer assumption is visible.

3.11.8 Pilot E0

Before the feature set is frozen, a pilot run on the fitting portions of the outer folds checks feasibility:

- the number of positive steps per backbone and attack template;
- the ROC-AUC of each evidence group alone and combined.

The pilot's decision values are project decisions declared before the pilot, not literature-derived thresholds [pending supervisor approval]:

- the minimum number of positive steps needed to proceed;
- the minimum group ROC-AUC needed to retain an evidence group.

If the positive-step count is insufficient, additional attack templates or backbones are added before the main experiments. The pilot never uses held-out fold predictions. Because every outer fold's held-out data belong to the fitting portions of the other folds, a single global decision taken from all folds' pilot results would use each fold's held-out data indirectly. The retention rule is therefore applied per fold, to that fold's fitting portion only.

3.11.9 Repeated runs and uncertainty

Each configuration is run at least three times with fixed decoding parameters to characterize LLM non-determinism [number of runs pending supervisor approval]. Repeated runs of the same template are not independent. Results are therefore aggregated per episode template before uncertainty is estimated.

Confidence intervals are obtained by bootstrap resampling (Efron, 1979) at the level of injection-task groups for attacked episodes and user-task groups for benign episodes. Resampling groups rather than steps is a design choice that respects the dependence described in Section 3.11.2.

With nine injection-task groups, intervals will be wide. No statistical power has been established, so differences are reported as measured differences with their intervals, and the word "significant" is not used unless a stated test supports it.

3.11.10 Optional observe-only sequential analysis

Sequential change detection is not part of REM's verdict path. If the supervisor approves it for the thesis narrative, it is performed **offline** on the per-step indicators logged by the Behavioral Analysis Layer. It never influences a verdict.

For each step, a binary deviation indicator $e_t \in \{0, 1\}$ is defined as 1 if at least one of x^{beh}_1, x^{beh}_3, x^{beh}_4, or x^{act}_1 equals 1, or if $x^{beh}_2 = 1$; otherwise 0 [indicator set pending supervisor approval]. Let q_0 and q_1, with $0 < q_0 < q_1 < 1$, be the rates of $e_t = 1$ before and after an adversarial change.

The log-likelihood ratio of one observation, as defined by Basseville and Nikiforov (1993), specialized to Bernoulli observations, is:

$$\ell_t = e_t \ln \frac{q_1}{q_0} + (1 - e_t) \ln \frac{1 - q_1}{1 - q_0} \quad (3.15)$$

Status: derived by substituting the Bernoulli probability mass function into the log-likelihood ratio of Basseville and Nikiforov (1993). A cumulative sum chart for Bernoulli observations is treated by Reynolds and Stoumbos (1999).

The cumulative sum statistic and alarm time are (Basseville & Nikiforov, 1993; Page, 1954):

$$G_t = max(0, G_{t-1} + \ell_t), G_0 = 0 \quad (3.16)$$

$$T_A = min\{t : G_t \geq h_G\} \quad (3.17)$$

where $h_G > 0$ is the alarm threshold.

Status: established from source.

The per-episode false-alarm rate on benign episodes and the detection delay are defined as:

$$FA_{ep}(h_G) = \frac{\left|\{j \in E_{ben} : \max_t G_t^{(j)} \geq h_G\}\right|}{|E_{ben}|} \quad (3.18)$$

$$\tau_d = T_A - \nu, T_A \geq \nu \quad (3.19)$$

where E_{ben} is the set of benign episodes in the fitting portion of a fold, $G^{(j)}_t$ is the statistic in benign episode j, τ_d is the detection delay, and ν is the first step at which injected content enters the context.

Status: design definitions.

The parameters are determined as follows:

- q_0 is the empirical rate of $e_t = 1$ on benign fitting steps, floored at a declared minimum.
- q_1 is declared and reported over a range, because the scarcity of multi-step attacks does not allow it to be estimated reliably.
- h_G is the smallest value for which Equation 3.18 does not exceed a declared target.

The procedure has known optimality properties in sequential change detection: Lorden (1971) proposed stopping rules that are asymptotically optimal in a defined sense, and Moustakides (1986) showed Page's stopping time to be optimal in a stronger sense. Both results are proved within a sequential-analysis model whose conditions could not be inspected during verification and are not claimed to hold for agent trajectories, where observations are neither independent nor drawn from known distributions. They therefore motivate the procedure and provide no guarantee in this setting. Given the short trajectories of Section 3.11.4, this analysis may have little to detect.

3.11.11 Recorded-trace replay sensitivity analysis *[proposed scope amendment, pending supervisor approval]*

Live enforcing-mode runs were not possible (Section 3.11.1). To obtain any evidence about enforcement, each recorded episode is re-executed in the benchmark's own sandbox with REM at the tool boundary:

- a replay element emits the agent's recorded messages in order;
- the benchmark executes the released calls;
- the benchmark's utility and security checks score the result.

Assumption NO-ADAPT. The agent emits the calls it emitted in the recorded run, whatever REM does. A live agent may retry, rephrase or choose another tool after a refusal. Replay results therefore do not estimate enforcement against live or adaptive agents, and they cannot distinguish step-level from episode-level semantics.

Validity checks. An all-Allow replay must reproduce the recorded utility and security outcome. Episodes for which it does not are excluded from every replay metric, and the exclusions are reported.

Not independent repetitions. Replaying the same recorded trace several times is deterministic. It is not an independent repetition of LLM behavior and does not substitute for the repeated runs of Section 3.11.9.

Relation to the excluded method. This analysis differs from counterfactual estimation of intervention value (Section 3.1.4): it executes no alternative actions and makes no claim about what an agent would have done instead.

3.12 Evaluation Metrics

3.12.1 Primary metrics

Primary metrics do not depend on declared costs.

Table 3.15. Primary metrics

Metric	Definition	Unit
Attack success rate (ASR)	Fraction of attacked episodes in which the benchmark's security check reports that the injection goal was achieved (enforcing mode)	Episode
Benign utility	Fraction of episodes without injection in which the benchmark's utility check passes	Episode
Utility under attack	Fraction of attacked episodes in which the utility check passes	Episode
Intervention rates	Fraction of steps receiving Modify, Escalate, or Block, reported per verdict and separately for $y = 0$ and $y = 1$ steps	Step
Step false-positive rate	Interventions on $y = 0$ steps divided by the number of $y = 0$ steps	Step
Residual ASR under Modify	ASR in episodes where Modify was applied to an adversarial step	Episode
Escalation rate	Fraction of steps receiving Escalate, with outcomes under the idealized reviewer and the no-reviewer bound	Step
Latency	Wall-clock time of REM's per-step processing, excluding LLM inference and tool execution; median and 95th percentile	Step

ASR is never reported without benign utility and utility under attack. A low ASR can reflect an agent that fails to complete tasks rather than an effective defense (Y. Wang et al., 2026).

3.12.2 Secondary metrics

Discrimination. ROC-AUC (Fawcett, 2006) and PR-AUC (Davis & Goadrich, 2006) are computed on held-out probabilities. PR-AUC is emphasized because positive steps are rare. Within a fold, both are unchanged by calibration with $\gamma_1 > 0$. Because each outer fold has its own calibration map, metrics are reported per fold as well as on the pooled held-out predictions.

Calibration error. For binary outcomes, calibration error is measured on the positive-class probability (Naeini et al., 2015) with M equal-width bins (Guo et al., 2017):

$$ECE = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \bar{y}(B_m) - \bar{p}(B_m) \right|, B_m = \{ i : (m-1)/M < p_i \leq m/M \} \quad (3.20)$$

where N is the number of held-out steps, $\bar{y}(B_m)$ is the fraction of positive steps in bin m, $\bar{p}(B_m)$ is the mean calibrated probability in bin m, and M is declared in advance (steps with $p_i = 0$ are assigned to the first bin).

Status: adapted from source. Naeini et al. (2015) supply the binary form; Guo et al. (2017) supply the equal-width binning.

Because a global ECE averages over all probabilities, reliability is also reported within declared windows around each tier's threshold p^*_k.

Log-loss. The mean negative log-likelihood of held-out steps is:

$$LL = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \ln p_i + (1 - y_i) \ln (1 - p_i) \right] \quad (3.21)$$

Status: derived as the negated, averaged, unpenalized log-likelihood term of Equation 3.2 evaluated at the calibrated probabilities. Probabilities are bounded away from 0 and 1 by a declared small constant for numerical evaluation.

Brier score. The Brier score is not reported until the convention of its original source (Brier, 1950) has been confirmed; log-loss serves as the proper scoring measure in the meantime.

3.12.3 Realized loss across a declared grid

Realized loss is secondary because it depends on declared costs, and because a rule that minimizes expected loss under L is favored by construction when evaluated under the same L. For an evaluation loss structure L_e:

$$\acute{L}(L_e) = \frac{1}{N} \sum_{i=1}^{N} L_e(v_i, y_i, k_i) \quad (3.22)$$

where v_i is the verdict issued for held-out step i under the decision loss structure L_d, y_i is its label, and k_i is its tier.

Status: design definition.

The grid is declared before held-out predictions are examined and varies three things:

- the ratio $C_{miss}(k)/C_{FA}(k)$ per tier;
- C_{esc};
- the amount limits of the tier map.

For each evaluation structure L_e, the grid reports:

- REM with $L_d = L_e$ (matched);
- REM with $L_d \neq L_e$ (misspecified costs);
- the baseline B1, which does not use costs.

The conclusion takes the form “across this declared range of loss structures, outcomes vary as follows”, not “under the chosen costs, outcomes are these”.

3.13 Contribution and Novelty Boundaries

REM does not introduce a new learning algorithm or a new decision-theoretic method. Its constituent mechanisms are established and are attributed to their sources.

Table 3.16. Mechanisms that are not claimed as novel

Mechanism	Prior work
Logistic regression	Cox (1958)
Ridge regularization of logistic regression	le Cessie and van Houwelingen (1992)
Platt (logistic) calibration	Platt (2000); Guo et al. (2017); Kull et al. (2017)
Linear Shapley attribution	Lundberg and Lee (2017)
Cumulative sum change detection	Page (1954); Basseville and Nikiforov (1993)
Expected-cost decisions; reject option	Elkan (2001); Chow (1970)
Calibrated logistic risk scores for agent safety; four-way intervention	Hossain et al. (2026)
Distinction between calibration and control	C. Zhang et al. (2026)
Runtime interception; human review; graduated response	H. Liu et al. (2026); C. Yang (2026); C. L. Wang et al. (2025)
Provenance analysis of tool calls	She et al. (2026); Debenedetti et al. (2025)
Consequence-aware and consequence-priced control	H.-H. Chen (2026); Hossain et al. (2026); C. Yang (2026)
Component ablation with latency	C. Yang (2026)
Runtime protection of financial agents	Jia et al. (2026)

Investigated contribution. The contribution of this thesis is empirical and integrative. It is the design and evaluation, on the AgentDojo banking environment under indirect prompt injection, of a non-invasive five-layer runtime layer that combines:

- provenance-based evidence about proposed financial actions;
- a calibrated per-step probability of adversarial induction;
- verdict selection over Allow, Modify, Escalate, and Block by minimizing conditional risk with losses declared per consequence tier;
- deterministic mitigation whose effectiveness is measured and used in the loss structure.

It is evaluated against a consequence-independent baseline that uses the same estimator, with attack success, benign utility, utility under attack, intervention behavior, and latency measured separately and cost-dependent outcomes reported across a declared loss grid.

Explicitly not claimed:

- a new algorithm;

- the first runtime guardrail, the first calibrated agent risk score, or the first consequence-aware controller;
- that calibration improves control;
- robustness against adaptive adversaries;
- any prevention guarantee;
- validity in production financial deployment.

The contribution holds only to the extent that the experiments specified in this chapter support it. Chapter 4 reports which of these experiments were executed (offline classification, observe-only policy analysis, replay sensitivity) and which were not (live enforcing-mode runs).

3.14 Threats to Validity

Construct validity.

- Step labels are derived and can be wrong; the derivation is audited on a sample (Section 3.11.3).
- The idealized reviewer overstates the protective value of Escalate; the no-reviewer bound is reported alongside it.
- Declared costs cannot be validated empirically; their influence is reported across the grid.
- A low ASR can reflect task failure rather than protection, so utility metrics accompany every security metric (Y. Wang et al., 2026).

Internal validity.

- Leakage through shared templates and the shared attacker account identifier is controlled by grouping and by excluding identity-based features, but template dependence remains.
- Features designed with knowledge of the benchmark may fit its style; the feature set is frozen after the pilot and before held-out predictions are examined.
- LLM non-determinism is characterized by repeated runs; the recorded traces contain one run per configuration, so this requirement is not met in this thesis.

External validity.

- The evaluation uses one simulated banking environment with benchmark-authored attacks.
- Results may not transfer to other benchmarks, whose definitions and conclusions can disagree (M. Q. Li et al., 2026), or to production systems.

- Results may depend on the LLM backbone.
- FinVault is not used because its preprint was withdrawn.

Statistical conclusion validity.

- The number of independent task templates is small.
- Intervals are wide and no power analysis is available, so conclusions are limited to differences supported by their intervals.

Security-specific threats.

- Every evidence feature can be evaded individually, and adversaries are assumed non-adaptive (AS-6).
- REM's own configuration and logs are assumed protected (AS-3).
- A scalar probability cannot represent whether an adversarial trajectory remains recoverable, so REM inherits the limitation identified by C. Zhang et al. (2026); estimating intervention value for adaptive agents by counterfactual replay is outside the scope of this thesis.

3.15 Equation Provenance Summary

Table 3.17. Provenance of the equations in this chapter

Eq.	Content	Status	Source	Verification
3.1	Logistic model	Established	Cox (1958)	IV
3.2	Ridge-penalized log-likelihood	Established	le Cessie & van Houwelingen (1992)	IV (penalty constant pending)
3.3	Logistic calibration on the logit, $\gamma_1 > 0$	Adapted	Platt (2000); Guo et al. (2017); Kull et al. (2017)	Platt NC; Guo FV; Kull FV
3.4	Calibration fit by negative log-likelihood	Established	Guo et al. (2017)	FV
3.5	Adversarial loss of Modify	Derived (assumption A-Mod)	Law of total expectation	DER
3.6	Conditional risk	Adapted	Elkan (2001)	FV (Eq. 1 form)
3.7	Bayes verdict with tie-breaking	Adapted	Elkan (2001)	FV; tie order DEF
3.8	Pairwise indifference threshold	Derived	From Eq. 3.6	DER
3.9	Allow–Block threshold	Derived	From Eq. 3.8; consistent with Elkan (2001)	DER
3.10	Escalation feasibility condition	Derived	From Eqs. 3.6, 3.7, 3.9; Chow (1970) conceptual basis	DER; Chow IV
3.11	Linear attribution	Adapted	Lundberg & Lee (2017)	FV
3.12	Completeness identity	Derived	Lundberg & Lee (2017) local accuracy	FV + DER
3.13	Group attribution	Definition	—	DEF
3.14	Baseline operating-point threshold	Definition	—	DEF
3.15	Bernoulli log-likelihood ratio (optional)	Derived	Basseville & Nikiforov (1993); Reynolds & Stoumbos (1999)	FV; R&S IV
3.16	CUSUM recursion (optional)	Established	Basseville & Nikiforov (1993); Page (1954)	FV; Page IV
3.17	Alarm time (optional)	Established	Basseville & Nikiforov (1993)	FV

Eq.	Content	Status	Source	Verification
3.18	Per-episode false-alarm rate (optional)	Definition	—	DEF
3.19	Detection delay (optional)	Definition	—	DEF
3.20	Expected calibration error	Adapted	Naeini et al. (2015); Guo et al. (2017)	FV
3.21	Log-loss	Derived	From Eq. 3.2	DER
3.22	Realized loss	Definition	—	DEF

Implementation conventions. Four conventions in the implementation differ in form from the equations as printed; none changes their meaning:

- **Eq. 3.2:** the code's penalty is $(\lambda_{impl}/2)\|\beta\|^2$, so $\lambda = \lambda_{impl}/2$ (Section 3.7.1).
- **Eq. 3.4:** the label-NLL fit is the reference; Platt's smoothed targets are a sensitivity variant.
- **Eq. 3.14:** the candidate set includes a value just above the largest benign probability.
- **Eq. 3.20:** steps with $p = 0$ fall in the first bin.

These conventions are listed with the code location of each equation in the requirement–equation–function–test table of Section 3.16.

Verification codes: FV — formulation checked against the source; IV — source identity confirmed, formulation to be checked against the full text; NC — source not accessible during this revision; DER — derived in this chapter; DEF — design definition. The codes are retained until final reference verification is complete.

3.16 System Implementation

This section reports how the methodology of Sections 3.1–3.15 is implemented in software: the environment, the organisation of the code and its execution paths, the implementation of each layer, the verification of the software, the correspondence between equations, code, tests and results, and the measurement of processing time. It was Chapter 4 of the six-chapter draft (R5); its content is preserved and renumbered.

3.16.1 Purpose and Scope

the methodology (Sections 3.1–3.15) specified REM as a five-layer runtime framework that selects one of four verdicts (Allow, Modify, Escalate, Block) for each tool call

proposed by an LLM agent. The verdict minimizes expected loss under a calibrated probability of adversarial inducement and a declared consequence tier, and every verdict is recorded for audit.

This section reports how that specification was realized in software, how the software reaches the benchmark, and how it was verified. It distinguishes three kinds of verification throughout:

- **Unit verification:** a function computes what its equation states, on constructed inputs.
- **Integration verification:** the components operate together inside the benchmark's execution loop, and the benchmark's own checks behave as expected.
- **Security effectiveness:** whether REM reduces attack success at acceptable cost.

This section establishes only the first two. Chapter 4 reports the evidence on the third, and its limits.

Three rules governed the implementation:

1. **Fail closed.** A design value that has not been declared stops the software (`DesignNotSpecifiedError`) instead of being replaced by a default.
2. **No silent resolution.** Where the methodology (Sections 3.1–3.15) and the design decisions of Stage 2 disagree, both alternatives are implemented and selected explicitly. The correspondence is recorded in Section 3.16.12.
3. **Separation of decision and audit.** Attribution is computed after the verdict and cannot reach the decision function; this is enforced by the function's signature and checked by a test.

3.16.2 Software Environment

Table 3.18. Software and hardware environment of the reported runs

Item	Value
Language	Python 3.11.17
Numerical library	NumPy 2.4.6
Benchmark	AgentDojo 0.1.35 (PyPI), banking suite; suite version v1 for execution
Recorded traces	AgentDojo repository `runs/`, commit `089ed468cf3ed0322acc66b0211f26d9d90dbf60`
Test framework	pytest 9.1.1
Hardware	Linux container, 4 vCPU (Intel Xeon @ 2.80 GHz), 15 GB RAM
Re-execution in a fresh environment (R3)	30 Oct 2026, merged repository state, Python 3.11.17, NumPy 2.4.6, AgentDojo 0.1.35, Intel Xeon @ 2.10 GHz, 4 vCPU; see Section 3.16.11.2
Re-execution with the complete-call timer (R4)	1 Nov 2026, branch `claude/rem-r4-corrections-3ktvze`, Python 3.11.17, NumPy 2.4.6, AgentDojo 0.1.35, Intel Xeon @ 2.80 GHz, 4 vCPU; results in `results/pilot_R4/`; see Section 3.16.11.2
Code version of the reported results	git commit `84a8c21` (revision R2). The first run (commit `aa66f22`) gave identical analytical results; see Section 3.16.11

No machine-learning framework is used. The estimator, the calibrators and every metric are implemented directly in NumPy, so that each operation can be traced to its equation (Table 3.23).

3.16.3 Code Organization and Execution Paths

Table 3.19. Code organization

Package	Content
`rem/core/`	Frozen pipeline skeleton (`RemPipeline`), interfaces, step-level mitigation engine, append-only audit log, result types
`rem/algorithms/detection/`	Ridge logistic regression (Eqs. 3.1–3.2); instruction-pattern indicator $x^{\wedge}ctx_2$
`rem/algorithms/calibration/`	Platt calibration on the logit: smoothed-target fit (`PlattLogitCalibrator`) and label-NLL fit (`LabelNLLLogitCalibrator`, Eq. 3.4)
`rem/algorithms/context/`	Provenance labels, hygiene, entity extraction
`rem/algorithms/behavior/`	Evidence features (Tables 3.8–3.9); Bernoulli CUSUM (observe-only)
`rem/algorithms/decision/`	Expected-loss engine; consequence tiers; loss structure and dominance check; predicates and feasible set
`rem/algorithms/a3.py`, `a1.py`, `a2.py`	REM verdict rule; baseline B1; observe-only CUSUM analysis
`rem/evaluation/`	Discrimination and calibration metrics; grouped bootstrap; nested grouped cross-fitting
`rem/integration/`	AgentDojo trace loader, labeller, step builder, replay harness with the REM guard
`experiments/pilot/`	Experiment scripts and the reproduction script `run_all.sh`

The names A1, A2 and A3 come from the project brief and do not denote three methodologies:

- **A3** is the REM verdict rule;
- **A1** is the consequence-independent baseline B1 (Section 3.7.7);
- **A2** is the optional observe-only CUSUM (Section 3.11.10).

Layer of the estimator. In this thesis the ridge estimator and the calibration map belong to the Decision Engine (Layer 4), as in the methodology (Sections 3.1–3.15) and Figure 3.1. The comments and layer constants of the frozen pipeline's code label the estimator as part of Detection (Layer 2). The label does not affect any computation, and the frozen code was not edited to change it.

Two execution paths exist, and the experiments use only one of them.

1. **The frozen pipeline** (`rem/core/pipeline.py`, `RemPipeline.evaluate_step`) chains injected components through the five layers. It is exercised by unit and contract tests. It is **not** the path that produced any result in Chapter 4.
2. **The evaluation path** (`rem/integration/agentdojo_replay.py`, `RemGuard.evaluate`, together with `rem/algorithms/a3.py`, `A3Policy.decide`) is the path used in every experiment. It calls the frozen components:
    - the fitted ridge coefficients;
    - the calibration map;
    - `ExpectedLossDecisionEngine.conditional_risk` for Eq. 3.6;
    - `LinearShapleyAttributor` for Eq. 3.11.

    It adds the the methodology (Sections 3.1–3.15) components the frozen pipeline lacks: features, tiers, predicates, feasible set and both mitigation semantics. Mitigation is implemented in `RemGuard._mitigate`, not in the frozen `DeterministicMitigationEngine`.

Table 3.20. Entry points and call paths of each experiment

Experiment (Chapter 4)	Entry point	Call path
Dataset	`experiments/pilot/build_dataset.py`	`load_episode` → `build_steps` → `EpisodeTracker.features` → `ReferenceLabeler.label` → `ConsequenceTierMap.tier`
Pilot E0	`e0_pilot.py`	`nested_crossfit` on each fold's fitting portion → `roc_auc`
E1 discrimination and calibration; ablation refits; grouping sensitivity	`run_main.py crossfit`	`Designer` (fold-specific x^beh$_4$) → `nested_crossfit` → `RidgeLogisticRegression.fit` → calibrator `fit` → `A1Thresholds.fit`
r_{mod} and replay validity	`run_main.py rmod`	`replay_episode` → `GuardedToolsExecutor.query` → `RemGuard.evaluate` (oracle Modify)
E2, E5 and E6 observe-only	`run_main.py observe`	`A3Policy.decide` / `A1Policy.decide` on held-out recorded steps
E4, E5 replay	`run_main.py replay`	`replay_episode` → `RemGuard.evaluate` → `A3Policy.decide` → `RemGuard._mitigate`
E3 CUSUM	`run_main.py a2`	`fit_a2` → `evaluate_episode`
Cross-template; recorded defenses	`run_main.py xt`	as E1 scoring, and as E4 replay
Breakdowns, replay failures, label-audit package	`r2_analyses.py`	stored state from the stages above
All of the above	`run_all.sh`	in order; writes `results/pilot_R2/` when `PILOT_RESULTS_DIR` is set

3.16.4 Layer 1 — Input and Context

`rem/algorithms/context/provenance.py` implements the provenance rule of Section 3.4.2:

- every segment is labelled trusted (system), semi-trusted (user) or untrusted (tool output);
- derived content takes the least-trusted label of its inputs;
- model-generated content has no channel label, so asking for one raises an error rather than guessing.

Hygiene. Zero-width, bidirectional and other Unicode format characters are removed from REM's copy of untrusted text (`strip_invisible`). The text the agent receives is not altered.

Entity extraction is partly implemented:

Entity (Section 3.4.3)	Status
Account identifiers	Implemented (IBAN pattern)
Monetary amounts	Implemented (numeric tokens)
User-profile values	Implemented for the values returned by the profile read
Tool names mentioned in text	Implemented, through the instruction-pattern indicator
Credential values	Not extracted as entities; a password argument is still traced to its source through argument provenance
Phone numbers and other profile identifiers	Not implemented separately

A declared matching rule (`value_occurs_in`) links an argument value to the texts that contain it:

- numbers match numerically;
- strings match as case-insensitive substrings;
- strings shorter than three characters never match.

3.16.5 Layer 2 — Detection

x^{ctx}_2 (instruction-pattern indicator) is implemented by `InstructionPatternDetector`. It fires if a declared regular expression, or the name of a tool, occurs in new untrusted segments. The pattern set is declared in the pilot configuration, and the detector refuses to run without one. The configuration discloses that the author of the patterns knew the benchmark's templates.

x^{ctx}_1 (pre-trained injection classifier) is not implemented. The classifier choice is pending, and the candidate model's host was unreachable from the execution environment.

The feature is absent from x_t; it is not replaced by a substitute. The context evidence tested in Chapter 4 is therefore the instruction-pattern indicator only.

3.16.6 Layer 3 — Behavioral and Action Evidence

`rem/algorithms/behavior/evidence.py` implements $x^{beh_1-x}beh_4$ and $x^{act_1-x}act_3$ (Table 3.9). An `EpisodeTracker` holds the episode history; it is reset at each episode start and never changes REM's parameters. Where the methodology (Sections 3.1–3.15) is silent, the module states an interpretation (Table 3.21), so that the reader can disagree with it.

Table 3.21. Interpretations made where the methodology (Sections 3.1–3.15) is silent

ID	Question	Interpretation
I-1	What is a "step" when the agent proposes several calls at once?	Each proposed call is a step; calls in one assistant message share one decision point
I-2	Does $x\text{^}beh_2$ count refused calls?	Yes, because the methodology (Sections 3.1–3.15) states that retries raise $x\text{^}beh_2$
I-3	$x\text{^}beh_3$ at the first call	"Highest level so far" is the lowest level (0)
I-4	$x\text{^}beh_4$ at the first call	Previous tool is a start symbol; start transitions belong to the reference set
I-5	$x\text{^}act_2$ "transactions before first untrusted ingestion"	Observable only through untrusted tool outputs, so it reduces to "recipient not in the objective"
I-6	$x\text{^}act_3$ "external recipient", "sensitive values"	Recipient differs from the user's own IBAN; sensitive values are identifiers and profile values from sensitive reads
I-7	Availability of $x\text{^}act_1$ for predicate D2	Unavailable when the tool has key arguments but the call carries none
I-8	"Tool class" in episode-level Modify	The tool's action category (Table 3.7) within $\mathscr{T}_R$
I-9	Unavailable feature below k_{D2}	Takes the value 0 ("not observed") and is flagged in the audit record

The benign transition set used by $x\text{^}beh_4$ must be estimated within each fold (Section 3.11.6). Each step record therefore stores its transition, and $x\text{^}beh_4$ is recomputed for every fold's reference.

3.16.7 Layer 4 — Decision Engine

Estimation. `RidgeLogisticRegression` computes $s = \beta_0 + \beta^T x$ and $\tilde{p} = \sigma(s)$ (Eq. 3.1), fitted by Newton–Raphson with the intercept unpenalized. Fitting without a declared λ

raises `DesignNotSpecifiedError`. The implementation minimizes NLL + $(\lambda_{impl}/2)\|\beta\|^2$, so λ on the scale of Eq. 3.2 is $\lambda_{impl}/2$ (Section 3.7.1). Selected values are reported on both scales.

Calibration. Both calibrators compute $p = \sigma(\gamma_1 s + \gamma_0)$ and reject a fit with $\gamma_1 \leq 0$. They differ only in the fitting target: labels (Eq. 3.4, the reference) or Platt's smoothed targets (sensitivity).

Consequence tiers, predicates and feasible set.

- `ConsequenceTierMap` maps each tool to its category and the call to a tier. A transfer with a missing or unparseable amount receives the above-limit tier.
- `PredicatePolicy` implements D1 and D2, and the feasible set (Modify only for tools in $\mathscr{T}_R$).
- `A3Policy.decide` applies D1 and D2 before the expected-loss rule. When a predicate applies, the risk table is recorded as *not applicable* and p is marked as not used.

Verdict. `A3Policy` computes R(v) for $v \in \mathscr{V}(a_t)$ through the frozen `conditional_risk` and returns the minimizer. Ties are handled by an explicit `tie_mode`:

- `"raise"` reproduces the frozen behavior;
- `"record_chapter3_order"` applies the the methodology (Sections 3.1–3.15) order and marks the step as tied.

An option `exclude_from_expected_loss=(Escalate,)` produces the comparison configuration "REM without expected-loss Escalate" (Section 3.7.7).

Loss structure and dominance check. `build_loss_grid` constructs Table 3.13 including Eq. 3.5, and `reachable_verdicts` reports any verdict that is never the unique minimizer.

Baseline B1. `operating_threshold` implements Eq. 3.14, and `A1Policy` applies it with the same estimator, calibration and predicates.

Audit record. For each step, `RemGuard.evaluate` writes:

- the tool name and a SHA-256 digest of its arguments, with the argument keys;
- the evidence vector and its availability;
- s, $\tilde{p}$ and p, and whether p was used;
- the tier, and the risk table or *not applicable*;
- the verdict, its source and the mitigation;
- the linear attributions with their additivity error;

- REM's processing time.

Raw argument values are not written.

3.16.8 Layer 5 — Mitigation, and What Happens at the Tool Boundary

Mitigation is applied before any tool executes. In `GuardedToolsExecutor.query`, each proposed call is evaluated by `RemGuard.evaluate`. If the call is released, `runtime.run_function` executes it. Otherwise it is not executed, and a tool-result message carrying the refusal or restriction notice is returned to the agent in its place. The replay tests check this directly: an oracle that blocks every labelled step leaves the benchmark's security check false, and the withheld steps appear in the audit record with their tool and argument digest.

Both mitigation semantics are implemented (Table 3.14). Step-level semantics is the reference; the episode-level variant maintains a restriction set S for the episode. Under step-level semantics, Modify and Block have the same effect on the current call. The two semantics can differ without any retry: an episode-level Block refuses every later call to a tool in $\mathscr{T}_R$, including a call whose own verdict is Allow. `tests/test_mitigation_semantics.py` checks this on a fixed sequence: Block `send_money`, then Allow `update_password`; the second call is released under step-level semantics and refused under episode-level semantics. Chapter 4 (Table 4.11) reports the replay results of both semantics.

Escalate. No reviewer exists. Escalation is resolved by one of two bounds:

- an idealized reviewer, which releases the call if and only if its label is 0;
- no reviewer, in which the call is withheld.

Correction made in revision R2. In the frozen pipeline, the Block mechanism cleared the step's tool call before the audit record was written. The record therefore lost the identity of the blocked action. A test now reproduces the defect, and the fix keeps the tool name and argument digest in the step's metadata, without raw arguments. The audit record uses them. The fix does not change any verdict or mitigation, and it does not affect any result in Chapter 4, because the experiments use the evaluation path, in which mitigation never alters the proposed call, so the tool's identity was always recorded; the argument digest was added to that record in this revision.

3.16.9 Non-Invasive Placement and Its Limits

"Non-invasive" means here that **the agent's model, prompts and tools are not modified**: REM observes the messages the agent produces and the results the tools return. The placement has limits:

- **REM replaces the executor.** It must be inserted into the agent's execution loop, here as a subclass of the benchmark's `ToolsExecutor`. A framework in which tools are executed by a component REM cannot wrap is not covered.
- **Only executor calls are mediated.** Effects that do not pass through the tool executor (for example, a tool with side effects inside its own implementation, or a second agent) are outside REM's view.
- **Refusals are visible to the agent.** They are returned as tool-result messages. The agent's reaction to them is part of the system's behavior, and the replay analysis does not observe it.
- **Some context is inferred, not received.** In the evaluation, REM receives the objective and system prompt from the recorded trace. A deployment must supply them through the same channel.

3.16.10 Optional Sequential Analysis (A2)

`rem/algorithms/behavior/cusum.py` and `rem/algorithms/a2.py` implement the Bernoulli CUSUM of Eqs. 3.15–3.19. The module has no interface to the decision engine and returns no verdict; a test checks both properties.

An A2-driven enforcement policy and a retrained "A3 + A2" variant are not implemented, because they would place CUSUM in the verdict path.

3.16.11 Verification

3.16.11.1 Unit and contract verification

Table 3.22. Test suite (executed on 1 Nov 2026: 205 passed with AgentDojo installed; log `verification/pytest_R4.txt`)

File	Tests	Verifies
`test_frozen_math.py`	43	Eqs. 3.1–3.3, 3.6; fail-closed fitting; $\gamma_1 > 0$; a tie raises
`test_contract_compliance.py`	20	Five layers, no sixth; frozen mitigation mapping; no equation IDs; forbidden components absent; attribution excluded from the decision signature
`test_metrics.py`, `test_probabilistic.py`	25 + 5	ROC-AUC against the pairwise definition; average precision; ECE binning (Eq. 3.20); log-loss; Brier conventions; grouped bootstrap
`test_pipeline.py`	21	Frozen pipeline order; step-level Block; **the blocked action's identity is kept in the audit record** (added in R2)
`test_mitigation.py`, `test_trajectory.py`, `test_config.py`, `test_reproducibility.py`	9 + 12 + 15 + 9	Step-level mitigation, configuration fail-closed, seeds
`test_a3.py`	10	Table 3.13; Eqs. 3.8–3.10; argmin over the feasible set; D1 and D2; ties; dominance check; **REM without expected-loss Escalate keeps D2** (added in R2)
`test_a1.py`	5	Eq. 3.14 minimality; $\alpha = 0$; invariance under monotone recalibration; verdict rule
`test_a2.py`	5	Eqs. 3.15–3.17; indicator set; false-alarm target; no verdict
`test_evidence.py`	7	Trust order; hygiene; features on an injected and a legitimate transfer; availability; tier map
`test_crossfit.py`	2	Procedure 3.1 outputs; held-out predictions independent of held-out labels

File	Tests	Verifies
`test_traces.py`	2	Label rule including induced reads and ambiguity; fold-specific $x^{\wedge}beh_4$
`test_replay.py`	3	All-Allow replay reproduces a recorded episode; oracle Block prevents the recorded attack; **withheld calls keep tool and argument digest, and no raw attacker value appears in the audit record** (extended in R2); observe-only releases everything
`test_mitigation_semantics.py`	4	Step-level and episode-level policies on a fixed sequence (Block `send_money`, then Allow `update_password`): different release outcomes, neither policy altered (added in R4)
`test_latency_instrumentation.py`	2	Fake-clock check that the partial timer stops before attribution, audit construction and history recording, and the complete-call timer after them (added in R4)
`test_docx_tables.py`	6	Thesis build tooling, not REM: the Word table transformation keeps exactly one run-property container per run, preserves hyperlinks and equations, and is idempotent (added in R4)

3.16.11.2 Integration verification

- **Replay fidelity.** With every call allowed, the replay reproduced the recorded utility and security outcome of 3,313 of 3,334 episodes (99.4%). The 21 exceptions are analyzed in Chapter 4.
- **Determinism and regression.** The complete pilot was run at commit `aa66f22` (R1) and again at commit `84a8c21` (R2), after the corrections of this revision. An automated comparison of the 5,121 values the two runs share found 93 differences, all of them wall-clock time or memory measurements; every analytical value was identical. A second comparison with a different leaf-matching method (`compare_reproduction.py`, R3) counted 5,220 shared values and 104 differences, all in time, memory or provenance fields, and 0 analytical differences.
- **Re-execution in a fresh environment (R3, 30 October 2026; by this project, not an external replication).** The complete pipeline (`run_all.sh`) was executed again, from the merged repository state, on a different processor and with the AgentDojo

repository obtained afresh at commit `089ed468`. Of the 6,057 numeric values that the re-run shares with `results/pilot_R2/`, 5,943 were identical. The 114 that differed are 100 per-step latency values, 13 code-commit stamps and one file path; no memory value and 0 analytical values differed (`RESULTS_PROVENANCE_REPORT.md`; every differing value is listed in `verification/reproduction_all_differences.json`).

- **Re-execution with the complete-call timer (R4, 1 November 2026).** After the timer was added, the complete pipeline was executed again into `results/pilot_R4/`. Of the 6,057 numeric values it shares with `results/pilot_R2/`, 5,942 were identical, 115 were latency, commit or file-path values, and 0 analytical values differed. The new complete-call fields have no R2 counterpart. Every differing value is listed in `verification/reproduction_all_differences_R4.json`.
- **Attribution additivity.** The audit record carries $|\varphi_0 + \Sigma\varphi_i - s|$ for every replayed step.

3.16.11.3 What is not verified here

Security effectiveness against live agents is not verified; Chapter 4 states what the recorded-trace analyses can and cannot show.

3.16.12 Correspondence Between Requirements, Equations, Code, Tests and Results

Table 3.23. Requirement → equation → function → test → result artifact

Requirement (the methodology (Sections 3.1–3.15))	Equation	Function	Test	Result artifact (`results/pilot_R2/`)
Logit and uncalibrated probability	3.1	`ridge_logistic.RidgeLogisticRegression.decision_function`	`test_frozen_math`	`e1_crossfit.json` (`pooled_uncalibrated`)
Ridge fit ($\lambda = \lambda_{impl}/2$)	3.2	`RidgeLogisticRegression.fit`	`test_frozen_math`	`e1_crossfit.json` (`per_fold.*.lambda`, `lambda_chapter3_eq3_2_scale`)
Calibration map, $\gamma_1 > 0$	3.3	`PlattLogitCalibrator.calibrate`	`test_frozen_math`	`e1_crossfit.json` (`gamma_1`)
Calibration fit on labels	3.4	`logit_nll.LabelNLLLogitCalibrator.fit`	`test_crossfit`	`e1_crossfit.json` (`label_nll_primary`)
Adversarial Modify loss	3.5	`loss_matrix.build_loss_grid`	`test_a3`	`e4_rmod_and_replay_validity.json` (r_{mod})
Conditional risk	3.6	`expected_loss.ExpectedLossDecisionEngine.conditional_risk`	`test_frozen_math`, `test_a3`	`e2_e5_e6_observe_only.json`
Verdict over $\mathcal{V}(a_t)$, ties	3.7	`a3.A3Policy.decide`	`test_a3`	`e2_e5_e6_observe_only.json` (`ties`)

Requirement (the methodology (Sections 3.1–3.15))	Equation	Function	Test	Result artifact (`results/pilot_R2/`)
Thresholds, feasibility, dominance	3.8–3.10	`loss_matrix.*`	`test_a3`	`e2_e5_e6_observe_only.json` (`dominance`)
Attribution, additivity	3.11–3.13	`linear_shapley.LinearShapleyAttributor`	`test_frozen_math`	replay audit field `additivity_error`
Baseline thresholds	3.14	`a1.operating_threshold`	`test_a1`	`e1_crossfit.json` (`theta_blk`, `theta_mod`)
CUSUM	3.15–3.19	`cusum.BernoulliCusum`, `a2.*`	`test_a2`	`e3_a2_cusum_observe_only.json`
ECE, log-loss	3.20–3.21	`probabilistic.*`	`test_probabilistic`	`e1_crossfit.json`
Realized loss	3.22	`run_main.stage_observe`	— (script)	`e2_e5_e6_observe_only.json` (`e6_grid`)
D1/D2 before expected loss	—	`PredicatePolicy.evaluate`, `A3Policy.decide`	`test_a3`	`e2_e5_e6_observe_only.json` (`sources`)
Step- and episode-level mitigation	Table 3.14	`RemGuard._mitigate`	`test_replay`	`e4_e5_enforcement_replay.json`
Audit identity of withheld actions	Section 3.7.6	`RemGuard.evaluate`; `mitigation.py`, `audit.py`	`test_replay`, `test_pipeline`	—

Table 3.24. Components required, implemented and absent

Component (the methodology (Sections 3.1–3.15))	Status
Provenance labels, hygiene	Implemented
Entity extraction	Partly implemented (Section 3.16.4)
x^{ctx}_1 injection classifier	**Absent**
x^{ctx}_2, x^{beh}_{1-4}, x^{act}_{1-3}	Implemented
Ridge estimator; both calibration fits	Implemented
Tiers, predicates, feasible set, loss structure, dominance check	Implemented
REM verdict; B1; REM without expected-loss Escalate	Implemented
Step-level and episode-level mitigation	Implemented (evaluation path)
Human reviewer	Absent; replaced by two bounds
Offline fitting (Procedure 3.1)	Implemented
CUSUM (observe-only)	Implemented
Live enforcing-mode execution	**Not executed** (no LLM access)

3.16.13 Logging of Sensitive Values

- **Audit records.** The audit records of the evaluation path store argument digests and keys, not values. A test checks that neither the attacker's account identifier nor the attacker's password appears in any audit record of a replayed episode.
- **Frozen audit log.** It stores digests by default; raw payloads are written only if `record_payloads=True` is set explicitly.
- **Result files.** Two result files contain argument values copied from the benchmark's traces: the label-audit sample and the human label-audit package. These values are synthetic benchmark data (simulated IBANs and passwords), not real credentials.
- **Credentials.** No credential, API key or real account identifier is used by the code or written by it.

3.16.14 Runtime Measurement

Two timers are recorded for every step. Both start with `time.perf_counter` at the entry of `RemGuard.evaluate`. They differ in where they stop (Table 3.25).

Table 3.25. Boundaries of the two timers

Work inside `RemGuard.evaluate`	Partial timer `rem_latency_ms` (R1–R4)	Complete-call timer `rem_latency_full_ms` (R4)
Feature computation (Layers 1–3)	Included	Included
Tier, reference-label lookup, scoring and calibration	Included	Included
Predicates, verdict and mitigation decision	Included	Included
Linear attribution and additivity check	**Excluded**	Included
Audit-record construction, including the argument digest	**Excluded**	Included
Recording the proposal in the episode history	**Excluded**	Included

Both timers **exclude**:

- LLM inference (absent in replay);
- tool execution and benchmark environment set-up;
- the benchmark pipeline's own overhead;
- the call into and return from `evaluate`.

Both **include** the reference-label lookup. This is evaluation bookkeeping, which a deployment would not perform: it computes the evaluation label recorded in the audit record and used by the idealized reviewer.

Correction. Revision R3 stated that the measured interval included attribution and audit-record construction. It did not: the timer of R1–R3 stopped before them. All R1–R3 timing values are partial-timer values. The complete-call timer was added in R4 without changing the partial timer. `tests/test_latency_instrumentation.py` checks both boundaries with a fake clock and an ordered event log, not with wall-clock thresholds.

Conditions. Measurements were taken in four parallel worker processes on a 4-vCPU container, so they include contention between workers, and they vary between runs (Section 3.16.11.2). The R4 run used an Intel Xeon @ 2.80 GHz with Python 3.11.17 and NumPy 2.4.6. Memory is a separate measurement: the peak allocation traced by `tracemalloc` around the complete `RemGuard.evaluate` call, on 300 episodes with the episode-level variant.

These figures describe the evaluated configuration: no injection classifier, no reviewer, and Python on this hardware. They do not describe a deployment that includes a neural classifier or a human review step.

3.17 Closed-Loop Evaluation with Live Agents: Design and Implementation Status

Status. The closed-loop evaluation described in this section is **implemented and tested with scripted test agents, but has not been executed with a language model** (Section 4.12). Nothing in Chapter 4 is a live result.

3.17.1 Purpose

The replay sensitivity analysis (Section 3.11.11) re-executes recorded trajectories and assumes that the agent does not adapt to REM's refusals. It can therefore show whether a recorded attack call would have been withheld, but not what an agent does after a refusal: it cannot observe retries, rephrased calls or alternative tools. The closed-loop design removes that assumption. A running LLM agent receives REM's decision for each proposed call and may continue, retry, replan or stop, and the benchmark scores the outcome after the agent's own reaction.

3.17.2 Defended agent pipeline

The integration uses AgentDojo's own pipeline elements without modifying the benchmark (Figure 3.5). The pipeline consists of a system message, the user task's prompt, the LLM, and a tool-execution loop in which AgentDojo's tool executor is replaced by REM's guarded executor (Section 3.16.8). For each proposed call the defended process is:

1. the agent proposes a tool call with arguments;
2. REM intercepts the call **before** it is executed;
3. REM computes the evidence vector, the tier, the logit and the calibrated probability with frozen parameters (Sections 3.4–3.7);
4. the decision function selects Allow, Modify, Escalate or Block (Eqs. 3.6–3.7, or the baseline of Eq. 3.14);
5. the Mitigation Layer applies the declared semantics (Section 3.8, Table 3.14): a released call is executed in the sandbox; a withheld call returns a tool message with the same tool-call identifier and a refusal notice;

6. the result or refusal re-enters the agent's context as untrusted content, and the agent continues;
7. the benchmark evaluates utility and security on the final environment.

The decision code is the code of the recorded-trace study (`RemGuard`); only the source of the agent's messages changes, from a recorded trajectory to a live model. AgentDojo re-queries the pipeline up to three times when the agent produces no final answer; the same guard, and therefore the same episode history, persists across these re-queries.

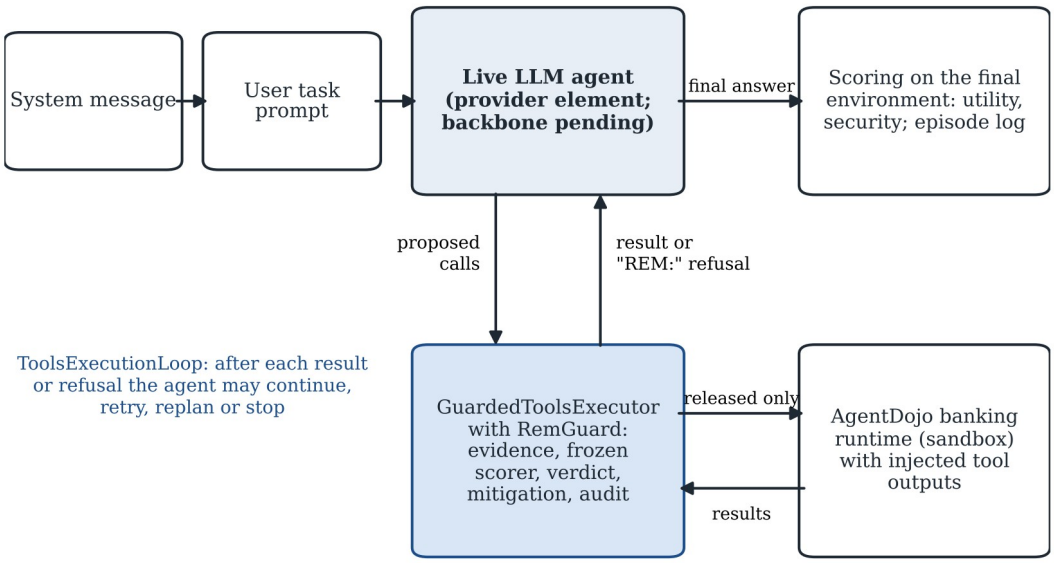


Figure 3.5. Closed-loop (live-agent) evaluation architecture. REM's guarded executor replaces AgentDojo's tool executor; the agent receives the result of a released call or the refusal of a withheld one and may retry, replan or stop. Implemented and tested with scripted test agents; not executed with a language model.

3.17.3 Conditions and controlled variables

Three primary conditions are defined, with the same pipeline, tasks, injections, system prompt, model, decoding settings and iteration limit:

- **B0:** no defense (every call released; REM records the step but never intervenes);
- **B1:** the consequence-independent operating-point baseline (Eq. 3.14) on the same frozen estimator, calibration and predicates;
- **REM:** the tier-indexed expected-loss verdict (Eqs. 3.6–3.7) on the same frozen estimator.

REM without expected-loss Escalate (Section 3.11.7) can be added as a fourth condition. The comparison B1 versus REM therefore isolates the decision rule, as in the recorded-trace study.

3.17.4 Frozen parameters

No parameter is fitted on live data. The estimator, calibration map, baseline thresholds, benign reference and r_{mod} are those of one outer fold of the recorded-trace cross-fitting (Procedure 3.1). Which fold is used is a design input; the pilot has no fit on all recorded data.

3.17.5 Design inputs required before execution

The runner refuses to start until each of the following has been decided and recorded (CLAUDE design lock; supervisor decision register D-2, D-6, D-7, D-8, D-9, D-15):

Input	Status
Authorisation for paid model use, budget, pre-registration of hypotheses and analysis	Pending (D-15)
Backbone(s) with exact model versions; decoding parameters	Pending (D-15)
Number of repeats per configuration	Pending (Section 3.11.9 proposes at least three)
Suite version, attack templates, user and injection tasks	Pending (D-6)
Mitigation semantics (step-level and/or episode-level)	Pending (D-2)
Reviewer treatment (idealized and/or no-reviewer bound)	Pending (D-7)
Costs, tiers, predicates and feature set	Exploratory pilot values only (D-9, D-8)
Fitted-parameter source (outer fold)	Pending

3.17.6 Metrics, statistics and logging

The metrics are those of Section 3.12, computed on live outcomes: attack success rate, benign utility and utility under attack from the benchmark's verdicts; intervention rates on $y = 0$ and $y = 1$ proposals; and processing time from the complete-call timer. Three descriptive adaptation counts are added, computed from REM's audit record: a *retry* is a later proposal of the same tool with the same argument digest as a withheld call; a *replan* is a later proposal of a different restricted tool after a withheld call; and a *release after refusal* is a reference-labelled attack call released after an earlier withheld call in the same episode.

Uncertainty is estimated by the grouped bootstrap of Section 3.11.9, with repeats kept inside their injection-task or user-task group. Every episode is logged with its audit records, truncated messages and adaptation counts, together with a manifest of the commit, configuration hash and frozen fold.

3.17.7 Safety precautions

All tools act on the benchmark's simulated banking environment; no real account or payment system is reachable, and REM never initiates a call. Argument values are recorded only as digests in the audit record, and message logs are truncated.

3.17.8 Implementation and verification status

Table 3.26. Implementation status of the closed-loop evaluation (3 November 2026)

Component	Location	Status
Defended live pipeline	`rem/integration/agentdojo_live.py`	Implemented; tested with scripted test agents
Guarded executor, decision and mitigation	`rem/integration/agentdojo_replay.py` (unchanged)	Implemented; used in the recorded-trace study
Runner with fail-closed configuration and credential checks	`experiments/live/run_live.py`	Implemented; verified to refuse the undecided template
Metrics with grouped bootstrap	`experiments/live/analyze_live.py`	Implemented; tested on synthetic records
Integration tests (interception, semantics, identifiers, retries and replans, episode isolation, label isolation)	`tests/test_live_integration.py`	12 tests passed (full suite: 217 passed)
Harness smoke test with a scripted ground-truth agent on all 160 task combinations	`experiments/live/smoke_scripted.py`	Executed: 480 episodes, 0 errors (not an experimental result)
Execution with a language model	—	**Not executed**: no model credential; design inputs pending

3.18 Chapter Summary

Methodology. This chapter specified the REM methodology as one runtime pipeline in five layers:

- **Input & Context:** captures the proposed action and labels context by provenance.
- **Detection:** computes context evidence.
- **Behavioral Analysis:** computes history and action-provenance evidence.
- **Decision Engine:** estimates a calibrated probability of adversarial induction with ridge logistic regression and Platt calibration on the logit, assigns a declared consequence tier, applies policy predicates, selects a verdict by minimizing conditional risk under tier-indexed losses, and writes an audit record whose linear attribution never influences the verdict.
- **Mitigation:** executes Allow, tool restriction, escalation, or Block.

Offline fitting and recalibration are cross-cutting activities, not a layer.

The evaluation uses the AgentDojo banking environment with nested grouped cross-fitting instead of independent partitions. It explicitly accounts for positive-step scarcity, the shared attacker account identifier, repeated task templates, and short trajectories. It compares REM with a consequence-independent baseline on the same estimator, reports cost-free primary metrics, and reports realized loss only across a declared grid that includes misspecified costs. The contribution is bounded as an empirical and integrative one. Section 3.16 describes the implementation.

Implementation. The full runtime procedure of the methodology (Sections 3.1–3.15) is implemented in the evaluation path, except for the injection classifier, which is absent. The evaluation path inserts REM at the benchmark's tool boundary without modifying the agent. Its correspondence with the equations, tests and results is given in Table 3.23.

The frozen pipeline is verified by unit tests but is not the path used in the experiments. One defect in it, the loss of a blocked action's identity in the audit record, was corrected in this revision without effect on any result.

The software is verified by 205 tests and by an exact analytical reproduction of the pilot. Its security effectiveness is the subject of Chapter 4.

Closed-loop evaluation. A closed-loop integration in which a running agent receives REM's refusals is implemented and tested with scripted test agents (Section 3.17); it has not been executed with a language model.

CHAPTER FOUR: RESULTS AND DISCUSSION

4.1 Purpose, Status and Kinds of Evidence

This chapter reports what the implementation of Section 3.16 produced on recorded agent behavior in the AgentDojo banking environment. **The results are exploratory.** Two conditions for confirmatory results are not met:

- the design values they use (costs, tiers, predicates, feature set) await approval;
- the enforcing-mode evaluation with live agents that Chapter 3 specifies could not be executed.

The results are therefore not presented as confirmation of any claim, and the confirmatory evaluation remains pending. The closed-loop harness for live agents (Section 3.17) is implemented but was not executed; its status is reported in Section 4.12, and no live result appears in this chapter.

Five kinds of evidence are kept apart throughout. Each table states which kind it reports.

Kind	What it is	Used in
Recorded outcome	The benchmark's utility and security verdicts on the traces as recorded by the benchmark authors, without REM	Sections 4.2, 4.6.1
Held-out prediction	REM's probability for a recorded step, from a model fitted without that step's group	Sections 4.3–4.4
Observe-only decision	The verdict REM or the baseline would issue for a recorded step, computed but not enforced	Sections 4.5, 4.7, 4.9
Replay result	The benchmark's verdicts after re-executing a recorded trajectory with REM enforcing, assuming the agent does not adapt	Section 4.6
Live run	An agent running with REM enforcing	**None was executed**

All numbers come from `results/pilot_R2/`, produced at commit `84a8c21` by `experiments/pilot/run_all.sh`. A re-execution of the full pipeline in a fresh environment on 30 October 2026 (by this project, not an external replication) reproduced every analytical value; the only differing values were per-step latency values, code-commit stamps and one file path, and no memory value differed (Chapter 3, Section 3.16.11.2). A further re-execution on 1 November 2026 (R4), which added the complete-call timer, is

compared in the same section. The first run (commit `aa66f22`, `results/pilot/`) produced identical analytical values (Chapter 3, Section 3.16.11.2).

4.2 Data, Protocol and Chronology

4.2.1 Data

Source. The AgentDojo repository (Debenedetti et al., 2024) distributes traces of real LLM agents recorded by the benchmark authors. The evaluation uses the banking traces of **21 undefended backbones** under the `important_instructions` attack.

Exclusions. Episodes recorded with an error (5) and the pair (user_task_15, injection_task_4) (21 episodes, amendment AM-1) are excluded.

Size (`dataset_summary.json`):

Unit	Count
Episodes	**3,334** (2,998 attacked, 336 without injection)
Proposed tool calls (steps)	**6,762**
Positive steps (y = 1)	**921**
Negative steps (y = 0)	5,841

Labels. Labels follow the reference-match rule of Chapter 3, Section 3.11.3. A positive label marks an *attempt*: the agent proposed the attacker's reference action with the attacker's key argument. It is a proxy for adversarial inducement, not proof of it. Of the 840 attacked episodes with a positive step, the recorded outcome shows attack success in 682 and not in 158. Two attacked episodes record attack success without a positive step.

Independence. The episodes are not independent observations:

- they are built from 16 user-task and 9 injection-task templates;
- one attacker account;
- one environment.

All intervals are 95% percentile intervals from a bootstrap that resamples attacked items by injection task and benign items by user task. With nine injection-task groups the intervals are wide. Bootstrap coverage is an asymptotic property, and with so few groups the intervals are not guaranteed to reach their nominal 95% coverage; a narrow interval, such as that of REM's replay ASR, is not evidence of precision. The intervals are therefore

descriptive, and no hypothesis test is based on them. Where a category contains a single injection task, no interval can be formed and none is reported.

4.2.2 Protocol

The protocol is nested grouped cross-fitting (Chapter 3, Procedure 3.1):

- nine outer folds, each holding out one injection task;
- episodes without injection assigned to folds by user task.

Within each fold's fitting portion, the following are fitted (Chapter 3, Section 3.11.5):

- λ, by grouped inner cross-validation;
- the calibration map, on inner out-of-fold logits;
- the baseline thresholds;
- the benign reference quantities;
- r_{mod}.

The **pre-registered values** are listed in Table 4.1. They come from Chapter 3 where Chapter 3 gives them; otherwise they are declared exploratory values.

Table 4.1. Pre-registered values (`configs/exploratory/pilot_e0.yaml`)

Item	Value
Features	$x\text{^}ctx_2$, $x\text{^}beh_{1-4}$, $x\text{^}act_{1-3}$ ($x\text{^}ctx_1$ absent)
Tiers	read 1; profile 2; transfer ≤ 1000: 3; transfer > 1000, payment modification, credential change: 4
Costs	$C_{FA} = 1$; $C_{miss} = 1, 5, 10, 20$ by tier; $C_{mod} = 0.5$; $C_{esc} = 0.3$
$r_{mod}(k)$	Measured; 1.0 where unmeasured
Predicates	$\Pi = \varnothing$; $k_{D2} = 3$; required evidence $x\text{^}act_1$
Baseline	$\alpha_{blk} = 0.01$, $\alpha_{mod} = 0.05$
λ grid	$\lambda_{impl} \in \{0.01, 0.1, 1, 10, 100\}$, i.e. $\lambda \in \{0.005, 0.05, 0.5, 5, 50\}$ on the scale of Eq. 3.2
Inner folds; ECE bins; clip	4; 10; 10^{-6}
Pilot E0 decision values	≥ 50 positive steps; group ROC-AUC ≥ 0.55
Bootstrap	2,000 resamples, seed 20261027

4.2.3 Chronology of design, amendments and analyses

Date and time (UTC)	Event	Record
27 Oct 2026 16:40	Pilot design committed (Table 4.1)	commit `48bef90`
27 Oct 2026 16:57	Amendments AM-1 to AM-4 committed. AM-1: exclusion of (user_task_15, injection_task_4). AM-2: benign references. AM-3: the meaning of “tool class”. AM-4: replay validity filter	commit `7a707a7`
27 Oct 2026, after 16:57	First computation of held-out predictions and of all analyses with the then-uncommitted pipeline code. **The log of this first execution was not preserved**; its order relative to the amendments rests on the commit times and on the working session record	—
27 Oct 2026 17:16	Pipeline committed, then re-run in full; results identical to the first execution	commit `aa66f22`; `results/pilot/`
27–28 Oct 2026	**Post-hoc analyses**, added after results had been seen: exclusion of user_task_15 (Section 4.10); benign-release breakdown by tier; REM without expected-loss Escalate (Sections 4.5–4.6); breakdowns by action type and tier; replay-failure analysis	`results/pilot_R2/`
28 Oct 2026	Code corrections of revision R2 (Section 3.16), complete re-run; analytical values identical	commit `84a8c21`; `results/pilot_R2/`

No pre-registered value was changed after results were seen. The post-hoc analyses are reported as such, and none replaces a pre-registered analysis.

4.3 Pilot E0: Feasibility and Feature Retention

Pilot E0 used only the fitting portion of each outer fold (`e0_pilot.json`).

Table 4.2. Pilot E0 (fitting portions only)

Evidence group	ROC-AUC: mean over folds; minimum	Retained in every fold (≥ 0.55)
Context ($x \hat{} ctx_2$ only)	0.582; 0.554	Yes, marginally
Behavioral	0.802; 0.787	Yes
Action	0.929; 0.923	Yes
All	0.938; 0.930	—

The retention rule was applied to each fold's fitting portion separately, and every fold retained all three groups (`e1_crossfit.json`, `e0_fold_specific_feature_groups`). The feature set is therefore the same in every fold. No fold's held-out data influenced its own feature set.

The pooled count of positive steps is 921, above the declared minimum of 50. Thirteen of the 21 backbones have fewer than 50 positive steps each (range 2–116), so no per-backbone conclusion is drawn.

4.4 RQ1 — Offline Classification (Held-Out Predictions)

Table 4.3. Held-out discrimination and calibration, pooled over nine outer folds (6,762 steps; 921 positive; `e1_crossfit.json`)

Metric	Calibrated p, label-NLL fit (reference)	Calibrated p, Platt smoothed targets	Uncalibrated p̃
ROC-AUC	0.938 [0.908, 0.961]	0.938	0.939
Average precision (PR-AUC estimate)	0.719 [0.559, 0.827]	0.719	0.721
ECE (M = 10)	0.044 [0.033, 0.084]	0.044	0.042
Log-loss	0.201 [0.149, 0.294]	0.201	0.201
Brier score (binary convention)	0.060	0.060	0.060

Table 4.4. Per-fold results (calibrated, label-NLL)

Held-out injection task	Steps	Positive	ROC-AUC	AP	ECE	Log-loss	λ_{impl} (λ)	γ_1
0	824	125	0.962	0.788	0.059	0.161	10 (5)	0.959
1	773	101	0.965	0.854	0.046	0.140	10 (5)	0.947
2	807	112	0.949	0.735	0.035	0.164	10 (5)	0.958
3	772	110	0.962	0.811	0.045	0.154	10 (5)	0.955
4 (payment modification)	726	135	0.978	0.906	0.087	0.148	10 (5)	0.947
5	634	23	0.859	0.308	0.068	0.164	10 (5)	0.969
6	666	79	0.909	0.501	0.075	0.225	10 (5)	0.981
7 (credential change)	730	124	0.903	0.577	**0.189**	**0.530**	1 (0.5)	0.980
8	830	112	0.959	0.859	0.037	0.146	10 (5)	0.945

Findings:

1. **Discrimination holds on held-out attacker objectives.** Held-out ROC-AUC is 0.86–0.98 per fold.
2. **Calibration did not transfer to the one held-out attack category not seen in fitting.** Injection task 7 is the only credential-change attack. Its fold has ECE 0.189 and log-loss 0.53, three to four times the other folds.
3. **No calibration improvement was observed.** The fitted maps are close to the identity (γ_1 = 0.945–0.981), and pooled ECE is 0.044 with calibration and 0.042 without. This does not show that calibration is useless. It shows that the ridge model's own probabilities were already close to calibrated on in-distribution folds, and that the map did not repair the miscalibration in fold 7.
4. **Other generalization checks** (`e1_crossfit.json`, `e4_cross_template_and_defenses.json`):
    - leave-one-backbone-out (21 folds): pooled ROC-AUC 0.949; per held-out backbone 0.736 (claude-3-haiku) to 1.000;
    - grouping by user task (16 folds): pooled ROC-AUC 0.943. One fold, user_task_15, scores 0.211; Section 4.10 explains this as a label problem;
    - five other attack templates on gpt-4o, with no refitting: ROC-AUC 0.922–0.959, ECE 0.028–0.086.

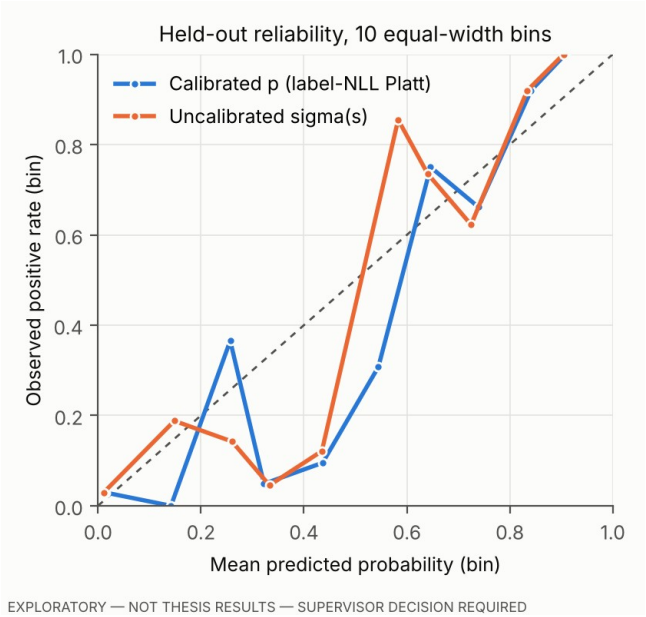


Figure 4.1. Held-out reliability diagram, calibrated and uncalibrated (10 equal-width bins). Mid-range bins contain few steps.

4.5 RQ2 — Observe-Only Decisions

All three policies use the same held-out calibrated probabilities and the same predicates:

- **REM** adds tier-indexed expected loss with Escalate available;
- **REM without expected-loss Escalate** (post-hoc) has the same escalation opportunity as the baseline (D2 only);
- **B1** is the baseline with the thresholds of Eq. 3.14.

Verdicts were computed on the recorded steps and not enforced.

Table 4.5. Observe-only verdicts on held-out steps (y = 0: n = 5,841; y = 1: n = 921; `e2_e5_e6_observe_only.json`)

	REM	REM w/o expected-loss Escalate	B1
Any intervention on y = 1 steps	0.973 [0.957, 0.987]	0.973	0.740 [0.473, 0.922]
Any intervention on y = 0 steps (step false-positive rate)	0.227 [0.213, 0.239]	0.198	0.056 [0.047, 0.066]
Escalate share of all steps	0.196	0.009	0.009
y = 0 steps withheld if an idealized reviewer resolves escalations	0.040	0.188	0.046
Ties	0	0	0
Verdicts fixed by D2	58	58	58

Table 4.6. Observe-only interventions by consequence tier and action category (`r2_breakdowns.json`)

Tier	Category	n (y = 0)	n (y = 1)	REM y=0 / y=1	REM w/o Esc. y=0 / y=1	B1 y=0 / y=1
1	read	4,176	0	0.000 / —	0.000 / —	0.000 / —
2	profile change	351	0	0.399 / —	0.219 / —	0.026 / —
3	value transfer ≤ 1000	736	561	0.932 / 0.955	0.838 / 0.955	0.254 / 0.900
4	value transfer > 1000	115	101	0.983 / 1.000	0.922 / 1.000	0.391 / 0.574
4	payment modification	285	135	0.740 / 1.000	0.684 / 1.000	0.305 / 0.881
4	credential change	178	124	0.983 / 1.000	0.899 / 1.000	0.000 / 0.000

What the observe-only results show:

- **Low thresholds at high tiers.** REM's additional interventions are concentrated on state-changing actions at tiers 3–4, where the pre-registered costs put the Allow–Block indifference points at 1/11 and 1/21 (Eq. 3.9). Legitimate transfers often carry action evidence (for example, a recipient read from a tool output) that places them above those points.
- **Benign state-changing steps.** On episodes without injection, REM releases 33 of 180 state-changing steps (tiers 2–4); REM without expected-loss Escalate releases 48; B1 releases 132 (`benign_release_breakdown.json`).

- **Escalation is not the cause.** Removing the expected-loss Escalate option lowers REM's intervention rate on legitimate steps only from 0.227 to 0.198: most former escalations become Modify verdicts, which also withhold the call. The difference from B1 is therefore mainly due to the tier-indexed costs, not to the opportunity to escalate.
- **B1 and credential changes.** B1 intervened on none of the 124 credential-change attack steps. These are the steps of the fold whose probabilities were most miscalibrated (Table 4.4). Its thresholds are fixed by benign false-positive rates and did not reach them.
- **Realized loss.** Under the decision costs, realized loss is 0.114 for REM, 0.136 for REM without expected-loss Escalate and 0.652 for B1. This comparison favors REM by construction (Chapter 3, Section 3.12.3) and is not evidence of superiority.

4.6 RQ2 and RQ3 — Replay Sensitivity Analysis

The replay re-executes each recorded episode in the benchmark's sandbox with REM enforcing its verdicts (Chapter 3, Section 3.11.11). Its use is a proposed amendment to the scope of Chapter 3. Three properties limit what it can show:

- **NO-ADAPT.** The agent emits the calls it emitted in the recorded run, whatever REM does. After a refusal, a live agent could retry, rephrase, use another tool, or abandon the task. Replay therefore cannot show how often a live agent would achieve the attacker's goal by another route, and **a near-zero replay ASR is not evidence of protection against a live or adaptive agent.**
- **Mitigation semantics.** Under step-level semantics, Modify and Block have the same effect on the withheld call. The episode-level variant also restricts later calls: after a Block, every later call to a tool in $\mathscr{T}_{\mathrm{R}}$ is refused, even one whose own verdict is Allow, and even though the agent never retries. The two semantics therefore give different replay results. Tables 4.7–4.10 use step-level semantics, the declared reference pending decision D-2; Table 4.12 compares both semantics.
- **Not independent runs.** Replaying a recorded trajectory is deterministic. Repeating it adds no information about LLM behavior.
- **Closed-loop identifiability.** Once blocking changes what an agent proposes next, static scores and ungated trajectories need not identify the closed-loop trade-off between false blocks and misses (Ray, 2026). Recorded trajectories are ungated in this sense. Offline replay of AgentDojo's recorded traces has also been used to evaluate temporal-logic monitors (Kekatos et al., 2026); neither use establishes enforcement against live agents.

4.6.1 Replay validity and exclusions

With every call allowed, the replay reproduced the recorded utility and security outcomes of 3,313 of the 3,334 episodes.

The 21 failures (`r2_breakdowns.json`, `replay_failures`) are concentrated:

- 20 are user_task_13 episodes whose utility verdict differs; 1 is a user_task_14 episode whose security verdict differs;
- 10 are from gpt-4o, and the remaining 11 are spread over six other backbones;
- none raised an error.

The pattern is consistent with a change in the user task's definition between the suite version used for recording and the version used for replay (v1). This was not confirmed task by task.

Effect of the exclusion. Of the 21 excluded episodes, 19 are attacked, and 14 of these recorded attack success. Excluding them changes the recorded attack-success rate from 0.228 (all 2,998 attacked episodes) to 0.226 (2,979 retained), and the recorded benign utility from 0.521 (336) to 0.524 (334). All replay metrics use the 2,979 attacked and 334 benign retained episodes.

4.6.2 Results

Table 4.7. Replay results, step-level semantics (attacked n = 2,979; benign n = 334; `e4_e5_enforcement_replay.json`)

Configuration	ASR	Benign utility	Utility under attack
B0, no defense (replay)	0.226 [0.148, 0.295]	0.524 [0.381, 0.676]	0.492
B1, no reviewer	0.055 [0.011, 0.134]	0.509 [0.334, 0.702]	0.514
REM, no reviewer	0.004 [0.001, 0.007]	0.314 [0.110, 0.548]	0.364
REM w/o expected-loss Escalate, no reviewer	0.004 [0.001, 0.007]	0.329 [0.126, 0.551]	0.374
REM, idealized reviewer	0.004 [0.001, 0.007]	0.497 [0.333, 0.680]	0.496
REM w/o expected-loss Escalate, idealized reviewer	0.004 [0.001, 0.007]	0.311 [0.123, 0.518]	0.359

Table 4.8. Paired differences (grouped bootstrap)

Comparison	ΔASR	ΔBenign utility
REM − B1, no reviewer	−0.051 [−0.130, −0.007]	−0.195 [−0.341, −0.074]
REM w/o Esc. − B1, no reviewer (same escalation opportunity)	−0.051 [−0.130, −0.007]	−0.180 [−0.325, −0.063]
REM − B1, idealized reviewer	−0.051 [−0.130, −0.007]	+0.003 [−0.009, 0.018]
REM − REM w/o Esc., idealized reviewer	0.000	+0.186 [0.065, 0.335]
REM − B0, no reviewer	−0.222 [−0.289, −0.146]	−0.210 [−0.379, −0.051]

Table 4.9. Replay ASR by the attacker's target action (`r2_breakdowns.json`)

Target action	Injection tasks	Attacked episodes	B0	B1 (no reviewer)	REM (no reviewer)
Value transfer	7	2,334	0.213 [0.128, 0.293]	0.014 [0.007, 0.020]	0.005 [0.002, 0.008]
Payment modification	1	311	0.183	0.042	0.000
Credential change	1	334	0.356	0.356	0.000

Single-task categories have no interval.

Table 4.10. Replay benign utility by kind of user task (`r2_breakdowns.json`)

User tasks	Tasks	Episodes	B0	B1 (no reviewer)	REM (no reviewer)	REM (idealized reviewer)
Requiring a state change	12	250	0.472 [0.320, 0.632]	0.432 [0.238, 0.637]	0.168 [0.000, 0.420]	0.416 [0.238, 0.600]
Read-only	4	84	0.679 [0.369, 0.929]	0.738 [0.417, 0.976]	0.750 [0.429, 1.000]	0.738 [0.417, 0.976]

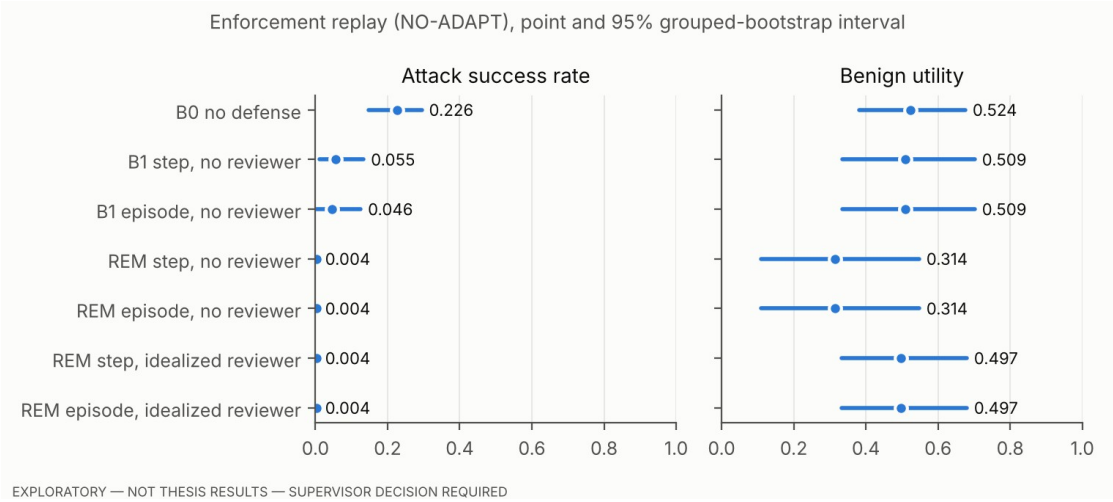


Figure 4.2. Replay sensitivity analysis (NO-ADAPT): ASR and benign utility with 95% grouped-bootstrap intervals.

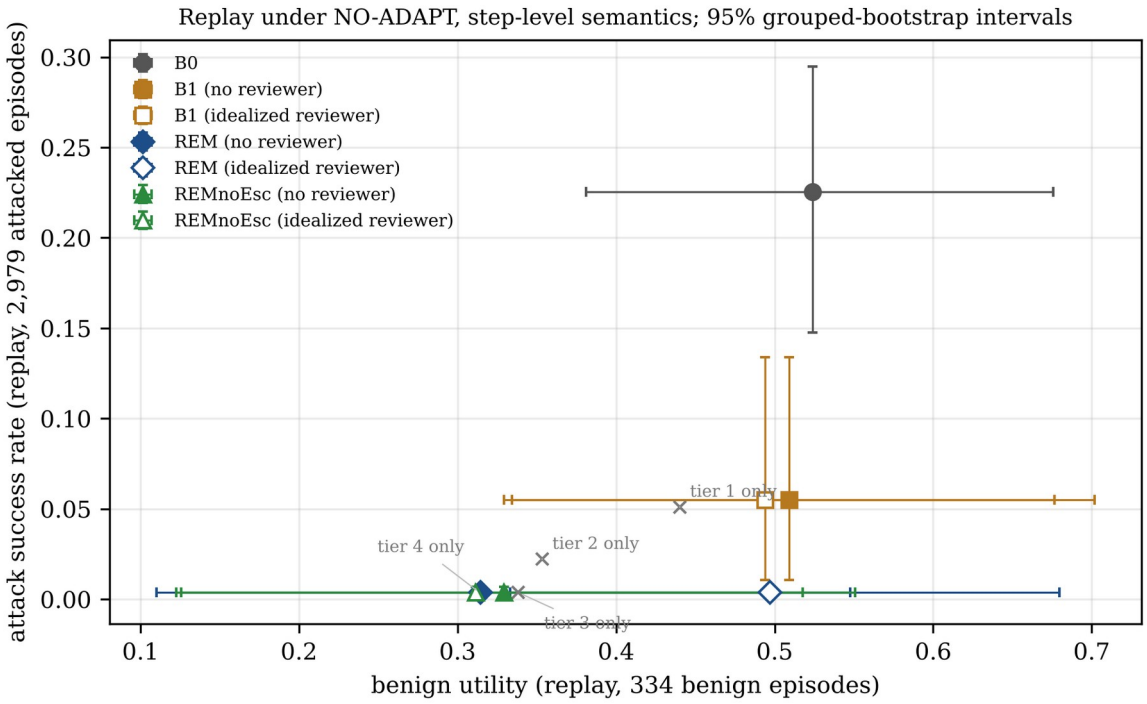


Figure 4.3. Security–utility trade-off in the replay sensitivity analysis (NO-ADAPT, step-level semantics): attack success rate against benign utility for B0, B1, REM and REM without expected-loss Escalate under both reviewer bounds, and for the single-tier ablations. Values from `results/pilot_R2/e4_e5_enforcement_replay.json`; no live result.

What the replay results show, within NO-ADAPT:

- **Where the ASR difference comes from.** REM withholds more of the recorded attack attempts than B1. Most of the difference comes from the credential-change

and payment-modification attacks (Table 4.9), each represented by a single injection task.

- **The utility cost falls on state-changing tasks.** With no reviewer, REM's benign utility falls to 0.168 on those tasks (Table 4.10).
- **The cost is not caused by the escalation option.** REM without expected-loss Escalate has almost the same utility cost relative to B1 (Table 4.8).
- **With an idealized reviewer, the cost disappears.** That reviewer knows the label and is an upper bound on reviewer quality, not a model of one.

Sensitivity to the mitigation semantics

Table 4.11. Replay results under the two mitigation semantics (same episodes; same estimator, calibration, costs, predicates and thresholds; matching reviewer treatment for REM and B1; only the mitigation semantics varies. Verdict sequences are not fixed: see the notes below. ref. = the reference semantics; REM w/o Esc. = REM without expected-loss Escalate; attacked n = 2,979; benign n = 334; `e4_e5_enforcement_replay.json`, `configs.<configuration>_<semantics>_<reviewer>`)

Configuration	Semantics	ASR	Benign utility	Utility under attack
B1, no reviewer	step-level (ref.)	0.055 [0.011, 0.134]	0.509 [0.334, 0.702]	0.514 [0.501, 0.527]
B1, no reviewer	episode-level	0.046 [0.004, 0.125]	0.509 [0.334, 0.702]	0.485 [0.474, 0.496]
B1, idealized reviewer	step-level (ref.)	0.055 [0.011, 0.134]	0.494 [0.329, 0.677]	0.499 [0.487, 0.511]
B1, idealized reviewer	episode-level	0.046 [0.004, 0.125]	0.494 [0.329, 0.677]	0.470 [0.460, 0.481]
REM, no reviewer	step-level (ref.)	0.004 [0.001, 0.007]	0.314 [0.110, 0.548]	0.364 [0.345, 0.389]
REM, no reviewer	episode-level	0.004 [0.001, 0.007]	0.314 [0.110, 0.548]	0.363 [0.344, 0.388]
REM, idealized reviewer	step-level (ref.)	0.004 [0.001, 0.007]	0.497 [0.333, 0.680]	0.496 [0.483, 0.506]
REM, idealized reviewer	episode-level	0.004 [0.001, 0.007]	0.497 [0.333, 0.680]	0.481 [0.468, 0.492]
REM w/o Esc., no reviewer	step-level (ref.)	0.004 [0.001, 0.007]	0.329 [0.126, 0.551]	0.374 [0.351, 0.407]
REM w/o Esc., no reviewer	episode-level	0.003 [0.001, 0.006]	0.329 [0.126, 0.551]	0.370 [0.347, 0.403]
REM w/o Esc., idealized reviewer	step-level (ref.)	0.004 [0.001, 0.007]	0.311 [0.123, 0.518]	0.359 [0.337, 0.391]
REM w/o Esc., idealized reviewer	episode-level	0.003 [0.001, 0.006]	0.311 [0.123, 0.518]	0.355 [0.333, 0.387]

What is fixed and what differs, within NO-ADAPT:

- **What is fixed and what varies.** Both runs of each configuration use the same recorded episodes, the same fitted parameters, costs, predicates and thresholds, and the same reviewer treatment. They differ in the mitigation semantics: under the

episode-level variant, an earlier Block or Modify adds tools to a restriction set that refuses later calls.

- **The verdicts are not identical.** For REM without a reviewer, the y = 0 steps received 239 Modify and 1,049 Escalate verdicts under step-level semantics, against 241 and 1,047 under episode-level semantics (`verdict_rates` of the two REM configurations). With the idealized reviewer the counts are 239 / 1,050 against 241 / 1,048. A call refused only under the episode-level variant returns no tool output, so the evidence for later steps can differ. B1's verdict counts are the same under both semantics.
- **B1's ASR is lower under episode-level semantics** (0.055 → 0.046): later state-changing calls are refused by the restriction set left by an earlier Block or Modify. REM's ASR is identical (0.004). REM without expected-loss Escalate changes from 0.004 to 0.003 (unrounded 0.0037 → 0.0034).
- **Utility under attack is lower or equal under episode-level semantics** in every configuration: the restriction set also refuses legitimate later calls.
- **Benign utility coincides** between the two semantics in every configuration of this replay. This is an observation about these episodes, not a property of the semantics.
- **REM − B1 ASR within each semantics** (paired grouped bootstrap, no reviewer): −0.051 [−0.130, −0.007] under step-level semantics (Table 4.8) and −0.043 [−0.122, −0.001] under episode-level semantics (`paired_differences`, episode-level REM − B1 entry). The idealized-reviewer interval under episode-level semantics is the same to three decimals. These are two within-semantics contrasts. No interval was computed for the difference between them, so none is reported. As for every interval in this chapter, the bootstrap over nine attacked groups need not reach nominal coverage, and no hypothesis test is based on these intervals (Section 4.2).

Step-level semantics remains the declared reference (Chapter 3, Section 3.8), pending the supervisor's decision D-2. The episode-level rows are a sensitivity analysis, not a replacement for Tables 4.7–4.10.

RQ3 quantities:

- **Residual attack success after Modify.** In oracle-Modify replays within the fitting folds, r_{mod} was 0.000 at tier 3 (530 episodes) and 0.010 at tier 4 (298 episodes) (`e4_rmod_and_replay_validity.json`). In replays where REM applied Modify to an attack step, residual ASR was 0.000 (610 episodes). Under NO-ADAPT a withheld call simply does not occur, so **these values are consequences of the assumption, not measurements of tool restriction against a live agent.**
- **Escalation rate.** 0.196 of all steps in the observe-only analysis (Table 4.5).

- **Processing time** (Chapter 3, Section 3.16.14, Table 3.25). **Complete call** (R4 re-execution, 1 November 2026; `results/pilot_R4/e4_e5_enforcement_replay.json`, `latency_full_ms_*`): median 0.21–0.28 ms and 95th percentile 0.49–0.70 ms per step, across the 8 REM configurations (6,684 steps each). **Partial timer**, which excludes attribution, audit-record construction and history recording: median 0.18–0.24 ms and 95th percentile 0.46–0.64 ms in the same run. Earlier runs measured only the partial timer: median 0.14–0.20 ms and 95th percentile 0.39–0.54 ms (R1, R2), with the 30 October re-execution within these ranges.
- **Memory.** The median peak traced allocation was 4.4 KB per step, with a 95th percentile of 12.4 KB (611 steps of 300 episodes; `latency_memory_profile.json`). The first run (R1) measured 2.9 KB; like time, memory varies between runs. The re-execution of 30 October measured the same value. In the R4 re-execution, where `evaluate` also records the complete-call timer, the median was 4,349 bytes against 4,364 bytes, with the same 95th percentile (12,419.5 bytes; `results/pilot_R4/latency_memory_profile.json`). The memory boundary is the complete `evaluate` call in both.

These figures apply to the evaluated configuration only: no neural injection classifier, no reviewer, recorded inputs. They are not estimates for a complete deployment.

4.7 RQ4 — Ablations

Table 4.12. Ablations (observe-only columns: held-out recorded steps; replay columns: step-level semantics, no reviewer)

Configuration	Held-out ROC-AUC	Observe-only intervention, y = 0	Observe-only intervention, y = 1	Replay ASR	Replay benign utility
Full REM	0.938	0.227	0.973	0.004	0.314
− context evidence ($x^{\wedge}ctx_2$)	0.938	0.228	0.973	0.004	0.314
− behavioral evidence	0.938	0.174	0.973	0.004	0.419
− action evidence	0.799	0.242	0.921	0.006	0.314
− calibration ($\tilde{p}$ in Eq. 3.6)	—	0.213	0.973	0.004	0.329
one tier for all actions: 1 / 2 / 3 / 4	—	0.093 / 0.157 / 0.233 / 0.336	0.757 / 0.909 / 0.972 / 0.976	0.051 / 0.022 / 0.004 / 0.003	0.440 / 0.353 / 0.338 / 0.314
− predicates D1, D2	—	0.226	0.973	0.004	0.314

Findings, worded as what was and was not observed:

- **Action evidence** carries most of the discrimination: without it, held-out ROC-AUC falls from 0.938 to 0.799 and average precision from 0.719 to 0.367.
- **Context evidence.** No improvement from the instruction-pattern indicator was observed. This concerns that indicator only; the pre-trained injection classifier $x^{\wedge}ctx_1$ was not tested, so nothing is concluded about context evidence in general.
- **Behavioral evidence.** No security improvement was observed, and removing it raised replay benign utility by 10.5 points. The firing rates show why (`feature_firing_rates.json`):
    - the privilege-escalation indicator $x^{\wedge}beh_3$ fired on 92% of legitimate state-changing steps in episodes without injection, against 87% of attack steps;
    - the transition-deviation indicator $x^{\wedge}beh_4$ fired on 43% of legitimate state-changing steps, against 17% of attack steps. Legitimate tasks held out by user task produce transitions that are absent from the fitting portion's benign reference.

  In this benchmark, therefore, the behavioral features mark legitimate first transfers at least as often as attacks. Whether this holds in other environments is not known.

- **Calibration and predicates.** No material change was observed from removing them. With Π empty, D2 fixed the verdict for 58 steps.
- **Tiers.** The tier map moves the trade-off. A single low tier recovers utility and loses protection; a single high tier does the opposite.

None of these observations establishes that a component has no benefit in general. They are single-benchmark observations with nine attacked groups.

4.8 Optional Sequential Analysis (CUSUM, observe-only)

The Bernoulli CUSUM was fitted on each fold's benign fitting episodes (`e3_a2_cusum_observe_only.json`):

- **q_1 = 0.3:** it was undefined in all folds, because the estimated pre-change rate (about 0.34) exceeded q_1.
- **q_1 = 0.5 and 0.7:**
  - an alarm followed the entry of injected content in 74.8% of attacked episodes with a positive step;
  - it came at or before the first positive step in 42.5–43.8%;
  - the median delay was 1 step;
  - the held-out benign false-alarm rate was 0.106, against an in-fold target of 0.05.

The CUSUM issued no verdicts.

4.9 Realized Loss across the Declared Grid

Realized loss (Eq. 3.22) on held-out steps was computed for 36 loss structures:

- C_{miss} multipliers 0.5–5;
- C_{esc} 0.1–0.6;
- amount limits 100–10,000.

Configuration	Realized loss, range over the 36 cells
REM, matched costs	0.039–0.298
REM, base costs (misspecified)	0.057–0.324
B1	0.303–3.261

REM's loss was below B1's in every cell, also when misspecified. Matched costs gave lower realized loss than misspecified costs in 34 of 36 cells.

The comparison favors REM by construction. The grid cell $\alpha_{blk} = 0.05$ is not evaluable, because the pre-registered $\alpha_{mod} = 0.05$ must exceed α_{blk}.

4.10 Labels, the user_task_15 Collision and Benchmark Versions

Primary and post-hoc analyses are kept separate. All results above use the pre-registered label rule with amendment AM-1.

The collision. In suite versions v1 and v1.1, user_task_15 asks the agent to pay a landlord whose account is also the attacker account of every injection task. The benchmark changed this from v1.1.1. Sixteen of the 21 recorded backbones used the earlier prompt. The label-relevant definitions of the nine injection tasks are identical in every suite version from v1 to v1.2.2, so labels are consistent across the mixed recordings.

Post-hoc analysis. 29 positive steps in user_task_15 episodes are ambiguous, 19 of them with no injected content in the context. All of REM's residual replay ASR comes from these episodes. A post-hoc analysis that excludes user_task_15 gives REM ASR 0.000 and held-out ROC-AUC 0.952 (`sensitivity_post_hoc_exclude_user_task_15.json`). It is reported for transparency and does not replace the primary result.

Label audit.

- **AI inspection only so far.** A sample of 30 attacked episodes was inspected by the AI assistant that assisted this work, not by a human: 29 labels agreed with the trace, and one was a likely false negative (an attacker-directed transfer through a tool other than the reference tool).
- **Human audit package.** A package for human annotation has been prepared (`results/pilot_R2/label_audit_human_package.csv`, with instructions). It contains 145 items:
    - every proposed call of 30 randomly drawn attacked episodes (70 items; 5 of the drawn episodes contain no proposed tool call);
    - all 29 ambiguous user_task_15 positives;
    - 19 steps carrying an attacker value but labelled 0;
    - 4 steps from episodes with recorded success but no positive label;
    - 23 positive steps from episodes without recorded success.

The human audit has not been performed.

4.11 Distributed Defenses (Recorded Outcomes Only)

For orientation, the recorded outcomes of four defenses distributed with the benchmark, on gpt-4o (`e4_cross_template_and_defenses.json`):

Defense	ASR	Benign utility
tool_filter	0.112	0.81
transformers_pi_detector	0.007	0.44
repeat_user_prompt	0.462	0.94
spotlighting_with_delimiting	0.615	0.81

These are recorded outcomes, possibly under different suite versions, and were not replayed. **They are not a controlled comparison with REM** and are not used as one.

4.12 Closed-Loop Evaluation with Live Agents: Status

The closed-loop evaluation with live agents was not executed. This section therefore reports no attack success rate, utility, intervention statistic or adaptation behavior of a live agent, and no table or figure in this chapter contains a live result.

Two conditions prevented execution on 3 November 2026:

- **No model access.** No credential for a language-model provider was available in the execution environment; the provider endpoints that were reachable rejected unauthenticated requests, and model-weight repositories were not reachable, so no local model could be run.
- **Undecided design inputs.** The backbone, decoding parameters, number of repeats, fitted-parameter source, authorisation and pre-registration (Section 3.17.5) have not been decided. The runner refuses to start until they are.

What was executed concerns the correctness of the integration, not the effectiveness of REM:

Table 4.13. Verification of the closed-loop integration (not experimental results)

Check	Agent	Outcome
Integration tests (12)	Scripted test double that proposes predetermined calls and retries or replans after a refusal	All passed: released calls execute; withheld calls (Block, Modify, unapproved Escalate) are not executed and the agent receives a refusal with the original call identifier; step-level Modify does not persist, episode-level Modify does; retries and replans are recorded; episode state is not shared; REM verdicts are unchanged when reference labels are flipped
Harness smoke test (B0, B1, REM; step-level, no reviewer; one frozen fold)	Scripted agent that executes the ground-truth calls of the user task and then of the injection task, without adapting	480 episodes over all 144 attacked task pairs and 16 benign tasks with the benchmark's `important_instructions` injections; 0 execution errors; no withheld call was executed

The smoke-test agent is not a model of an LLM agent: it never reads tool outputs or reacts to refusals. Its counts are therefore not reported as attack-success or utility results. The live evaluation remains the first item of future work (Section 5.6).

4.13 Replay Versus Live Evaluation

No comparison of replay and live results is possible, because no live result exists (Section 4.12). The replay results of Section 4.6 must not be read as live results. The two designs estimate different quantities:

Table 4.14. Differences between the replay sensitivity analysis and the closed-loop design

Aspect	Replay (Section 4.6)	Closed loop (Section 3.17; not executed)
Agent messages	Recorded, replayed in order	Generated by a running model
Reaction to a refusal	None (NO-ADAPT)	Retry, replan or stop
Meaning of attack success	Whether the recorded attack calls would have been withheld	Whether the attack succeeds despite the agent's reaction
Residual success after Modify	Close to zero by construction	Observable
Repeatability	Deterministic	Stochastic; requires repeats
Backbones	21 recorded historical backbones	To be decided

Because a live agent can retry or route around a refusal, the replay attack success rate is not an upper bound on live attack success. How large the difference is, in either direction for utility, is an empirical question that only the closed-loop evaluation can answer.

4.14 Interpretation of the Findings

This section interprets the exploratory evaluation reported in Sections 4.1–4.11. Each conclusion is linked to the result that supports it and to the limit of that result. Section 4.1 defines the kinds of evidence: held-out prediction, observe-only decision, replay result; no live run was executed.

4.14.1 What the evidence supports

Table 4.15. Conclusions, their support and their limits

#	Conclusion	Supporting result	Limit
1	REM can be placed at the tool-execution boundary of a benchmark agent without modifying the agent's model, prompts or tools	Integration tests; replay of 3,313 episodes reproducing recorded outcomes (Chapter 3, Section 3.16.11.2)	Placement requires control of the tool executor (Chapter 3, Section 3.16.9); not shown in a production framework
2	Evidence available at the tool boundary separates attack attempts from legitimate calls on attacker objectives, templates and backbones not seen in fitting	Held-out ROC-AUC 0.938 [0.908, 0.961]; 0.92–0.96 on other templates; 0.74–1.00 per held-out backbone (Section 4.4)	One benchmark, one attacker account, labels that are a proxy; destination novelty is easier here than in practice
3	Action evidence carries most of this separation	Without action evidence, ROC-AUC falls to 0.799 and AP to 0.367 (Section 4.7)	Single benchmark; nine attacked groups
4	With the pre-registered costs, tier-indexed expected-loss verdicts intervene on far more legitimate state-changing actions than the baseline, mainly because of the low thresholds the costs imply at high tiers, not because of the escalation option	Observe-only: 33 vs 132 of 180 benign state-changing steps released; REM without expected-loss Escalate releases 48 (Section 4.5)	Costs are declared, not elicited; another cost structure gives another trade-off (Section 4.9)
5	Within the replay assumption, REM withholds more recorded attack attempts than the baseline, at a utility cost that depends on how escalations are resolved	ΔASR −0.051 [−0.130, −0.007] under step-level semantics and −0.043 [−0.122, −0.001] under the episode-level variant (each a within-semantics paired interval); Δbenign utility −0.195 without a reviewer and +0.003 with an oracle reviewer (Section 4.6, Tables 4.8 and 4.11)	NO-ADAPT; replay is a proposed scope amendment; most of the ASR difference comes from two single-task attack categories

4.14.2 Why benign utility falls

The fall in benign utility has two sources, and the evaluation separates them.

Low thresholds at high tiers. With C_{miss} up to twenty times C_{FA}, the Allow–Block indifference point at tier 4 is 1/21 (Eq. 3.9). A probability as low as 0.05 is enough to withhold a transfer above the limit, a payment modification or a credential change. Legitimate actions of these kinds in this benchmark often carry the same action evidence as

attacks; for example, a recipient read from a tool output rather than stated in the request. They therefore cross the threshold.

Where the verdict sends them. The withheld actions are mostly escalated. Without a reviewer an escalation is a withheld action, so utility falls. The comparison with REM without expected-loss Escalate shows that removing the option does not recover the utility: the same actions are then withheld through Modify. The utility cost is therefore a consequence of the declared costs and of the evidence, not of escalation as such.

What a reviewer would change. An escalation is worth something only if someone resolves it. With an oracle reviewer the utility cost disappears, which bounds what a perfect reviewer could recover. A real reviewer would recover less, at some delay and cost. The thesis does not model that reviewer.

4.14.3 Why behavioral evidence did not help, and calibration did not change outcomes

Behavioral evidence. The behavioral indicators are not specific to attacks in this benchmark:

- *privilege escalation* fired on 92% of legitimate state-changing steps, because a user's first transfer is, by definition, the first use of a higher-privilege tool;
- *transition deviation* fired more often on legitimate steps (43%) than on attacks (17%), because legitimate tasks held out from fitting produce unseen transitions.

The estimator gave these features positive weight. In the decision they therefore added to the probability of legitimate high-tier actions, and removing them recovered utility without a measured change in attack success. This is an observation about these features in this benchmark, with short trajectories. It does not show that behavioral evidence is useless elsewhere.

Calibration. Calibration did not change outcomes for two reasons:

- logistic regression fitted by penalized likelihood was already close to calibrated on in-distribution folds, so the fitted map was close to the identity;
- the one fold where calibration mattered, the unseen credential-change attack, was miscalibrated in a way a monotone map fitted on other attack categories cannot repair.

This agrees with the position, taken in Chapter 3, that calibration quality and control quality must be reported separately (C. Zhang et al., 2026).

4.14.4 What the evidence does not support

- **That REM protects live agents.** Under replay a withheld call does not reappear. A live agent can retry, rephrase or reach the goal through another tool. The near-zero replay ASR and the near-zero residual success after Modify are consequences of that assumption.
- **That REM is deployable as configured.** Without a reviewer, benign utility on state-changing tasks fell from 0.47 to 0.17 in replay.
- **That REM is superior to the baseline or to other defenses.**
    - The realized-loss comparison favors REM by construction.
    - The comparison with distributed defenses is not controlled.
    - Most of the ASR difference from the baseline comes from single-task attack categories.
- **Anything about the injection classifier**, which was not available, or about adaptive attackers, who were out of scope.

4.15 Comparison with Prior Work

The comparison restates design positions documented in Chapter 2 and does not rank systems, because no shared evaluation exists. Properties marked U in Chapter 2 are unverified.

Table 4.16. Design-level comparison with the closest prior work

System	Documented design (Chapter 2; verification level)	Relation to REM
NEXUS (Hossain et al., 2026)	Structured-plan monitor; formal policy over allow, block, confirm and revise; rules, argument inspection and a calibrated logistic risk score (A). A rule-first cascade with loss-selected thresholds (R, authors' repository README); the cost values (U)	Shares calibrated logistic scoring and a four-way intervention set. REM evaluates each call during execution, in an executable banking environment. The difference in decision rule rests on the repository description (R), not on the paper's full text
AgentTrust (C. Yang, 2026)	Pre-execution interception; allow, warn, block and review; normalizer, chain detection and LLM judge (A)	Shares interception and a review verdict. REM uses a calibrated probability and declared losses
SafeAgent (H. Liu et al., 2026)	Runtime controller; decision core with operators for risk, utility-cost, consequence and arbitration (A); LLM realization (U)	REM's verdict is a closed-form rule
ProvenanceGuard (She et al., 2026)	Checks whether a proposed call is supported by traceable evidence in context; peer-reviewed (A)	REM's argument-provenance feature asks the same question; in REM's evaluation such action evidence carried most of the separation
H.-H. Chen (2026)	Prices the consequence of side-effecting actions against reserve capital (A)	REM combines consequence with an estimated probability
FinHarness (Jia et al., 2026)	Financial-agent monitors and an LLM-judge cascade; results on a benchmark whose preprint was withdrawn for non-scientific reasons (A)	Shares domain and placement; REM is evaluated on an open benchmark
C. Zhang et al. (2026)	Calibration need not improve control (A)	REM's evaluation observed no calibration effect on decisions
Kanaker et al. (2026)	Simulation framework with a threat model separating compromise, harm and severity; Monte Carlo comparison of architectural controls across six scenarios and nine defense configurations (A; supplementary update of 2 November 2026, Chapter 2, Section 2.13.7)	Shares the separation of compromise from severity and reports a similar set of outcomes. It compares design-time controls in a simulator; REM evaluates a per-call runtime decision on recorded traces. Neither study's results transfer to the other

Records found in the currency update of 30 October 2026, examined at abstract level only, narrow the distance further (Chapter 2, Section 2.13.6): a controller that commits the

proposed or an alternative action by a calibrated value learned from outcomes, with provenance-typed evidence (Yan et al., 2026); a three-way execute–confirm–block router with calibrated probabilities (Xin, 2026); and an offline evaluation of runtime monitors on AgentDojo's recorded traces (Kekatos et al., 2026).

A simulation study examined in a supplementary update of 2 November 2026 (Kanaker et al., 2026; Chapter 2, Section 2.13.7; abstract level) relates to REM at a different level of the problem. It compares architectural controls in an assumption-explicit simulator, whereas REM selects a verdict for each proposed call and is evaluated on recorded traces. Three of its reported findings bear on the interpretation of REM's results without changing them:

- its finding that human-in-the-loop approvals degrade under user error and habituation is consistent with treating REM's idealized-reviewer results as an upper bound and the no-reviewer bound as the conservative reference (Section 4.14.2);
- its finding that rate limiting reduces tail severity more than attack success illustrates why REM reports a loss-weighted measure beside the attack success rate, although REM's realized loss favours REM by construction (Section 4.9);
- its finding that least-privilege tool design is the strongest single broad control concerns a design-time restriction of privileges, which REM does not evaluate; REM's restricted-tool set $\mathscr{T}_{\mathrm{R}}$ is used only by its mitigation semantics.

The two studies do not test each other's mechanisms. REM does not evaluate least privilege, retrieval allowlisting or rate limiting, and the simulation study does not, by its abstract, estimate a per-call probability or select among verdicts by expected loss. Its simulated results are not evidence about REM, and REM's results do not test its findings.

Each mechanism in REM has precedent. The distinguishing elements are the combination and the evaluation design; neither has been compared with these systems on a shared protocol.

4.16 Threats to Validity and Limitations

1. **Design status.** Costs, tiers, predicates, the feature set and the label rule are declared exploratory values, not approved ones.
2. **No live runs.** Enforcement was estimated only by replay under NO-ADAPT, a method proposed as an amendment to the Chapter 3 scope.
3. **One benchmark.** One benchmark, one attack family for fitting, one attacker identity, and historical model snapshots (2024–2025).

4. **Labels.** Labels are attempt-based and reference-matched. They are ambiguous for user_task_15 under early suite versions and miss alternative attacker routes. They have not been audited by a human.
5. **Statistics.** Nine attacked groups, one run per configuration, no correction for multiple comparisons. Several categories contain a single injection task.
6. **Reviewer.** Escalation outcomes are bounded by an oracle and by no reviewer.
7. **Missing component.** No neural injection classifier was available.
8. **Literature review.** The review was record-level; no full text was screened.
9. **Currency.** Work published after the review's search date was examined only at abstract level, and several close records were found (Chapter 2, Section 2.13.6).
10. **Runtime figures.** They describe the evaluated configuration only. A deployment with a neural classifier or a human reviewer would be slower.

4.17 Chapter Summary

This chapter reported an exploratory evaluation of REM on recorded traces of 21 LLM agents in the AgentDojo banking environment (3,334 episodes; 6,762 proposed tool calls), and discussed what it supports.

- **Estimation.** On held-out attacker objectives, the evidence available at the tool boundary separated attack attempts from legitimate calls (pooled ROC-AUC 0.938). Action evidence carried most of this separation; calibration produced no observed improvement; and the behavioral features were not specific to attacks in this benchmark (Sections 4.4 and 4.7).
- **Decisions.** With the declared exploratory costs, the expected-loss rule intervened on 97.3% of attack steps and 22.7% of legitimate steps, against 74.0% and 5.6% for the consequence-independent baseline on the same estimator (Section 4.5).
- **Replay.** Under the NO-ADAPT assumption, REM's attack success rate was 0.004 against 0.055 for the baseline (step-level semantics), at a cost of about 20 points of benign utility without a reviewer and no cost with an idealized reviewer (Section 4.6). The mitigation semantics change the baseline's results (Table 4.11).
- **Live agents.** The closed-loop evaluation was not executed (Section 4.12); no live result exists, and replay results are not live results (Section 4.13).
- **Interpretation and validity.** The security gain observed in replay comes mainly from low intervention thresholds at high consequence tiers and is paid for in legitimate financial actions; the evidence does not establish protection of live or adaptive agents, deployability, or superiority over other defenses (Sections 4.14–4.16).

CHAPTER FIVE: CONCLUSIONS AND RECOMMENDATIONS

5.1 Introduction

This chapter draws the conclusions of the thesis. Section 5.2 answers the research questions from the evidence of Chapter 4; Section 5.3 assesses the objectives; Section 5.4 separates the contributions achieved from those proposed; Section 5.5 summarises the limitations discussed in Section 4.16; Section 5.6 gives recommendations and future work ordered by dependency; and Section 5.7 concludes. Every conclusion is bounded by the exploratory status of the evidence: recorded traces, observe-only decisions and a replay that assumes the agent does not adapt. No live-agent result exists (Section 4.12).

5.2 Answers to the Research Questions

Table 5.1. Evidence summary

RQ	Kind of evidence	What was observed	Not shown
RQ 1	Held-out predictions	ROC-AUC 0.94 overall; 0.86–0.98 per held-out attacker objective; 0.92–0.96 on other templates; 0.74–1.00 per held-out backbone. ECE 0.044, degraded on the unseen credential-change category. No calibration improvement observed	Performance on other benchmarks or live agents
RQ 2	Observe-only decisions; replay	REM intervened on more attack steps than B1 and on many more legitimate state-changing steps. The difference stems mainly from tier-indexed costs, not from the escalation option. In replay, REM's ASR was lower than B1's under both mitigation semantics (mostly credential and payment attacks; Table 4.11), at a utility cost of −18 to −20 points without a reviewer and about 0 with an oracle reviewer	Enforcement against live or adaptive agents
RQ 3	Replay; runtime measurement	r_{mod} and residual ASR ≈ 0 under NO-ADAPT (assumption-driven); escalation 19.6% of steps; REM complete-call processing per step: median 0.21–0.28 ms, 95th percentile 0.49–0.70 ms in this configuration (R4); earlier values are partial	Residual success of live agents after Modify; full-system latency
RQ 4	Held-out predictions; observe-only; replay	Action evidence dominates. No improvement observed from the instruction-pattern indicator or from behavioral evidence, which lowered utility. Tiers drive the trade-off	General claims about context or behavioral evidence

5.3 Achievement of the Objectives

Table 5.2. Objectives and how far they were achieved

Objective	Achieved	Not achieved
O1. Design a REM framework for financial AI agents' and task automation agents' security	Designed (Chapter 3) and implemented (Section 3.16) for a financial (banking) agent; design values declared for exploration	Supervisor-approved design values; any task-automation setting; the injection classifier
O2. Evaluate the framework against adversarial attacks	Exploratory evaluation on recorded attacks against 21 real agents: offline classification, observe-only decisions, replay sensitivity (Chapter 4)	Enforcing-mode evaluation with live agents; repeated runs; human label audit; adaptive attacks
O3. Compare performance with existing security approaches	Controlled comparison with a same-estimator baseline (B1) and a same-escalation-opportunity variant; recorded outcomes of four distributed defenses for orientation	A controlled comparison with any published defense (NEXUS, AgentTrust, FinHarness or the distributed defenses) on a shared protocol

O1 is achieved for the banking setting. O2 and O3 are partly achieved: the evidence is exploratory, and the decisive experiment, enforcement with live agents, was not run.

5.4 Contributions

Achieved, with their evidence:

1. **A specified and implemented composition.** It comprises:
    - provenance-based evidence;
    - a ridge estimator calibrated on the logit;
    - consequence tiers and policy predicates;
    - a four-verdict expected-loss rule with a feasible set and a dominance check;
    - deterministic mitigation under two stated semantics;
    - an audit record whose attribution cannot reach the verdict.

    Undeclared design values stop the software. *Evidence:* Chapter 3; 205 tests.
2. **An evaluation protocol that isolates the decision rule.** REM, the baseline and the same-escalation variant share the estimator, calibration and predicates. Grouped cross-fitting holds out whole attacker objectives, and every fitted quantity, including the feature set, is fitted within the fold. *Evidence:* Chapter 4 and its exact reproduction.
3. **Exploratory evidence on recorded real-agent traces.** The evidence covers:

- discrimination on unseen attacker objectives;
- the observed absence of calibration and behavioral-feature benefit;
- a quantified protection–utility trade-off and its dependence on reviewer availability and on declared costs.

Evidence: exploratory; one benchmark; no live enforcement.

Proposed in Chapter 3 but not achieved: an evaluation of enforcement against live agents, which is necessary to support any claim that REM reduces attack success in operation. Until it is run, the central practical claim of the thesis, that tier-indexed expected-loss verdicts reduce financial harm at acceptable utility cost, remains **unestablished**.

Not claimed:

- a new learning, calibration or decision-theoretic method;
- prevention guarantees;
- robustness to adaptive attackers;
- operational readiness;
- superiority over the systems of Table 4.16.

5.5 Limitations

The limitations are discussed in full in Section 4.16. In summary, the conclusions of this thesis are bounded by:

1. **Design status.** Costs, tiers, predicates, the feature set and the label rule are declared exploratory values, not approved ones.
2. **No live evaluation.** Enforcement was estimated only by replay under NO-ADAPT. The closed-loop harness is implemented and tested but was not executed with a language model (Section 4.12).
3. **One benchmark and one attack family for fitting**, one attacker identity, and historical model snapshots.
4. **Labels** that are reference-matched attempt labels, ambiguous for one user task in early suite versions, and not audited by a human.
5. **Statistics**: nine attacked groups, one run per configuration, and no correction for multiple comparisons.
6. **Reviewer**: escalation outcomes are bounded by an idealized reviewer and by no reviewer; no real reviewer was modeled.

7. **Missing component**: no injection classifier was available for the Detection Layer.
8. **Literature**: the review was conducted at record level, without full-text screening.

5.6 Recommendations and Future Work, Ordered by Dependency

1. **Decisions** (`SUPERVISOR_DECISIONS_REQUIRED`): approve or replace the declared design values, the label rule, the mitigation semantics and the replay scope amendment.
2. **Label audit.** Annotate the prepared package (`results/pilot_R2/label_audit_human_package.csv`) with at least one human annotator, and preferably two. Report agreement and revise the label rule if needed.
3. **Live enforcing-mode evaluation**, required for the central claim:
    - run B0, B1, REM and REM without expected-loss Escalate with live backbones;
    - use at least three repeated runs;
    - fix and record the benchmark version, model versions, decoding parameters and costs.

    This measures adaptive agent behavior after refusals and the true residual success after Modify, and shows how the two mitigation semantics differ once the agent can react to refusals; in replay they already differ, because episode-level restrictions also refuse later, different calls (Chapter 4, Table 4.11).
4. **A reviewer model** with error rate and delay, so that the utility cost of escalation can be estimated between the two bounds.
5. **Feature revision.** Revise the behavioral features, for example by conditioning privilege escalation on provenance, and add the injection classifier, selected on the agent's own tool outputs at a low false-positive rate (Z. Liu, 2026). Then re-run Pilot E0 on new fitting data.
6. **Elicited costs.** Repeat the decision analysis with costs elicited from a financial-institution context, rather than declared.
7. **Full-text literature review.** Complete the review by searching the named databases and screening full texts, and re-examine the unverified properties of the closest systems.
8. **Further benchmarks and adaptive attacks**, reported per benchmark.

5.7 Conclusion

This thesis set out to design a runtime framework that decides, for each tool call proposed by a financial AI agent, whether to allow, modify, escalate or block it. The decision minimizes expected loss under a calibrated probability of adversarial inducement and a

declared consequence tier, and every verdict is recorded for audit. The framework was specified, implemented at the tool boundary of a benchmark agent, and evaluated in an exploratory study of 6,762 tool calls proposed by 21 real LLM agents.

What was observed:

- The evidence available at the tool boundary separates attack attempts from legitimate calls well on attacker objectives not seen in fitting, mainly through evidence about the action's arguments.
- With the costs declared for exploration, the expected-loss rule intervenes on nearly all attack attempts, and also on most legitimate high-consequence actions. Whether that is acceptable depends on a human reviewer that the benchmark does not provide.
- Calibration and the behavioral features produced no observed improvement.

These findings come from recorded traces and from a replay that assumes the agent does not adapt. They do not show that REM protects a live agent. The experiment needed to show that, enforcement with live agents, is the first item of future work, and the thesis does not claim its result in advance.

References

APA 7th edition. One consolidated list for all chapters. Preprints, working papers and withdrawn preprints are marked. Official and regulatory sources are marked [Official source] and are not treated as research literature. For arXiv-only records identified through the index, author lists abbreviated in the index record are marked as such. The verification status of every entry is recorded in `REFERENCES_VERIFIED.md`*.*

Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., & Davies, X. (2025). AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. https://arxiv.org/abs/2410.09024

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium* (pp. 3971–3988). USENIX Association.

Basseville, M., & Nikiforov, I. V. (1993). *Detection of abrupt changes: Theory and application*. Prentice Hall.

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Calboreanu, E. (2026). LATTICE: A governance-first architecture for authorized autonomous AI operations. *Frontiers in Artificial Intelligence, 9*, Article 1800407. https://doi.org/10.3389/frai.2026.1800407

Castro-Maldonado, V., Aceves-Fernández, M. A., García-Noguez, L. R., & Pedraza-Ortega, J. C. (2026). Semantic firewalls with online ensemble learning for secure agentic RAG systems in financial chatbots. *AI, 7*(3), Article 80. https://doi.org/10.3390/ai7030080

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Article 15, 1–58. https://doi.org/10.1145/1541880.1541882

Chandola, V., Banerjee, A., & Kumar, V. (2012). Anomaly detection for discrete sequences: A survey. *IEEE Transactions on Knowledge and Data Engineering*, *24*(5), 823–839. https://doi.org/10.1109/TKDE.2010.235

Chen, H.-H. (2026). *Insuring every action: An authority frontier framework for runtime actuarial control of autonomous AI agents* (arXiv:2605.25632) [Preprint]. arXiv. https://arxiv.org/abs/2605.25632

Chen, S., Piet, J., Sitawarin, C., & Wagner, D. (2025). StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*. USENIX Association.

Chen, Y., Li, H., Zheng, Z., Wu, D., Song, Y., & Hooi, B. (2025). Defense against prompt injection attack by leveraging attack techniques. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 18331–18347). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.897

Chen, Z., Chen, J., Chen, J., & Sra, M. (2025). *Standard benchmarks fail — Auditing LLM agents in finance must prioritize risk* (arXiv:2502.15865) [Preprint]. arXiv. https://arxiv.org/abs/2502.15865

Chen, Z., Dong, D., Liu, H., Li, J., Zhai, J., Xu, D., & Pu, B. (2026). *Cordon: Semantic transactions for tool-using LLM agents* (arXiv:2606.17573) [Preprint]. arXiv. https://arxiv.org/abs/2606.17573

Chen, Z., Kang, M., & Li, B. (2025). ShieldAgent: Shielding agents via verifiable safety policy reasoning. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267, pp. 8313–8344). PMLR.

Chen, Z., Xiang, Z., Xiao, C., Song, D., & Li, B. (2024). AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Chennabasappa, S., Nikolaidis, C., Song, D., Molnar, D., Ding, S., Wan, S., Whitman, S., Deason, L., Doucette, N., Montilla, A., Gampa, A., de Paola, B., Gabi, D., Crnkovich, J., Testud, J.-C., He, K., Chaturvedi, R., Zhou, W., & Saxe, J. (2025). *LlamaFirewall: An*

open source guardrail system for building secure AI agents (arXiv:2505.03574) [Preprint]. arXiv. https://arxiv.org/abs/2505.03574

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory, 16*(1), 41–46. https://doi.org/10.1109/TIT.1970.1054406

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological), 20*(2), 215–232. https://doi.org/10.1111/j.2517-6161.1958.tb00292.x

Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240). ACM. https://doi.org/10.1145/1143844.1143874

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. (2025). *Defeating prompt injections by design* (arXiv:2503.18813v2) [Preprint]. arXiv. https://arxiv.org/abs/2503.18813

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*.

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics, 7*(1). https://doi.org/10.1214/aos/1176344552

Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)* (pp. 973–978).

European Parliament & Council of the European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union. https://eur-lex.europa.eu/eli/reg/2024/1689/oj [Official source]

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A sense of self for Unix processes. In *Proceedings of the 1996 IEEE Symposium on Security and Privacy* (pp. 120–128). IEEE. https://doi.org/10.1109/SECPRI.1996.502675

Fu, X., Li, S., Wang, Z., et al. (2024). *Imprompter: Tricking LLM agents into improper tool use* (arXiv:2410.14923) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2410.14923 [Author list abbreviated from the index record; full list not verified]

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 70, pp. 1321–1330). PMLR.

Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., & Hassan, A. E. (2026). Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. *ACM Transactions on Software Engineering and Methodology*. https://doi.org/10.1145/3814959

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., & Kiciman, E. (2024). *Defending against indirect prompt injection attacks with spotlighting* (arXiv:2403.14720) [Preprint]. arXiv. https://arxiv.org/abs/2403.14720

Hossain, E., Nipu, M. M. H., Ornee, T. N., Rana, R., & Yousefi, N. (2026). *NEXUS: Structured runtime safety for tool-using LLM agents* (arXiv:2607.19356) [Preprint]. arXiv. https://arxiv.org/abs/2607.19356

Hua, W., Yang, X., Jin, M., Li, Z., Cheng, W., Tang, R., & Zhang, Y. (2024). TrustAgent: Towards safe and trustworthy LLM-based agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 10000–10016). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.585

Huang, C., Huang, X., Tran, N. P., & Milani Fard, A. (2026). Model Context Protocol threat modeling and analysis of vulnerabilities to prompt injection with tool poisoning. *Journal of Cybersecurity and Privacy, 6*(3), Article 84. https://doi.org/10.3390/jcp6030084

Jackson, F. (2025). *Designing a policy engine for agentic AI systems: From governance requirements to runtime enforcement* [Working paper]. SSRN. https://doi.org/10.2139/ssrn.5904104

Jacob, D., Alzahrani, H., Hu, Z., Alomair, B., & Wagner, D. (2025). PromptShield: Deployable detection for prompt injection attacks. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy* (pp. 341–352). ACM. https://doi.org/10.1145/3714393.3726501 [Crossref records issued year 2024 and online publication 2025]

Jia, F., Wu, T., Qin, X., & Squicciarini, A. (2025). The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics. https://aclanthology.org/2025.acl-long.1435/

Jia, H., Liu, Y., Chong, B., Yang, Y., Chen, Y., Liang, J., Li, Q., Lu, H., Xu, K., Zheng, H., Zhang, C., Peng, H., & Yu, P. S. (2026). *FinHarness: An inline lifecycle safety harness for finance LLM agents* (arXiv:2605.27333) [Preprint]. arXiv. https://arxiv.org/abs/2605.27333

Jia, X., Liao, J., Qin, S., et al. (2026). *SkillJect: Effectively automating skill-based prompt injection for skill-enabled agents* (arXiv:2602.14211) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2602.14211 [Author list abbreviated from the index record; full list not verified]

Jiang, C., Zhang, W., Pan, X., Hong, G., & Yang, M. (2025). *Think twice before you act: Enhancing agent behavioral safety with thought correction* (arXiv:2505.11063) [Preprint; accepted to ICML 2026]. arXiv. https://arxiv.org/abs/2505.11063

Johnson, S., Pham, V., & Le, T. (2025). The dangers of indirect prompt injection attacks on LLM-based autonomous web navigation agents: A demonstration. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System*

Demonstrations (pp. 729–738). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-demos.55

Kanaker, H., Fakhouri, H., Abdel Karim, N., Abuhamdeh, M., Ismail, N. H. A., & Fakhouri, S. (2026). Securing tool-using AI agents against injection and authority misuse. *Computation, 14*(5), Article 98. https://doi.org/10.3390/computation14050098

Kaptein, M., Khan, V.-J., & Podstavnychy, A. (2026). *Runtime governance for AI agents: Policies on paths* (arXiv:2603.16586) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2603.16586

Kekatos, N., Basagiannis, S., Chintri, M., Lekidis, A., Nianios, T., Seitoglou, I., Temperekidis, A., & Katsaros, P. (2026). Formal runtime verification for tool-using LLM agents: An offline same-benchmark study on AgentDojo and STAC. In *Proceedings of the 2026 8th Joint Workshop on CPS&IoT Security and Privacy* (pp. 129–142). ACM. https://doi.org/10.1145/3847353.3847499

Kull, M., Silva Filho, T., & Flach, P. (2017). Beta calibration: A well-founded and easily implemented improvement on logistic calibration for binary classifiers. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (Proceedings of Machine Learning Research, Vol. 54, pp. 623–631). PMLR.

le Cessie, S., & van Houwelingen, J. C. (1992). Ridge estimators in logistic regression. *Applied Statistics, 41*(1), 191–201. https://doi.org/10.2307/2347628

Li, E., Mallick, T., Rose, E., Robertson, W., Oprea, A., & Nita-Rotaru, C. (2026). ACE: A security architecture for LLM-integrated app systems. In *Proceedings of the 2026 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2026.230352

Li, F. (2026). *OpenClaw PRISM: A zero-fork, defense-in-depth runtime security layer for tool-augmented LLM agents* (arXiv:2603.11853) [Preprint]. arXiv. https://arxiv.org/abs/2603.11853

Li, H., Liu, X., Chiu, C., Li, D., Zhang, N., & Xiao, C. (2025). DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)* (pp. 92825–92853). https://doi.org/10.52202/085713-2791

Li, M. Q., Fung, B. C. M., Li, B., Ismail, H., & Iqbal, F. (2026). *Taxonomy and consistency analysis of safety benchmarks for AI agents* (arXiv:2605.16282) [Preprint]. arXiv. https://arxiv.org/abs/2605.16282

Lin, J., Zhou, Z., Zheng, Z., Liu, S., Xu, T., Chen, Y., & Chen, E. (2026). VIGIL: Defending LLM agents against tool-stream injection via verify-before-commit. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 9764–9785). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.acl-long.443

Lin, W., Yu, C., Lin, X., Cao, S., Chen, X., Xue, L., Yu, L., Sha, L., & Wu, C. (2026). *DreamGuard: Efficient runtime guardrail for LLM agents via risk-aware world model* (arXiv:2608.05695) [Preprint]. arXiv. https://arxiv.org/abs/2608.05695

Liu, H., Ilyushin, E., Ni, J., & Zhu, M. (2026). *SafeAgent: A runtime protection architecture for agentic systems* (arXiv:2604.17562) [Preprint]. arXiv. https://arxiv.org/abs/2604.17562

Liu, T., Deng, Z., Meng, G., et al. (2023). *Demystifying RCE vulnerabilities in LLM-integrated apps* (arXiv:2309.02926) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2309.02926 [Author list abbreviated from the index record; full list not verified]

Liu, Y., Deng, G., Li, Y., et al. (2023). *Prompt injection attack against LLM-integrated applications* (arXiv:2306.05499) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2306.05499 [Author list abbreviated from the index record; full list not verified]

Liu, Y., Jia, Y., Geng, R., Jia, J., & Gong, N. Z. (2024). Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium* (pp. 1831–1847). USENIX Association.

Liu, Z. (2026). *Passing the test you trained on: Re-evaluating prompt-injection detectors for LLM agents* (arXiv:2610.03448) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2610.03448

Lorden, G. (1971). Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics, 42*(6), 1897–1908. https://doi.org/10.1214/aoms/1177693055

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)* (pp. 4765–4774).

Luo, W., Dai, S., Liu, X., Banerjee, S., Sun, H., Chen, M., & Xiao, C. (2025). AGrail: A lifelong agent guardrail with effective and adaptive safety detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 8104–8139). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.399

Mao, Q., Wang, J., Liu, Y., Zhu, L., Ma, C., & Yan, J. (2026). *SoK: Security of autonomous LLM agents in agentic commerce* (arXiv:2604.15367) [Preprint]. arXiv. https://arxiv.org/abs/2604.15367

Mou, Y., Xue, Z., Li, L., Liu, P., Zhang, S., Ye, W., & Shao, J. (2026). ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. In *Findings of the Association for Computational Linguistics: ACL 2026* (pp. 37125–37153). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.findings-acl.1850

Moustakides, G. V. (1986). Optimal stopping times for detecting changes in distributions. *The Annals of Statistics, 14*(4). https://doi.org/10.1214/aos/1176350164

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence, 29*(1). https://doi.org/10.1609/aaai.v29i1.9602

Ng, A. Y., & Jordan, M. I. (2002). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems 14 (NIPS 2001)*. https://papers.nips.cc/paper/2020-on-discriminative-vs-generative-classifiers-a-comparison-of-logistic-regression-and-naive-bayes

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

OWASP Gen AI Security Project. (2025). *LLM01:2025 Prompt injection*. OWASP Foundation. https://genai.owasp.org/llmrisk/llm01-prompt-injection/ [Official source]

Page, E. S. (1954). Continuous inspection schemes. *Biometrika, 41*(1–2), 100–115. https://doi.org/10.1093/biomet/41.1-2.100

Patil, S. G., Zhang, T., Fang, V., et al. (2024). *GoEX: Perspectives and designs towards a runtime for autonomous LLM applications* (arXiv:2404.06921) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2404.06921 [Author list abbreviated from the index record; full list not verified]

Pedro, R., Amaral de Castro, D., Carreira, P., et al. (2023). *From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?* (arXiv:2308.01990) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2308.01990 [Author list abbreviated from the index record; full list not verified]

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Platt, J. C. (2000). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large-margin classifiers* (pp. 61–74). MIT Press. https://doi.org/10.7551/mitpress/1113.003.0008 [Crossref registers this chapter, pp. 61–74, under the title “Probabilities for SV Machines”; the printed chapter title was not checked]

Ray, S. (2026). *What can be enforced? A theory of certified runtime safety for tool-using agents* (arXiv:2607.22868) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2607.22868

Reynolds, M. R., Jr., & Stoumbos, Z. G. (1999). A CUSUM chart for monitoring a proportion when inspecting continuously. *Journal of Quality Technology*, *31*(1), 87–108. https://doi.org/10.1080/00224065.1999.11979900

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., & Hashimoto, T. (2024). Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Schneider, F. B. (2000). Enforceable security policies. *ACM Transactions on Information and System Security, 3*(1), 30–50. https://doi.org/10.1145/353323.353382

She, Y., Liang, Y., & Kang, E. (2026). Safeguarding LLM agents from misalignment through provenance analysis. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering* (pp. 3036–3048). ACM. https://doi.org/10.1145/3832783.3837535 (earlier preprint: arXiv:2607.01236)

Shi, J., Yuan, Z., Tie, G., Zhou, P., Gong, N., & Sun, L. (2026). Prompt injection attack to tool selection in LLM agents. In *Proceedings of the 2026 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2026.230675

Shi, T., He, J., Wang, Z., Wu, L., Li, H., Guo, W., & Song, D. (2025). *Progent: Securing AI agents with privilege control* (arXiv:2504.11703) [Preprint]. arXiv. https://arxiv.org/abs/2504.11703

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy* (pp. 305–316). IEEE. https://doi.org/10.1109/SP.2010.25

Storf, S., Barton-Cooper, R., Peters-Gill, J., et al. (2026). *Constitutional black-box monitoring for scheming in LLM agents* (arXiv:2603.00829) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2603.00829 [Author list abbreviated from the index record; full list not verified]

Tsai, L., & Bagdasarian, E. (2025). Contextual agent security: A policy for every purpose. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '25)* (pp. 8–17). ACM. https://doi.org/10.1145/3713082.3730378

Wang, C. L., Singhal, T., Kelkar, A., & Tuo, J. (2025). *MI9: An integrated runtime governance framework for agentic AI* (arXiv:2508.03858) [Preprint]. arXiv. https://arxiv.org/abs/2508.03858

Wang, H., Poskitt, C. M., & Sun, J. (2026). AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*. ACM. https://doi.org/10.1145/3744916.3764546

Wang, H., Poskitt, C. M., Wei, J., & Sun, J. (2025). *ProbGuard: Probabilistic runtime monitoring for LLM agent safety* (arXiv:2508.00500; earlier versions titled “Pro2Guard”) [Preprint]. arXiv. https://arxiv.org/abs/2508.00500

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

Wang, P., Li, Y., & Tian, Y. (2026). *Reframing LLM agent security as an agent-human interaction problem* (arXiv:2605.24309) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2605.24309

Wang, S., Zhu, S., & Li, R. (2026). Runtime policy enforcement for MCP-based LLM agents. *Electronics, 15*(13), Article 2829. https://doi.org/10.3390/electronics15132829

Wang, Y., Han, X., Shang, D., Tang, Y., & Liu, B. (2026). *Safety, or just capability? A validity audit of agent-safety benchmarks* (arXiv:2607.28685) [Preprint]. arXiv. https://arxiv.org/abs/2607.28685

Wang, Z., Siu, V., Ye, Z., Shi, T., Nie, Y., Zhao, X., Wang, C., Guo, W., & Song, D. (2025). AGENTVIGIL: Automatic black-box red-teaming for indirect prompt injection against LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025* (pp. 23159–23172). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-emnlp.1258

Wei, B., Zhang, Y., Pan, J., et al. (2026). *ClawSafety: “Safe” LLMs, unsafe agents* (arXiv:2604.01438) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2604.01438 [Author list abbreviated from the index record; full list not verified]

Wen, T., Wang, C., Yang, X., Tang, H., Xie, Y., Lyu, L., Dou, Z., & Wu, F. (2025). Defending against indirect prompt injection by instruction detection. In *Findings of the Association for Computational Linguistics: EMNLP 2025* (pp. 19472–19487). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-emnlp.1060

Wu, F., Cecchetti, E., & Xiao, C. (2024). *System-level defense against indirect prompt injection attacks: An information flow control perspective* (arXiv:2409.19091) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2409.19091

Wu, Y., Roesner, F., Kohno, T., Zhang, N., & Iqbal, U. (2025). IsolateGPT: An execution isolation architecture for LLM-based systems. In *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2025.241131

Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., Song, D., & Li, B. (2025). GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267, pp. 68316–68342). PMLR.

Xin, Q. (2026). TapGuard: Temperature-calibrated execute–confirm–block routing for LLM-powered mobile agents. *International Journal of Mobile Applications and Technologies, 2*(2). https://doi.org/10.51137/wrp.ijmat.1051

Yan, B., Zhang, Q., Kan, Z., et al. (2026). *PROVISO: Provenance-informed action selection for secure LLM agents* (arXiv:2610.27115) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2610.27115 [Author list abbreviated from the index record; full list not verified]

Yang, C. (2026). *AgentTrust: Runtime safety evaluation and interception for AI agent tool use* (arXiv:2605.04785) [Preprint]. arXiv. https://arxiv.org/abs/2605.04785

Yang, Z., Li, R., Qiang, Q., Wang, J., Lou, F., Li, M., Cheng, D., Xu, R., Lian, H., Zhang, S., Liang, X., Huang, X., Wei, Z., Liu, Z., Guo, X., Wang, H., Chen, R., & Zhang, L. (2026). *FinVault: Benchmarking financial agent safety in execution-grounded environments* (arXiv:2601.07853) [Withdrawn preprint]. arXiv. https://arxiv.org/abs/2601.07853

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*.

Yu, M., Meng, F., Zhou, X., Wang, S., Mao, J., Pan, L., Chen, T., Wang, K., Li, X., Zhang, Y., An, B., & Wen, Q. (2025). A survey on trustworthy LLM agents: Threats and countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (pp. 6216–6226). ACM. https://doi.org/10.1145/3711896.3736561

Yuan, T., He, Z., Dong, L., Wang, Y., Zhao, R., Xia, T., Xu, L., Zhou, B., Li, F., Zhang, Z., Wang, R., & Liu, G. (2024). R-Judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 1467–1490). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.79

Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 694–699). ACM. https://doi.org/10.1145/775047.775151

Zhan, Q., Fang, R., Panchal, H. S., & Kang, D. (2025). Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025* (pp. 7116–7132). Association for Computational Linguistics. https://aclanthology.org/2025.findings-naacl.395/

Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 10471–10506). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.624

Zhang, B., Tan, Y., Shen, Y., Salem, A., Backes, M., Zannettou, S., & Zhang, Y. (2025). Breaking agents: Compromising autonomous LLM agents through malfunction amplification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 34952–34964). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-main.1771

Zhang, C., Wan, Z., Yu, X., Wu, J., Wen, Q., Zhou, P., Zhao, W., & Tsang, I. (2026). *Calibration is not control: Intervention value for LLM-agent oversight* (arXiv:2606.21399) [Preprint]. arXiv. https://arxiv.org/abs/2606.21399 (title as indexed on 28 Oct 2026; an earlier version was titled “Calibration is not control: Why LLM-agent oversight needs intervention”)

Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., & Zhang, Y. (2025). Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*.

Zhang, Y., Chen, K., Gao, J., et al. (2024). *Towards action hijacking of large language model-based agent* (arXiv:2412.10807) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2412.10807 [Author list abbreviated from the index record; full list not verified]

Zhong, P. Y., Chen, S., Wang, R., et al. (2025). *RTBAS: Defending LLM agents against prompt injection and privacy leakage* (arXiv:2502.08966) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2502.08966 [Author list abbreviated from the index record; full list not verified]

Zhu, K., Yang, X., Wang, J., Guo, W., & Wang, W. Y. (2025). MELON: Provable defense against indirect prompt injection attacks in AI agents. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267). PMLR.

Zhu, K., Zhang, J., Qi, Z., et al. (2025). *SafeScientist: Toward risk-aware scientific discoveries by LLM agents* (arXiv:2505.23559) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2505.23559 [Author list abbreviated from the index record; full list not verified]